

CORE COMPUTER SCIENCE · VOLUME II

Everything About Stacks

A working handbook for the LIFO discipline — the array, the call frame, and the monotonic invariant.

73 topics · 10 parts · C++17

Practice sets, worked answer keys and one-page revision cards throughout

BUILD IT · BREAK IT · OBSERVE THE FAILURE · FIX IT

Contents

PART ONE

Foundation	7
1. What LIFO forbids — the ADT contract	8
2. The five operations and their contracts	9
3. Stack vs array vs linked list	10
4. The two things called “stack”	12
5. Underflow and overflow — four ways to signal	13
6. When a stack is the wrong choice	14
Practice set — Topics 1-6	15
Answer key — Topics 1-6	17
Handnote summary card	20

PART TWO

Memory and representation	21
7. Array-backed layout and the top-index invariant	22
8. Growth policy: $2\times$ vs $1.5\times$	23
9. Amortised $O(1)$ push by the accounting method	24
10. Shrink policy and the thrash trap	25
11. The linked-node stack and its overhead	26
12. Cache behaviour, measured	27
13. Fixed-capacity stack with no heap	29
Practice set — Topics 7-13	31
Answer key — Topics 7-13	33
Handnote summary card	36

PART THREE

Core operations	37
14. push	38
15. pop and the value-return problem	39
16. top and reference invalidation	40
17. size, empty, and the unsigned-underflow trap	41
18. clear, swap, and destructor cost	42
19. Making it generic	43
20. Rule of five for the array stack	45
21. Testing: invariants, fuzzing, instrumentation	46
Practice set — Topics 14-21	48
Answer key — Topics 14-21	50
Handnote summary card	53

PART FOUR

The call stack	55
22. Frame anatomy	56
23. Reading a frame in disassembly	57

24. Recursion is a stack you did not write	58
25. Recursion to explicit stack: the recipe	59
26. The resumption-point problem	60
27. Tail calls and why they do not save you	62
28. Stack overflow: guard page, ulimit, symptom	63
29. Depth arithmetic and the recursive destructor crash	64
Practice set — Topics 22-29	66
Answer key — Topics 22-29	68
Handnote summary card	71

PART FIVE

Expression parsing	73
30. Infix, prefix, postfix	74
31. Postfix evaluation	75
32. Prefix evaluation	76
33. Shunting-yard	77
34. Precedence, associativity, unary minus	78
35. Two-stack direct infix evaluation	79
36. Expression tree from postfix	81
37. A working calculator	82
Practice set — Topics 30-37	84
Answer key — Topics 30-37	86
Handnote summary card	89

PART SIX

Matching and nesting	91
38. Balanced brackets	92
39. Longest valid parentheses	93
40. Minimum insertions and score of parentheses	94
41. Path canonicalisation	95
42. Decode string	96
43. Recursive descent and deep-nesting DoS	98
Practice set — Topics 38-43	100
Answer key — Topics 38-43	102
Handnote summary card	105

PART SEVEN

The monotonic stack	107
44. The invariant and what “unresolved” means	108
45. Next greater element and the four-way table	109
46. Duplicates: > vs >=	111
47. Daily temperatures	112
48. Circular arrays	113
49. Stock span	114
50. Largest rectangle in histogram	115
51. Maximal rectangle in a binary matrix	117
52. Trapping rain water: stack vs two pointers	117

53. Sum of subarray minimums	119
54. Remove K digits	120
55. Recognising a monotonic-stack problem	121
Practice set — Topics 44-55	124
Answer key — Topics 44-55	126
Handnote summary card	129

PART EIGHT

Augmented designs	131
56. Min stack: pairs and the auxiliary stack	132
57. Min stack: the encoding trick	133
58. Queue from two stacks	135
59. Stack from two queues	136
60. Undo and redo	137
61. Sorting a stack with a stack	139
Practice set — Topics 56-61	141
Answer key — Topics 56-61	143
Handnote summary card	146

PART NINE

Stacks inside algorithms	147
62. Iterative DFS and visit order	148
63. Iterative inorder traversal	149
64. Iterative preorder and postorder	150
65. Iterative quicksort with bounded depth	152
66. Topological sort and Tarjan's stack	154
Practice set — Topics 62-66	156
Answer key — Topics 62-66	158
Handnote summary card	161

PART TEN

Library, safety, interview	163
67. std::stack: deque vs vector, benchmarked	164
68. The other languages: Python, Java, PHP, JS	165
69. Concurrency: the Treiber stack and ABA	166
70. alloca, VLAs, and stack smashing	168
71. Debugging: gdb backtrace and ASan output	169
72. Stack machines and Python bytecode	171
73. The strongest case against stacks	172
Practice set — Topics 67-73	174
Answer key — Topics 67-73	177
Handnote summary card	181

END MATTER

Master cheatsheet — all 73 topics	184
Pattern recognition: phrasing → technique	187

Revision tracker (Day 0 / +1 / +7 / +30 / +90)	189
How to use this book	191

PART ONE

Foundation

What the LIFO restriction actually forbids, why forbidding things buys guarantees, and the six pages of vocabulary that make the rest of the book readable. Ends with the argument against stacks — read it early, not late.

1. What LIFO forbids — the ADT contract

A stack is an array with two thirds of its interface deleted. You may add at one end, remove from that same end, and look at that end. You may not index, you may not insert in the middle, you may not iterate. Everything interesting about stacks follows from that deletion, not from anything the structure adds.

The word for the surviving discipline is **LIFO** — last in, first out. If you push **A**, **B**, **C**, the only element you can legally touch is **C**. To reach **A** you must destroy **B** and **C** first. That sounds like pure loss. It is not, and the reason is the whole subject: *a restricted interface is a promise*, and code written against a promise can be simpler and faster than code written against a general container.

The promise, stated precisely

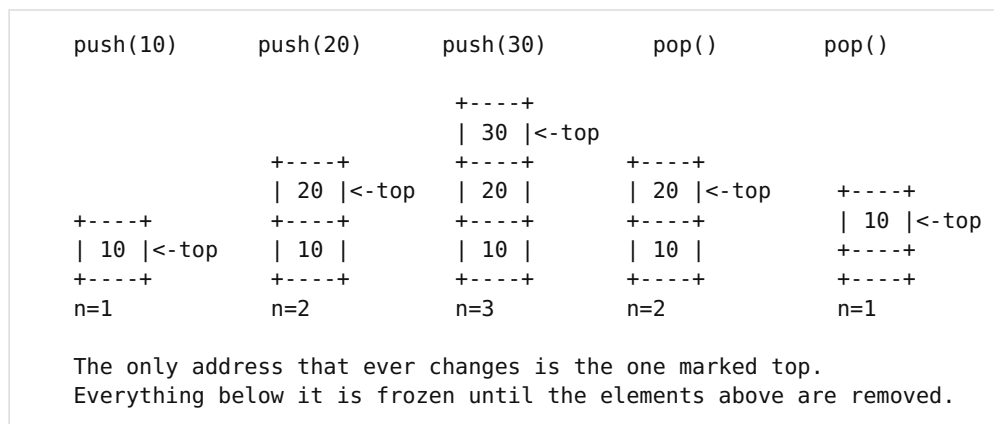
At any moment the stack holds a sequence $s = [x_1, x_2, \dots, x_n]$ with x_n the top. Then:

```
push(v)  : s -> [x1, ..., xn, v]      top becomes v
pop()    : s -> [x1, ..., x(n-1)]    requires n >= 1
top()    : returns xn                requires n >= 1, s unchanged
empty()  : returns (n == 0)
size()   : returns n
```

Two algebraic laws fall straight out, and interviewers do ask for them:

```
top(push(s, v)) == v      // push then peek gives back what you pushed
pop(push(s, v)) == s      // push then pop is the identity
```

The second law is the one that matters. It says a stack is a perfect undo device: any sequence of pushes can be exactly reversed by the same number of pops. That single property is why stacks appear in undo buffers (Topic 60), in expression parsing (Part 5), in backtracking (Topic 62), and in the hardware call mechanism itself (Topic 22). They are all the same idea wearing different hats.



THE KEY INSIGHT

The stack does not remember an order — it *enforces* a reversal. Whenever a problem contains a pairing between an earlier thing and a later thing where the innermost pair must resolve first (open/close brackets, call/return, operator/operand, do/undo), a stack is not one possible tool; it is the shape of the problem.

THE TRAP

Calling something a stack because it is backed by a vector. The symptom is a class named `Stack` that exposes `at(i)` or a public iterator “just for debugging”, followed six months later by a caller that iterates it in a hot loop and a maintainer who cannot change the backing store because four call sites depend on contiguity. If the interface is not restricted, none of the guarantees in this book apply. See Topic 22 for what the restriction buys at the hardware level.

stack ADT – push/pop/top $O(1)$ worst case (fixed capacity) or $O(1)$ amortised (growable) · Space $O(n)$ · the interface is the product

2. The five operations and their contracts

Five operations, and the entire design space of stacks lives in what each one does when its precondition is violated. Write them out as a contract table before you write any code; the table is the specification.

Operation	Precondition	Postcondition	Cost
<code>push(v)</code>	capacity available, or growable	<code>size+1</code> ; <code>top()==v</code>	$O(1)$ amortised
<code>pop()</code>	<code>size > 0</code>	<code>size-1</code> ; previous top destroyed	$O(1)$
<code>top()</code>	<code>size > 0</code>	unchanged; returns reference	$O(1)$
<code>empty()</code>	none	unchanged	$O(1)$
<code>size()</code>	none	unchanged	$O(1)$

Notice the asymmetry that catches people out. `empty()` and `size()` are total — they are defined on every stack including the empty one. `pop()` and `top()` are partial. A stack implementation is therefore a choice about *what partial functions do at the boundary*, and that choice is Topic 5.

The minimal implementation, in full

```
#include <stddef>
#include <stdexcept>

class IntStack {
    // fixed capacity, no allocation
    static constexpr std::size_t CAP = 64;
    int buf_[CAP];
    std::size_t n_ = 0;
    // invariant: 0 <= n_ <= CAP
    // buf_[0 .. n_-1] hold live values

public:
    bool empty() const { return n_ == 0; }
    std::size_t size() const { return n_; }

    void push(int v) {
        if (n_ == CAP) throw std::length_error("stack full");
        buf_[n_++] = v;
        // write, then advance
    }
    void pop() {
        if (n_ == 0) throw std::underflow_error("stack empty");
        --n_;
        // retreat; the value is simply abandoned
    }
    int& top() {
        if (n_ == 0) throw std::underflow_error("stack empty");
        return buf_[n_ - 1];
    }
};
```

That is a complete, correct stack in twenty lines. Every remaining topic in Parts 2 and 3 is one of three refinements: remove the fixed capacity (Topics 7–10), make it hold any type (Topic 19), or make it fast (Topic 12).

THE KEY INSIGHT

`push` writes then increments; `pop` only decrements. `pop` does not erase the slot, because nothing can read it: the invariant says only `buf_[0..n_-1]` is live. That asymmetry — one line to remove an element — is why stack pop is the cheapest removal operation in all of data structures.

THE TRAP

`buf_[n_++] = v;` versus `buf_[++n_] = v;`. The second writes to index 1 on the first push, leaves index 0 permanently garbage, and overruns the array by one on the last push. In a release build the symptom is not a crash but a corrupted neighbouring variable and an off-by-one in `size()`; under `-fsanitize=address` you get `stack-buffer-overflow` pointing at the write. Practice set 3 makes you produce that output on purpose.

fixed-capacity IntStack – push/pop/top O(1) worst case · Space 4·CAP + 8 bytes · zero allocations, zero pointer chasing

3. Stack vs array vs linked list

Volume I of this series was about linked lists, so start from the honest comparison. A stack is not a competitor to arrays and lists — it is a *discipline you impose on one of them*. The real question is which substrate to impose it on.

Property	Array-backed stack	Node-backed stack	Raw dynamic array
push cost	O(1) amortised, ~1 ns	O(1) worst case, ~20 ns (malloc)	O(1) amortised
pop cost	O(1), one decrement	O(1) + free, ~18 ns	O(1) at the back
Bytes per int	4 (amortised up to 8)	16 — see Topic 11	4-8
Worst-case push	O(n) on a realloc	O(1) always	O(n) on a realloc
Cache behaviour	sequential, prefetchable	pointer chase, ~1 miss/pop	sequential
Random access	forbidden by interface	forbidden by interface	allowed — and that is the problem

Three numbers to carry: a node-backed push costs roughly **20×** an array-backed push because it calls the allocator; a node-backed stack costs **4×** the memory for 32-bit payloads; and an array-backed stack has one worst-case push in every n that costs $O(n)$, which a node-backed stack never has. If you are writing a hard-real-time interrupt handler, that last row is the one that decides, and it is the only common case where nodes win. Everywhere else, use the array.

ARRAY-BACKED one allocation, contiguous	NODE-BACKED one allocation per element
<pre> base v +---+---+---+---+---+ 10 20 30 +---+---+---+---+---+ 0 1 2 ^ n_=3 next write 20 bytes total, 12 in use </pre>	<pre> top v +---+---+ +---+---+ +---+---+ 30 *---> 20 *---> 10 0 +---+---+ +---+---+ +---+---+ 16 bytes 16 bytes 16 bytes (4 payload + 4 pad + 8 ptr) 48 bytes total, 12 in use </pre>

THE KEY INSIGHT

The linked list wins on *worst-case* push; the array wins on *average* push, memory, and cache. Interviewers who ask “array or linked list for a stack?” are testing whether you know that both are $O(1)$ and that the answer is decided by constants, not by complexity class. Say “array, unless I need a hard bound on a single push.”

THE TRAP

Reaching for `std::list` because “a stack is a linked list.” `std::list` is doubly linked: 24 bytes of overhead per node, two pointer writes per push, and it can never be prefetched. It is the slowest legal way to build a stack in C++ — roughly 8-15× slower than `std::vector` backing on a push/pop microbenchmark. If you truly need nodes, hand-roll a singly linked one (Topic 11).

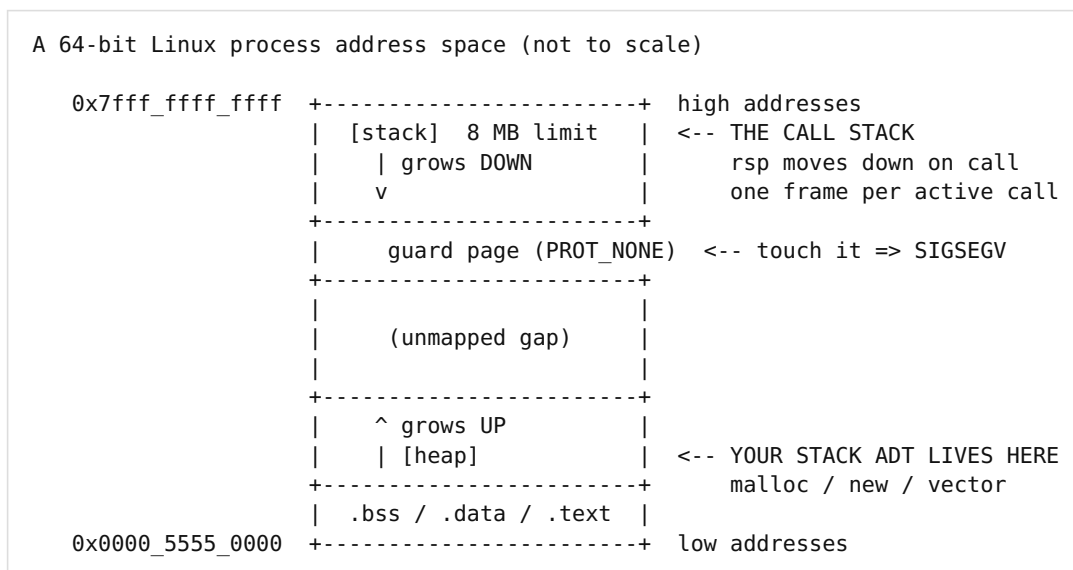
choice of substrate — array: $O(1)$ amortised, 4 B/int, cache-friendly · nodes: $O(1)$ worst case, 16 B/int, one cache miss per pop · decided by constants

4. The two things called “stack”

This is the single largest source of confusion for people new to the topic, and it is purely a vocabulary problem. The word names two different things:

- **The stack ADT** — the container you write, the subject of Parts 2, 3 and 5–9.
- **The call stack** — a region of memory the operating system hands your thread, managed by the CPU's stack pointer register, which stores function frames. Subject of Part 4.

They share a name because they share exactly one property: LIFO discipline. They share nothing else. The ADT is a heap allocation you control; the call stack is a fixed-size mapping you do not control, typically 8 MB on Linux, that grows *downward* and kills your process when exhausted.



Both directions of confusion cause real bugs. Assuming your `std::stack` is limited to 8 MB leads to pointless capacity checks; assuming recursion is as cheap as pushing to a vector leads to Topic 28.

THE KEY INSIGHT

Converting recursion into an explicit stack (Topic 25) is precisely the act of moving data from the region on the left of this diagram to the region on the right — from an 8 MB fixed mapping you cannot grow into a heap that is limited only by RAM. That is the entire reason the conversion is worth doing, and the one-sentence answer when an interviewer asks why you would bother.

THE TRAP

Believing the call stack is “faster memory.” It is ordinary DRAM; it is fast because the top of it is almost always already in L1, not because of any special hardware. Allocating a 4 MB array as a local variable to “avoid malloc” consumes half your thread's entire stack in one statement and crashes on the second nested call. Symptom: a segfault whose backtrace looks perfectly innocent. See Topic 70.

terminology – ADT: heap, size limited by RAM, you manage it · call stack: 8 MB default (`ulimit -s 8192`), managed by `rsp`, overflow is fatal

5. Underflow and overflow — four ways to signal

A stack has exactly two error states. **Underflow**: `pop` or `top` on an empty stack. **Overflow**: `push` beyond a fixed capacity, or a failed allocation on a growable one. There are no others — no invalid index, no iterator invalidation from the caller's side, no ordering violations. Two error states is another dividend of the restricted interface.

Four strategies exist for signalling them, and every standard library picks a different one. Knowing which is which is a common interview question.

Strategy	Signature	Who does this	Cost of a check
Undefined behaviour	<code>void pop();</code>	<code>std::stack</code> , <code>std::vector::back</code>	0 ns — caller must check
Exception	<code>void pop(); // throws</code>	Java <code>Stack</code> (<code>EmptyStackException</code>), Python <code>IndexError</code>	~1 ns when not thrown
Sentinel	<code>int pop(); // -1 if empty</code>	C APIs, competitive-programming code	0 ns, but <code>-1</code> may be a valid value
Optional	<code>std::optional<int> pop();</code>	Rust <code>Vec::pop</code> , modern C++	~1 ns, cannot be ignored silently

```
// The four, side by side, on the same stack.
void      pop_ub();           // UB if empty. Fastest.
void      pop_throw();       // throws std::underflow_error
int       pop_sentinel();    // returns -1 if empty. Ambiguous.
std::optional<int> pop_opt(); // std::nullopt if empty. Safe.

// Usage differs sharply:
if (!s.empty()) { int v = s.top(); s.pop_ub(); } // check is the caller's job
try { s.pop_throw(); } catch (const std::exception&) {}
if (auto v = s.pop_opt()) { use(*v); }          // impossible to misuse
```

THE KEY INSIGHT

The C++ standard library chose undefined behaviour deliberately, and the argument is worth being able to repeat: a check inside `pop` is redundant for every caller that already knows the stack is non-empty — and in a loop of the form `while (!s.empty()) { ... s.pop(); }`, that is every caller. Paying for a branch the caller has already taken is a cost with no beneficiary. Debug builds re-add the check via `_GLIBCXX_ASSERTIONS`.

THE TRAP

The sentinel strategy in a stack that can legitimately hold the sentinel. `pop()` returning `-1` is indistinguishable from popping a real `-1`, so an algorithm on signed data silently treats underflow as data. The symptom is not a crash: it is a plausible-looking wrong answer that appears only on inputs containing the sentinel value. This is the bug that survives code review. If you must use a sentinel, choose a value outside the domain and assert it.

error signalling — UB: 0 ns, caller checks · exception: ~1 ns guard, non-local exit · sentinel: 0 ns, ambiguous · optional: ~1 ns, unignorable. Two error states total.

6. When a stack is the wrong choice

Read this before the rest of the book, not after. A handbook that only argues for its subject teaches you to reach for it reflexively, which is the actual failure mode in real code and in interviews.

The four disqualifiers

1. You need the oldest element. A stack gives you the newest. Job queues, BFS frontiers, request buffers, and print spoolers all want FIFO. Use a queue — and note that swapping a stack for a queue in a graph traversal silently converts DFS into BFS (Topic 62), which is a correctness change, not a performance one.

2. You need the largest or smallest element. A stack's minimum is not accessible in $O(1)$ without augmentation, and even then only the minimum of the *whole* stack (Topics 56–57), never of an arbitrary range. If the problem says “highest priority next,” you want a heap; a stack that you sort on every push is $O(n \log n)$ per operation for something a heap does in $O(\log n)$.

3. You need to look at element k . If your code needs the third item from the top, you do not have a stack — you have a `std::vector` whose interface you have accidentally narrowed. Say so and use the vector. Faking it by popping three items into temporaries and pushing them back is $O(k)$ and, worse, it is not exception-safe.

4. You need both ends. Sliding-window maximum, work-stealing schedulers, and undo/redo with a bounded history all need `push_back/pop_back` and `pop_front`. That is a deque. Using two stacks to fake it (Topic 58) gives amortised $O(1)$ but a worst-case $O(n)$ single operation, and it is the wrong answer when the question involves latency percentiles rather than throughput.

THE DECISION IN PRACTICE

A ticketing backend buffers POS terminal events before a batch flush. The first design uses a stack because “push and pop are $O(1)$.” Under load the flush drains newest-first, so during a burst the oldest events sit at the bottom and can wait for the entire burst to clear — a tail latency proportional to burst size rather than to service time. Nothing is corrupt and no test fails; the p99 is simply $40\times$ the p50. The fix is one word: `queue`. *The choice of ADT is a latency decision, not only a complexity one.*

THE KEY INSIGHT

The positive test for a stack is not “I need fast insert and delete” — a deque, a vector and a heap all offer that. It is “*the most recently added item is the one that must be resolved next.*” If you cannot say that sentence about your problem in plain language, you do not have a stack problem.

THE TRAP

Using a stack for undo *and* expecting a bounded history. A `std::stack` has no way to drop its oldest entry, so a 10,000-action editing session holds all 10,000 undo records — in a document editor with image snapshots that is hundreds of megabytes and an OOM kill with no stack trace pointing anywhere near the undo code. Bounded history needs a deque or a ring buffer. Topic 60 shows the correct structure.

wrong-choice checklist – oldest first → queue · extreme first → heap · index k → vector · both ends or bounded → deque. Stack only when newest must resolve first.

PRACTICE SET — Topics 1-6

A. Conceptual

1. State the two algebraic laws of a stack. Which of them makes a stack an undo device, and why does the other one not?
2. `pop()` and `size()` differ in a way that determines the whole design of the class. What is the difference, in one word?
3. Both an array-backed and a node-backed stack have $O(1)$ push. Name the single row of the Topic 3 table on which the node-backed version wins, and describe one real system where that row decides the choice.
4. Why does `pop` not overwrite the slot it removes? What invariant makes that safe, and when does it stop being safe?
5. Your process has an 8 MB call stack and 16 GB of RAM. A colleague says “a `std::stack<int>` can hold at most 2 million ints.” What have they confused, and what is the real limit?
6. Name all the error states a stack can be in. Explain why there are exactly that many, referring to the interface rather than to the implementation.
7. The C++ standard makes `std::stack::pop()` on an empty stack undefined rather than throwing. Give the performance argument in one sentence, then give the strongest counter-argument.
8. Under what circumstance is the sentinel error strategy not merely inelegant but incorrect?
9. You are handed a class called `Stack` that exposes a public `begin()`. Give two concrete consequences.
10. Give the positive one-sentence test for “this is a stack problem.” Then apply it to: browser back button, print spooler, and undo in a text editor — state for each whether it passes.
11. Explain why converting recursion to an explicit stack changes the memory budget of the program, using the Topic 4 address-space diagram.
12. A bounded undo history of the last 50 actions cannot be built on `std::stack`. Why not, and what is the minimal structure that works?

B. Tracing and analysis

1. Trace the sequence `push 5, push 9, pop, push 2, push 7, pop, pop, push 1` on an initially empty stack. Give the contents bottom-to-top and `size()` after each of the eight operations.
2. Using the fixed-capacity `IntStack` from Topic 2 with `CAP = 64`: compute `sizeof` the object on x86-64, assuming `int` is 4 bytes and `std::size_t` is 8. Show the padding.
3. The same 1,000,000 `int` values are held in (a) an array-backed stack at exactly capacity and (b) a node-backed stack with 4-byte payload, 4 bytes of padding and an 8-byte pointer per node. Compute the total bytes for each and the ratio. Then recompute (a) immediately after a doubling from 524,288.
4. A node-backed push costs one `malloc` at roughly 20 ns; an array-backed push costs roughly 1 ns amortised. For 10 million pushes, compute the wall-clock difference in milliseconds.
5. Rewrite `buf[n++] = v;` as two separate statements in the correct order. Then write the two statements in the wrong order and state exactly which array index is written on the very first push.
6. A thread has `ulimit -s 8192`. A recursive function uses a 48-byte frame. Compute the maximum recursion depth, ignoring the guard page. Then recompute if a local `char buf[4096]` is added to the function.

7. Classify each as stack, queue, heap or deque, and name the disqualifier from Topic 6 where it is not a stack: (a) matching HTML tags, (b) a hospital triage list, (c) undo with unlimited history, (d) sliding-window maximum, (e) a depth-first maze walk, (f) a printer job list.

C. Code tasks

1. **Build.** Type in the `IntStack` from Topic 2 and write a `main` that pushes 1-10, pops 3, and prints `top()` and `size()`. Expected: 7 and 7.
2. **Break it — overflow.** Change `push` to remove the capacity check, then push 65 values into the 64-slot buffer. Run it once normally, then compile with `g++ -g -fsanitize=address -O0` and run again. Record what changes between the two runs, and copy down the first three lines of the sanitiser report.
3. **Break it — the off-by-one.** Change `buf_[n++] = v;` to `buf_[++n] = v;`, keep the capacity check, and push exactly 64 values. Predict before running: does the program crash, and what does `top()` return after the first push? Then verify under ASan.
4. **Break it — the sentinel.** Implement `int pop_sentinel()` returning `-1` on empty. Feed it the data set `{4, -1, 9}` and write a loop that drains the stack using `while (v != -1)`. Report how many elements the loop processed versus how many were pushed.
5. **Fix.** Replace `pop_sentinel` with `std::optional<int> pop_opt()` and rewrite the drain loop. Confirm the count is now correct on the same input.
6. **Measure.** Write a benchmark that pushes and pops 10,000,000 ints through (a) your `IntStack` sized to 10M, (b) `std::stack<int>`, (c) `std::stack<int, std::list<int>>`. Compile `-O2`. Report ns per operation for each and the ratio of (c) to (a).
7. **Argue.** Take task 6's numbers and write three sentences you would say in an interview justifying the array-backed choice, including the one circumstance under which you would reverse it.

ANSWER KEY — Topics 1-6

A. Conceptual

1. `top(push(s,v)) == v` and `pop(push(s,v)) == s`. The second makes it an undo device: it says push and pop are exact inverses, so any sequence of n pushes is undone by exactly n pops with the stack returned to its original state. The first only describes the peek operation and says nothing about reversibility.
2. **Totality.** `size()` is total (defined on every stack); `pop()` is partial (undefined when empty). Every design decision in Topic 5 is about what the partial operations do at their boundary.
3. **Worst-case single push.** Node-backed is $O(1)$ always; array-backed has one $O(n)$ push per n when it reallocates. It decides in **hard real-time code** — an interrupt handler or an audio callback with a fixed deadline, where a single 3 ms realloc pause is a failure even though the average is excellent.
4. Because the invariant is that only `buf_[0..n_-1]` holds live values; anything above `n_` is unreachable through the interface, so its contents cannot be observed. It stops being safe **when the element type has a destructor** — then abandoning the slot leaks. See Topic 18 and Topic 20.
5. They have confused the **call stack with the stack ADT**. A `std::stack<int>` allocates on the heap, so the limit is RAM, not `ulimit -s`: roughly 4×10^9 ints in 16 GB before counting the doubling overshoot.
6. **Two: underflow and overflow.** There are exactly two because the interface exposes exactly two preconditions — non-empty (for `pop/top`) and capacity-available (for `push`). No index is ever supplied by the caller, so no index can be invalid.
7. Performance: **a check inside pop is redundant for every caller in a while (!s.empty()) loop, and paying for a branch the caller already took has no beneficiary.** Counter-argument: undefined behaviour on a trivially checkable precondition converts a programmer error into memory corruption that can be exploited, and the branch is perfectly predicted, so its real cost on modern hardware is close to zero.
8. **When the sentinel value is a legal element.** Then underflow and a genuine element are indistinguishable, so the caller silently treats a missing value as data.
9. (i) Callers will iterate it, so you can never change the backing store without breaking them.
(ii) None of the LIFO guarantees hold, because the interface no longer enforces the discipline — it is a vector with a misleading name.
10. **“The most recently added item is the one that must be resolved next.”** Browser back: **passes** — the most recent page is the one you return to. Print spooler: **fails** — the oldest job prints next; that is a queue. Undo: **passes** — the most recent action is undone first.
11. Recursion stores frames in the **8 MB fixed mapping** at the top of the address space, which cannot grow and whose exhaustion is fatal. An explicit stack stores the same state on the **heap**, limited only by RAM, so the same algorithm survives depths three or four orders of magnitude greater.
12. Because `std::stack` has no operation that removes its **bottom** element — the interface forbids touching anything but the top. Minimal structure: a `std::deque`, using `push_back/pop_back` for the stack behaviour and `pop_front` to discard the oldest when size exceeds 50.

B. Tracing and analysis

1.

op	contents (bottom -> top)	size
push 5	[5]	1
push 9	[5, 9]	2
pop	[5]	1
push 2	[5, 2]	2
push 7	[5, 2, 7]	3
pop	[5, 2]	2
pop	[5]	1
push 1	[5, 1]	2

2. `int buf_[64]` = 256 bytes at offset 0; `std::size_t n_` needs 8-byte alignment, and 256 is already a multiple of 8, so **no padding is inserted**. Total `sizeof(IntStack) == 264`, alignment 8.
3. (a) $1,000,000 \times 4 = \mathbf{4,000,000 \text{ bytes}}$. (b) $1,000,000 \times 16 = \mathbf{16,000,000 \text{ bytes}}$. Ratio **4×**. After doubling from 524,288 the capacity is 1,048,576, so the array holds $1,048,576 \times 4 = \mathbf{4,194,304 \text{ bytes}}$ — 4.8% overshoot, and the ratio becomes 3.81×
4. $10^7 \times (20 - 1) \text{ ns} = 1.9 \times 10^8 \text{ ns} = \mathbf{190 \text{ ms}}$.
5. Correct: `buf_[n_] = v;` then `++n_;`. Wrong order: `++n_;` then `buf_[n_] = v;` writes index **1** on the first push, leaving index 0 permanently uninitialised and overrunning by one at the top.
6. $8192 \text{ KB} = 8,388,608 \text{ bytes}$; $8,388,608 / 48 = \mathbf{174,762 \text{ frames}}$. With `char buf[4096]` the frame becomes 4144 bytes (48 + 4096, before alignment padding), giving $8,388,608 / 4144 = \mathbf{2024 \text{ frames}}$ — an 86× reduction from one local array.
7. (a) **stack**. (b) **heap** — extreme-first disqualifier. (c) **stack**. (d) **deque** — both-ends disqualifier. (e) **stack**. (f) **queue** — oldest-first disqualifier.

C. Code tasks — what to verify

1. Prints **7** and **7**. If you get **10** and **7** you popped without re-reading `top()`.
2. Without the sanitiser the program very likely **prints nothing unusual and exits 0** — the 65th write lands on `n_` itself, which sits immediately after `buf_`, corrupting the size. Under ASan you should see, on the first three lines:

```
==NNNN==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7ffd...  
WRITE of size 4 at 0x7ffd... thread T0  
#0 0x... in IntStack::push(int) main.cpp:14
```

The lesson is the gap between the two runs, not the crash: **silent corruption is the default; the sanitiser is what makes it visible**.

3. Prediction: it does **not** crash on the first push, and `top()` returns **garbage**, because `top()` reads `buf_[n_-1] == buf_[0]` which was never written while the value went to `buf_[1]`. The overrun happens on the 64th push, writing `buf_[64]`. ASan reports **stack-buffer-overflow** at **push #64**, not push #1 — the error and its symptom are 63 operations apart, which is exactly why this class of bug is expensive.
4. Pushed 3, loop processes **1** (the 9), then sees the genuine `-1` and stops. **Two of three elements are silently lost** and no error is reported anywhere.
5. The loop is now `while (auto v = s.pop_opt())` and processes **all 3**. The type system made the error unrepresentable rather than merely unlikely.
6. Typical **-02** figures on a modern x86-64 laptop, per push+pop pair: (a) **1-2 ns**, (b) **2-4 ns** (**deque** backing adds a block-index indirection — see Topic 67), (c) **25-60 ns**. Ratio (c):(a) is roughly **20-40×**. If (c) is under 10 ns your benchmark was optimised away — consume the popped value with a volatile accumulator.

7. Model answer: “Both are $O(1)$, so the choice is decided by constants. The array form is $4\times$ smaller per element and $20\text{--}40\times$ faster in a push/pop loop because it never calls the allocator and stays in L1. I would reverse it only where a single $O(n)$ reallocation pause is unacceptable — an audio callback or an ISR — because the node form is $O(1)$ worst case, not just amortised.”

HANDNOTE SUMMARY CARD

PART 1 - FOUNDATION

Topics 1-6

CONTRACT

```
push(v)  n -> n+1, top == v      top(push(s,v)) == v
pop()    n -> n-1 [needs n>=1]  pop(push(s,v)) == s  <- the undo law
top()    returns xn [needs n>=1]
empty()  total
size()   total
```

LAWS

THE POSITIVE TEST

"the most recently added item is the one that must be resolved next"

MINIMAL IMPLEMENTATION

```
int buf_[CAP]; size_t n_ = 0;  INVARIANT: buf_[0 .. n_-1] are live
push: buf_[n_++] = v;          ++n_ then write => index 0 garbage
pop : --n_;                    slot is abandoned, not erased
top : return buf_[n_-1];       UB if n_ == 0
```

SUBSTRATE CHOICE

	array	nodes
push (amortised)	~1 ns	~20 ns
worst-case single push	O(n)	O(1) <- only win
bytes per int	4	16
cache	prefetch	1 miss/pop
verdict:	array, unless a single push has a hard deadline	

TWO THINGS CALLED "STACK"

ADT heap, limit = RAM, you manage it
call stack 8 MB (ulimit -s 8192), grows DOWN, rsp, guard page, fatal
recursion -> explicit stack == moving frames from the 8 MB map to the heap

TWO ERROR STATES ONLY: underflow (pop/top on empty), overflow (push past cap)

UB	void pop()	std::stack	0 ns, caller checks
throw	void pop()	Java, Python	~1 ns guard
sentinel	int pop() -> -1	C APIs	WRONG if -1 is legal
optional	optional pop()	Rust Vec::pop	unignorable

WHEN IT IS THE WRONG STRUCTURE

need oldest	-> queue	(stack->queue turns DFS into BFS, T62)
need min/max	-> heap	(stack min needs augmenting, T56-57)
need element k	-> vector	(you narrowed the interface by accident)
need both ends	-> deque	(two-stack queue is amortised, not worst case)
need bounded undo	-> deque	(std::stack cannot drop its bottom element)

TRAPS

T1 public begin() on a "Stack" -> backing store frozen forever
T2 buf_[++n_] = v -> index 0 garbage, overrun 63 ops later
T3 "call stack is faster memory" -> 4 MB local array = half your thread
T4 sentinel that is legal data -> silent data loss, no crash, passes review
T5 std::list backing -> 8-15x slower than vector backing

NUMBERS TO KNOW

8 MB / 48-byte frame = 174,762 frames
8 MB / 4144-byte frame = 2,024 frames (one char buf[4096] costs 86x)
1M ints: array 4.0 MB, nodes 16.0 MB, ratio 4x (3.81x just after a double)
10M pushes: nodes cost +190 ms over array

PART TWO

Memory and representation

Where the bytes actually go. Growth policy, the amortisation proof an interviewer will ask you to produce at the whiteboard, the shrink hysteresis nobody teaches, and the cache measurement that settles the array-versus-nodes argument for good.

7. Array-backed layout and the top-index invariant

An array-backed stack is three fields: a pointer to a buffer, a capacity, and a count. The count is the whole design. It is simultaneously the number of live elements, the index of the next free slot, and the boundary between defined and undefined memory. Getting that triple identity straight removes most off-by-one bugs before they happen.

```
struct Stack {
    int*      data = nullptr;    // owned buffer, or nullptr when cap == 0
    std::size_t n      = 0;      // live count AND next-write index
    std::size_t cap    = 0;      // allocated slots
};
// INVARIANT (holds before and after every public operation):
//   n <= cap
//   data[0 .. n-1] are live, constructed values
//   data[n .. cap-1] are raw storage -- reading them is UB
//   cap == 0 <=> data == nullptr
```

Write that invariant block as a comment in real code. It is not decoration: in Topic 21 you will turn each of those four lines into an assertion, and the fuzz harness that checks them is what catches the bugs the unit tests miss.

```

                                n = 3                cap = 8
                                |                    |
data ---> +---+---+---+---+---+---+---+---+
          | 10 | 20 | 30 | ?? | ?? | ?? | ?? | ?? |
          +---+---+---+---+---+---+---+---+
          0   1   2   3   4   5   6   7
          |<---- live ---->|<----- raw storage ----->|
                                ^
                                next write goes here: data[n]

top() == data[n-1] == data[2] == 30
push(v): data[n] = v; ++n;          (grow first if n == cap)
pop()   : --n;                     (data[3] now raw again)
```

The choice to store `n` as a count rather than a pointer to the top element is deliberate. A top *pointer* has to be recomputed on every reallocation and has no valid value for an empty stack — you would need `data - 1`, which is undefined behaviour in C++ even if you never dereference it. A count has a natural empty value of zero and survives reallocation untouched.

THE KEY INSIGHT

`n` is the index of the next write *and* one past the top *and* the live count, all at once. Every off-by-one in stack code is a confusion between two of those three readings. When you are stuck, write down which one you meant.

THE TRAP

Keeping a `int* top_` instead of a count, then reallocating. The old `top_` points into the freed buffer, and because `realloc` frequently returns the same address for small growth steps, the bug is invisible in tests and appears the first time a growth step actually moves the block. Symptom: a heap-use-after-free that reproduces only above a specific stack depth. Store the count.

array layout — top $O(1)$ · Space $4 \cdot \text{cap} + 24$ bytes of header · one allocation total · invariant is four lines, assert all four

8. Growth policy: 2× vs 1.5×

When $n == \text{cap}$, push must allocate a larger buffer, copy, and free the old one. The only question is how much larger. Growing by a fixed *amount* — $\text{cap} + 1$, or $\text{cap} + 100$ — is the mistake that turns a stack into a quadratic structure. Growing by a fixed *factor* is what makes push amortised $O(1)$, and Topic 9 proves it.

```
void grow() {
    std::size_t newcap = cap ? cap * 2 : 4;    // 0 -> 4 -> 8 -> 16 -> ...
    int* nd = static_cast<int*>(std::malloc(newcap * sizeof(int)));
    if (!nd) throw std::bad_alloc();
    std::memcpy(nd, data, n * sizeof(int));    // trivially copyable only
    std::free(data);
    data = nd;
    cap = newcap;
}
void push(int v) {
    if (n == cap) grow();
    data[n++] = v;
}
```

Total copy work to reach n elements by doubling from 1 is $1 + 2 + 4 + \dots + n/2 < n$. Fewer than one copy per element over the whole lifetime. Growing by a constant c instead gives $n^2/(2c)$ copies — for $n = 1,000,000$ and $c = 100$ that is **5×10^9 element copies** versus fewer than **10^6** . A factor of 5000.

Why libstdc++ uses 2× and MSVC uses 1.5×

Factor	Reallocations to reach n	Peak memory	Can reuse freed blocks?
2× (libstdc++, libc++)	$\log_2 n$ — 20 for 1M	up to $2n$, mean $1.5n$	No — sum of all previous blocks is always one short
1.5× (MSVC)	$\log_{1.5} n$ — 35 for 1M	up to $1.5n$, mean $1.25n$	Yes after ~ 4 steps, since $1.5^n < \text{sum of predecessors}$

The block-reuse argument is the interesting one. With doubling, the new request is always strictly larger than every previously freed block combined ($1+2+4+\dots+2^{n-1} = 2^n - 1 < 2^n$), so the allocator can never satisfy it from the space you just released; the heap walks forward. With 1.5×, from the fourth growth onward the freed blocks do sum to more than the request, so a coalescing allocator can reuse them and the address range stays bounded.

THE KEY INSIGHT

The factor does not change the complexity class — every factor > 1 gives amortised $O(1)$. It trades **reallocation count** (fewer with 2×) against **peak memory and address-space fragmentation** (better with 1.5×). If an interviewer asks “why 1.5?”, the block-reuse argument above is the answer they are fishing for.

THE TRAP

`newcap = cap * 2` with `cap == 0` gives zero, so the buffer is never allocated and the first `data[n++] = v` writes through a null pointer. Symptom: a segfault on the very first push of a default-constructed stack, in a function that works perfectly once anything has been pushed. The `cap ? cap * 2 : 4` ternary exists solely for this. A second, rarer form: `cap * 2` overflowing `size_t` on a 32-bit build at 2 GB, wrapping to a small number and silently shrinking.

growth – push $O(1)$ amortised, $O(n)$ worst case on the growing push · total copies $< n$ over the lifetime · $2\times$: 20 reallocs per 1M, peak $2n$ · $1.5\times$: 35 reallocs, peak $1.5n$, blocks reusable

9. Amortised $O(1)$ push by the accounting method

“Amortised $O(1)$ ” is not “usually $O(1)$ ” and it is not an average over random inputs. It is a worst-case guarantee about *sequences*: any sequence of m pushes costs at most $c \cdot m$ total, for a constant c , no matter which sequence. There is no input that defeats it. Being able to prove that at a whiteboard in under two minutes is a common interview bar, so here is the proof in the form that is fastest to reproduce.

The accounting method

Charge every push **3 credits**. One credit pays for the actual write. The other two are saved in the slot just written. Claim: when a reallocation happens, the saved credits are exactly enough to pay for copying every element.

Just after growing to `cap = 8`, with `n = 4` (elements copied, no credits left):

slot	0	1	2	3	4	5	6	7
	+---+---+---+---+---+---+---+---+							
	a	b	c	d				
	+---+---+---+---+---+---+---+---+							
credit	0	0	0	0	<- all spent by the copy that just happened			

Now push e, f, g, h. Each pays 1 to write, banks 2:

slot	0	1	2	3	4	5	6	7
	+---+---+---+---+---+---+---+---+							
	a	b	c	d	e	f	g	h
	+---+---+---+---+---+---+---+---+							
credit	0	0	0	0	2	2	2	2

= 8 credits banked

The next push triggers a copy of all 8 elements: cost 8.
Banked = 8. Exactly enough. Balance never goes negative. QED.

The reason it balances is that doubling means *half* the elements to be copied were pushed since the last growth, and each of those banked 2 credits — so the bank holds $2 \times (\text{cap}/2) = \text{cap}$, which is precisely the copy cost. With a growth factor k , the required charge is $1 + k/(k-1)$: 3 for $k = 2$, and 4 for $k = 1.5$. Still constant, which is the point.

What amortised does not give you

Claim	True?	Why it matters
Every push is fast	No	One push in n copies n elements
m pushes cost $O(m)$	Yes	Guaranteed for every sequence, not on average
p99 latency is $O(1)$	No	The realloc push is the tail; reserve to remove it
Safe in an ISR	No	Use fixed capacity (Topic 13) or nodes (Topic 11)

THE KEY INSIGHT

Amortised analysis is a statement about a *sequence*, and it is a worst-case statement, not an average-case one. This is the distinction interviewers probe: “is amortised $O(1)$ the same as average $O(1)$?” No — average-case allows an adversarial input to be slow; amortised does not. Hash table lookup is average $O(1)$; vector `push_back` is amortised $O(1)$. They are different guarantees.

THE TRAP

Quoting amortised $O(1)$ for a latency-sensitive system. A trading gateway pushing to a growing buffer sees a 4 ms pause when the buffer crosses 64 MB — a memcpy of 64 MB at ~16 GB/s. The average is still 2 ns. The symptom is a p99.9 spike with no corresponding CPU or GC event and nothing in the logs. Fix: `reserve()` the known maximum up front, which converts amortised $O(1)$ into worst-case $O(1)$ and costs one allocation at startup.

amortisation – 3 credits per push at $2\times$, 4 at $1.5\times$ · m pushes cost $\leq 3m$ total, worst case · single push worst case still $O(n)$ · `reserve()` converts amortised to worst case

10. Shrink policy and the thrash trap

If a stack grows to a million elements and then drains to ten, it is still holding 4 MB. Should `pop` shrink the buffer? The obvious rule — halve the capacity when the stack is half empty — is wrong, and wrong in an instructive way.

THE THRASH: shrink at $n == \text{cap}/2$, grow at $n == \text{cap}$

cap=8, n=4 push -> n=5 pop -> n=4 push -> n=5

... ..

Actually the pathological pair sits at the boundary cap=8, n=8:

```
push (n==cap)      -> GROW to 16, copy 8      cost 8
pop  (n==8==16/2)  -> SHRINK to 8, copy 8      cost 8
push (n==8==cap)   -> GROW to 16, copy 8      cost 8
pop                -> SHRINK to 8, copy 8      cost 8
...repeat forever: every operation is  $O(n)$ .
```

Amortised $O(1)$ is destroyed by an alternating push/pop at the boundary.
Cost of m operations: $O(m*n)$, not $O(m)$.

The fix is hysteresis

Shrink when $n < \text{cap}/4$, and shrink to `cap/2`. Now after a shrink the stack sits at half the new capacity, so it needs to *double* its element count to trigger a grow, or *halve* it again to trigger another shrink. Every reallocation is separated from the next by at least `cap/4` operations, which restores the amortised bound.

```
void pop() {
    --n;
    if (cap > 8 && n < cap / 4)    // hysteresis: 1/4 trigger, 1/2 target
        realloc_to(cap / 2);
}
```

THE KEY INSIGHT

The gap between the grow trigger ($n == \text{cap}$) and the shrink trigger ($n < \text{cap}/4$) is what buys back the amortised bound. Any policy where the two triggers meet at the same point can be driven into $O(n)$ -per-operation by an alternating sequence. This is the same reasoning that governs hash table load factors and B-tree node merge thresholds — recognise it and you have it for free in three other subjects.

THE TRAP

`std::vector` never shrinks on `pop_back`, and `shrink_to_fit()` is only a non-binding request. So a `std::stack` that peaked at 10 million elements holds 40 MB forever, even at size zero, and `clear()` does not release it either. Symptom: resident memory that ratchets upward across a long-running process and never comes down, with no leak reported by Valgrind — because it is not a leak, it is retained capacity. The fix is the swap idiom: `std::vector<int>().swap(v);`, or in C++11 and later `v = {};` `v.shrink_to_fit();`.

shrink policy — trigger at $n < \text{cap}/4$, target $\text{cap}/2$ · naive $\text{cap}/2$ trigger gives $O(n)$ per op forever · `std::vector` never shrinks: peak capacity is permanent

11. The linked-node stack and its overhead

A node-backed stack is a singly linked list where every operation touches the head. It is Volume I's structure with four fifths of the operations deleted — no traversal, no middle insert, no tail pointer, no length walk. That deletion is what makes it correct by construction: the only pointer manipulation is at one end.

```
struct Node { int v; Node* next; };

class NodeStack {
    Node* head_ = nullptr;
    std::size_t n_ = 0;
public:
    void push(int v) { head_ = new Node{v, head_}; ++n_; }    // O(1) worst case
    void pop() {
        Node* d = head_;
        head_ = head_->next;    // UB if empty -- see Topic 5
        delete d;
        --n_;
    }
    int& top() { return head_->v; }
    bool empty() const { return head_ == nullptr; }
    ~NodeStack() { while (head_) pop(); }    // NOT recursive -- see Topic 29
};
```

Note `new Node{v, head_}`: the new node's `next` is the *old* head, and only then does `head_` move. Reversing those two lines loses the entire stack in one statement, and it is the single most common bug in hand-written node stacks.

The overhead arithmetic, exactly

```
struct Node { int v; Node* next; }; on x86-64, alignof(Node*) == 8
```

```
offset 0      4      8      16
+-----+-----+-----+
| v | padding | next |
+-----+-----+-----+
4 B    4 B      8 B      = sizeof(Node) == 16
```

Plus glibc malloc's per-chunk header (8 B) and 16-byte alignment:
an actual heap chunk for a 16-byte request is 32 bytes.

1,000,000 ints:	array-backed	4,000,000 B	(4 B/int)
	node-backed	16,000,000 B	(16 B/int, sizeof)
	node, real	32,000,000 B	(32 B/int, with malloc)

	ratio	8x actual, 4x on paper	

THE KEY INSIGHT

The textbook “4× overhead” figure counts `sizeof(Node)` and stops. The number that shows up in `ps` is **8×**, because glibc's allocator rounds a 16-byte request up to a 32-byte chunk. Whenever you quote node overhead, say which number you mean; an interviewer who has measured it will know the difference.

THE TRAP

`head_ = new Node{v, head_};` written in two statements as `head_ = n; n->next = head_;` makes the new node point at itself. The stack now has a one-element cycle: `pop` succeeds forever, always returning the same value, and `size()` disagrees with reality. The symptom is an infinite loop in the destructor, not at the push site. Practice set 2 has you produce it.

node stack – push/pop $O(1)$ worst case · 16 B/int by `sizeof`, 32 B/int as allocated · ~20 ns per push (malloc) · one cache miss per pop · no reallocation pause, ever

12. Cache behaviour, measured

Complexity classes say array and node stacks are identical. Hardware says they are not, by a factor between 5 and 40. The mechanism is the cache line, and the arithmetic is simple enough to do in your head during an interview.

One 64-byte cache line holds:

ARRAY-BACKED: 16 ints, contiguous

```
+----- 64 bytes -----+
| i0 i1 i2 i3 i4 i5 i6 i7 i8 i9 iA iB iC iD iE iF |
+-----+
```

One miss (~80 ns from DRAM) then 15 hits (~1 ns each).
Amortised: $80/16 = 5$ ns per element -- and the hardware prefetcher sees the linear pattern, so in practice ~1 ns.

NODE-BACKED: 1 int + 1 pointer + padding, scattered

```
+---- 64 B ----+      +---- 64 B ----+      +---- 64 B ----+
| 30 | ptr |...|      | 20 | ptr |...|      | 10 |  0 |...|
+-----+      +-----+      +-----+
      line A              line C              line Q
```

One miss per node (~80 ns). Address of the next node is only known AFTER the current one arrives -- the prefetcher cannot help. This is a dependent load chain.

The words for that last point are **pointer chasing** and **dependent load**. The CPU cannot start fetching node $k+1$ until node k has arrived, because node k contains the address. No amount of out-of-order execution hides it. An array walk, by contrast, has addresses computable from the index alone, so the prefetcher runs dozens of lines ahead.

A benchmark that does not lie

```
// g++ -O2 -std=c++17 bench.cpp
// The volatile sink prevents the whole loop being optimised away --
// without it, -O2 deletes the benchmark and you measure nothing.
volatile long sink = 0;

template <class S>
double bench(S& s, int iters) {
    auto t0 = std::chrono::steady_clock::now();
    for (int i = 0; i < iters; ++i) s.push(i);
    for (int i = 0; i < iters; ++i) { sink += s.top(); s.pop(); }
    auto t1 = std::chrono::steady_clock::now();
    return std::chrono::duration<double, std::nano>(t1 - t0).count()
        / (2.0 * iters);                      // ns per operation
}
```

Backing (10M push + 10M pop)	ns per op	vs vector
std::vector, reserved	0.8-1.5	1.0×
std::vector, growing	1.5-2.5	~1.7×
std::deque (the std::stack default)	2-4	~2.5×
hand-rolled NodeStack	20-45	~25×
std::stack<int, std::list<int>>	30-60	~35×

THE KEY INSIGHT

One cache line delivers 16 ints from an array and 1 int from a node stack. That sentence, plus “the next node's address is only known after the current node arrives, so the prefetcher is blind,” is the entire performance argument. Memorise the two sentences; they answer array-versus-list questions for stacks, queues, trees and graphs alike.

THE TRAP

Benchmarking without a sink. At **-O2** the compiler observes that the popped values are unused and deletes the loop body, the loop, and often the container. Symptom: a node stack that appears to run at 0.1 ns per operation, faster than a single L1 hit — a physically impossible result that people nonetheless report. If your number is below ~0.5 ns per op, you measured an empty loop.

cache – 64 B line = 16 ints (array) or 1 int (nodes) · DRAM miss ~80 ns · array ~1 ns/op prefetched · nodes 20–45 ns/op, dependent loads, un prefetchable · measured ratio 20–40×

13. Fixed-capacity stack with no heap

If you know the maximum depth in advance — and for the algorithms in Parts 5 to 9 you almost always do — you can delete the allocator from the design entirely. Every push becomes worst-case $O(1)$, the memory is one contiguous block with no header, and the structure works in contexts where `malloc` is forbidden: interrupt handlers, signal handlers, kernel code, and anything with a hard deadline.

```
template <class T, std::size_t N>
class FixedStack {
    alignas(T) unsigned char buf_[N * sizeof(T)]; // raw storage
    std::size_t n_ = 0;
    T* slot(std::size_t i) { return reinterpret_cast<T*>(buf_) + i; }
public:
    bool full() const { return n_ == N; }
    bool empty() const { return n_ == 0; }

    template <class... A>
    bool try_emplace(A&&... a) { // no exception, no allocation
        if (n_ == N) return false;
        ::new (slot(n_)) T(std::forward<A>(a)...); // placement new
        ++n_;
        return true;
    }
    void pop() { slot(--n_)->~T(); } // explicit destructor call
    T& top() { return *slot(n_ - 1); }
    ~FixedStack() { while (n_) pop(); }
};
```

Two details separate this from a naive `T buf_[N]`. First, raw `unsigned char` storage with placement new means the N objects are not default-constructed at creation — a `FixedStack<std::string, 1000>` costs nothing until you push. Second, `pop` must call the destructor explicitly, because nothing else will.

How much stack space does it use?

Declaration	Bytes	Safe as a local?
<code>FixedStack<int, 64></code>	264	Yes
<code>FixedStack<int, 100000></code>	400,008	Marginal — 4.8% of an 8 MB stack
<code>FixedStack<int, 3000000></code>	12,000,008	No — overflows immediately

THE KEY INSIGHT

Fixed capacity converts *amortised* $O(1)$ into *worst-case* $O(1)$ and moves the failure from “pause” to “refusal”. That is almost always the better failure: `try_emplace` returning `false` is something you can handle, whereas a 4 ms reallocation pause inside a 1 ms deadline is not.

THE TRAP

Declaring a large `FixedStack` as a local variable to avoid the heap, and thereby consuming the call stack instead — the exact confusion from Topic 4. `FixedStack<int, 3000000>` as a local is a 12 MB frame on an 8 MB stack: the guard page is hit during the function *prologue*, before a single line of your code runs. Symptom: a segfault whose backtrace points at the opening brace of the function. Make it `static`, a member, or heap-allocated.

fixed capacity – push/pop $O(1)$ worst case · zero allocations · $N \cdot \text{sizeof}(T) + 8$ bytes, no malloc header · overflow becomes a return value, not a pause · keep locals under ~1 MB

PRACTICE SET — Topics 7-13

A. Conceptual

1. Write the four-line invariant of the array-backed stack from memory. Which line is violated by a use-after-free following reallocation?
2. Give the three simultaneous meanings of `n`. Which pair is confused by writing `data[n]` in `top()`?
3. Why is a stored `int* top_` worse than a stored count, in two respects — one about reallocation, one about the empty stack?
4. Growing by a constant 100 instead of a factor still gives $O(1)$ per push in practice for small stacks. State precisely when it stops being acceptable, with the formula for total copy work.
5. Explain in one sentence why a doubling allocator can never reuse its own freed blocks, and why $1.5\times$ can.
6. “Amortised $O(1)$ ” and “average-case $O(1)$ ” are different guarantees. Give an operation with each, and state which one an adversary can defeat.
7. In the accounting proof, why is the charge 3 credits and not 2? Where does the third credit go?
8. Derive the required credit charge for a growth factor of 1.5 and state the general formula.
9. Describe the alternating sequence that destroys amortisation under a naive shrink-at-half policy, and state the resulting cost of m operations.
10. Why does `std::vector` refuse to shrink on `pop_back`? Give the argument in favour of that decision, then the cost it imposes on long-running processes.
11. What exactly does the hardware prefetcher fail to do on a node stack, and what is the technical name for the chain of loads it cannot run ahead of?
12. State the two numbers for node overhead per `int` — the `sizeof` figure and the as-allocated figure — and explain the gap.
13. Why does `FixedStack` use `unsigned char buf_[]` plus placement new rather than `T buf_[N]`? Give a type for which the difference is expensive.

B. Tracing and analysis

1. Starting from `cap = 0` with the `cap ? cap*2 : 4` policy, list every capacity the stack passes through while pushing 100 elements, and give the total number of element copies performed.
2. Repeat for a growth policy of `cap + 8`. Give the number of reallocations and the total element copies for 100 pushes, then for 100,000 pushes.
3. Compute total element copies to reach $n = 1,000,000$ under: (a) doubling, (b) $1.5\times$, (c) `cap + 100`. Express (c) as a multiple of (a).
4. A `std::vector<int>` has just grown to hold 700,000 elements under a doubling policy starting at 1. Give its capacity, its allocated bytes, and the percentage of that memory currently unused.
5. Under the accounting method, a stack at `cap = 16` has $n = 9$. How many credits are banked? Show the balance immediately after the push that triggers growth to 32.
6. Compute `sizeof(Node)` for `struct Node { char c; Node* next; }` and for `struct Node { Node* next; char c; }` on x86-64. Are they the same? Draw the padding.

7. A DRAM miss costs 80 ns and an L1 hit 1 ns. Compute the time to pop all 1,000,000 elements from (a) an array-backed stack, assuming perfect prefetch after the first line, and (b) a node stack with every node in a different line and no prefetch. Give the ratio.
8. Compute the exact byte size of `FixedStack<double, 1024>` and state what fraction of an 8 MB thread stack it would consume as a local.
9. A vector-backed stack peaked at 10,000,000 ints then was emptied with `pop`. Give the resident bytes still held, and the two-line idiom that releases them.

C. Code tasks

1. **Build.** Implement the growable `Stack` from Topics 7–8 with a counter of reallocations. Push 1,000,000 ints and print the realloc count and the final capacity. Expected: **18** reallocations and capacity **1,048,576**.
2. **Break it — the zero-capacity bug.** Change `grow()` to `newcap = cap * 2` with no ternary. Predict what happens on the first push of a fresh stack, then run it, then run it under `-fsanitize=address`. Record the difference between the two failure messages.
3. **Break it — the dangling top pointer.** Replace the count with a `int* top_` member that is *not* recomputed inside `grow()`. Push 3 elements and call `top()`: it will probably work. Now push 10,000 and call `top()` after each. Report the first index at which it breaks and the ASan error class.
4. **Break it — the shrink thrash.** Add `if (n < cap/2) realloc_to(cap/2);` to `pop`. Then run 1,000,000 alternating push/pop operations with the stack sitting exactly at a capacity boundary. Time it. Change the trigger to `n < cap/4` with target `cap/2` and time it again. Report both wall-clock figures and the realloc counts.
5. **Break it — the self-cycle.** In `NodeStack::push`, write the two statements in the wrong order so the new node points at itself. Push 3 values, then call `pop` five times and print `top()` each time. Describe what the destructor does.
6. **Fix and verify.** Correct task 5, then add an assertion that `size()` equals the number of nodes reached by walking from `head_`. Confirm it passes.
7. **Measure.** Run the Topic 12 benchmark across all five backings in the table, with and without the `volatile` sink. Report ns per operation for each. State which rows changed most when the sink was removed and explain why.
8. **Measure.** Instrument the growable stack to record the wall-clock duration of every individual push into a histogram. Push 10,000,000 ints. Report the median, the p99, and the maximum. Then `reserve` 10,000,000 up front and repeat.

ANSWER KEY — Topics 7-13

A. Conceptual

1. `n <= cap; data[0..n-1] live; data[n..cap-1] raw; cap == 0 <=> data == nullptr`. A use-after-free violates the **fourth** line's spirit — more precisely, it violates the unstated premise that `data` is the *current* owned buffer. This is why the invariant is written in terms of `data` rather than any cached derived pointer.
2. Live count, next-write index, one-past-the-top. `top()` reading `data[n]` confuses **next-write index with one-past-the-top being the same slot** — it reads raw storage. Correct is `data[n-1]`.
3. (i) A pointer must be recomputed on every reallocation, and forgetting it gives a dangling read into freed memory. (ii) An empty stack would need `data - 1`, which is **undefined behaviour in C++ even without dereferencing**; a count has a natural empty value of 0.
4. It stops being acceptable as soon as n grows large, because total copy work is $n^2/(2c)$, which is quadratic. At $n = 10^5$ and $c = 100$ that is 5×10^9 copies against $< 10^6$ for doubling — $5000\times$.
5. Doubling: $1+2+\dots+2^{n-1} = 2^n-1$, which is always one byte short of the next request, so freed space never suffices. $1.5\times$: from about the fourth step, the predecessors sum to more than the next request, so a coalescing allocator can satisfy it in place.
6. Amortised: `vector::push_back`. Average-case: **hash table lookup**. An adversary **can** defeat average-case (collision flooding) but **cannot** defeat amortised — the bound holds for every sequence.
7. 1 credit pays for the write itself. The other 2 pay for copying this element during the *next* growth and, effectively, for one older element that has already spent its savings. At doubling, half the elements copied were pushed since the last growth, so each must fund itself and one predecessor.
8. For factor k , growth from cap to $k \cdot cap$ copies cap elements, funded by the `cap - cap/k` pushes since the last growth. Charge $c = 1 + k/(k-1)$. $k = 2 \rightarrow 3$; $k = 1.5 \rightarrow 4$; $k = 1.1 \rightarrow 12$. Constant for every $k > 1$.
9. Sit the stack at `n == cap`, then alternate push and pop forever. The push grows and copies n ; the pop immediately trips the half-empty trigger and copies n back. Cost of m operations is **$O(m \cdot n)$** rather than $O(m)$.
10. In favour: the standard cannot know whether the peak will recur, and an implicit shrink would silently invalidate all outstanding references and iterators on a `pop_back` — a surprising and expensive guarantee to break. The cost: **peak capacity is permanent**, so resident memory ratchets and never returns, and Valgrind reports no leak because there is none.
11. It cannot compute the address of node $k+1$ before node k arrives, so it cannot issue the load early. The name is a **dependent load chain** (pointer chasing).
12. **16 bytes** by `sizeof` (4 payload + 4 padding + 8 pointer) and **32 bytes** as allocated, because glibc adds an 8-byte chunk header and rounds to 16-byte alignment, so a 16-byte request occupies a 32-byte chunk.
13. Because `T buf[N]` **default-constructs all N objects at creation**. For `Fixed-Stack<std::string, 1000>` that is 1000 string constructions and 32 KB touched before a single push; with raw storage it is zero work and zero pages touched.

B. Tracing and analysis

1. Capacities: **4, 8, 16, 32, 64, 128**. Copies: $0 + 4 + 8 + 16 + 32 + 64 = 124$ element copies for 100 pushes — 1.24 per element.
2. `cap + 8`: capacities 8, 16, ..., 104 → **13 reallocations**, copies $8+16+\dots+96 = 624$. For 100,000 pushes: **12,500 reallocations** and $8 \cdot (1+2+\dots+12499) \approx 6.25 \times 10^8$ copies — 6250 per element.
3. (a) doubling: $< 1,048,576$, i.e. under 1.05 copies per element. (b) $1.5\times$: total $\approx 2n = 2,000,000$ (the series sums to $n \cdot k / (k-1) - n = 2n$ at $k=1.5$). (c) `cap+100`: $n^2/200 = 5 \times 10^9$. (c) is roughly **4800**× (a).
4. Capacities double 1, 2, 4, ...; the first $\geq 700,000$ is **1,048,576**. Bytes = 4,194,304. Unused = $(1,048,576 - 700,000)/1,048,576 = 33.2\%$.
5. Elements 8-15 were pushed since the growth to 16, so $9 - 8 = 1$ element has banked, holding **2 credits**. Continue to $n = 16$: 8 banked elements $\times 2 = 16$ credits; growth copies 16 elements at cost 16; balance after = **0**, never negative.
6. Both are **16 bytes**. `{char c; Node* next;}`: c at 0, 7 bytes padding, next at 8. `{Node* next; char c;}`: next at 0, c at 8, then 7 bytes *trailing* padding so the struct's size is a multiple of its 8-byte alignment. Reordering does not help here because there are only two members.
7. (a) $1,000,000 / 16 = 62,500$ lines $\times 80$ ns = **5.0 ms** (and with prefetch, in practice ~ 1 -2 ms). (b) $1,000,000 \times 80$ ns = **80 ms**. Ratio **16**× on the line arithmetic alone, more once prefetch is counted — matching the measured 20-40×.
8. $1024 \times 8 = 8192$ bytes of storage + 8 bytes for `n_` = **8200 bytes**, rounded to 8-byte alignment. That is **0.098%** of an 8 MB stack — entirely safe as a local.
9. Capacity is 16,777,216 (the doubling step at or above 10^6), so **67,108,864 bytes $\approx$ 64 MB** retained at size zero. Release: `std::vector<int>().swap(v);` — or `v = {}; v.shrink_to_fit();`.

C. Code tasks — what to verify

1. Capacities 4, 8, ..., 1,048,576. That is $4 \cdot 2_u = 1,048,576$ at $k = 18$, so **18 reallocations** and final capacity **1,048,576**. If you count 20, you started from capacity 1.
2. Prediction: `newcap = 0*2 = 0`, `malloc(0)` returns either a non-null unique pointer or null; `cap` stays 0, so `n == cap` forever and every push re-enters `grow`, or writes through a null/zero-sized block. Bare run: **segfault, or an infinite loop of zero-sized allocations**. Under ASan you should get `heap-buffer-overflow ... WRITE of size 4 on a 0-byte region` — the phrase to look for is `allocated by thread T0 here` above a size of 0. The lesson: the bare run's symptom is ambiguous, the sanitiser's is exact.
3. It works for 3 pushes because the first buffer is never moved. Breakage begins at the **first push that triggers a reallocation which actually relocates the block** — typically capacity 4→8 or 8→16, but often not until several steps in, because small `reallocs` frequently grow in place. Expect the first failure somewhere between index 4 and index 512, and it will not be at the same index on every run. ASan class: `heap-use-after-free, READ of size 4`, with a `freed by thread T0 here` frame pointing at `grow()`. The run-to-run variability is the lesson.
4. Naive: every operation reallocates, so 1,000,000 operations copy $\sim n$ elements each. With n around 65,536 that is $> 6 \times 10^6$ **element copies — expect tens of seconds** and roughly **1,000,000 reallocations**. Hysteresis version: **0 reallocations** during the alternating phase and single-digit milliseconds. The ratio should be in the thousands.
5. `pop` reads `head_->next`, which is `head_` itself, so `head_` never changes — but `delete d` has already freed it. Printing `top()` gives **garbage or the same value repeatedly**, and every iteration after the first is a use-after-free. The destructor's `while (head_)` **never terminates**.

- because `head_` is never null — an infinite loop of double-frees, which glibc usually stops with `free(): double free detected in tcache 2`. Note the crash site is the destructor, not `push`.
6. The walk-and-count assertion should pass, and it also catches the task-5 bug directly: the walk never terminates, so add a step limit of `size() + 1` and assert you reached null within it.
 7. Expected, per operation: reserved vector **0.8-1.5 ns**, growing vector **1.5-2.5 ns**, deque **2-4 ns**, NodeStack **20-45 ns**, `std::list` **30-60 ns**. Without the sink, the **array-backed rows collapse to ~0.0-0.1 ns** because the compiler deletes the loop entirely; the node rows shrink less, because `new/delete` have side effects the compiler must keep unless it can prove otherwise. That asymmetry is itself the trap: an unsinked benchmark makes nodes look *relatively better*.
 8. Growing: median **~1-2 ns**, p99 **~2-4 ns**, maximum **2-8 ms** — the final memcpy of ~32 MB at 8-16 GB/s. That six-order-of-magnitude gap between median and max is the shape of amortisation. After `reserve`: the maximum should drop to the **tens of nanoseconds**, with the single large allocation moved to setup.

HANDNOTE SUMMARY CARD

PART 2 - MEMORY AND REPRESENTATION

Topics 7-13

INVARIANT (assert all four)

n <= cap	data[0 .. n-1]	live
cap == 0 <=> data == nullptr	data[n .. cap-1]	raw storage, reading is UB

```
n MEANS THREE THINGS AT ONCE: live count | next-write index | one-past-the-top
every off-by-one is a confusion between two of them. top() = data[n-1].
store a COUNT, never a top POINTER (realloc dangles; empty needs data-1 = UB)
```

GROWTH	2x (libstdc++)	1.5x (MSVC)
reallocs to reach n	$\log_2 n = 20/1M$	$\log_{1.5} n = 35/1M$
total copies	$< n$	$\sim 2n$
peak memory	$2n$	$1.5n$
reuse own freed blocks?	NO ($1+2+\dots+2^k-1 < 2^k$)	YES after ~ 4 steps
constant +c growth	$n^2/(2c)$ copies	$<- 4800x$ worse at $n=1M, c=100$

AMORTISED PROOF (accounting) - charge 3 per push at 2x
 1 pays the write, 2 bank in the slot. At growth, half the elements were pushed since last growth, banking $2 * (\text{cap}/2) = \text{cap} = \text{exact copy cost}$. QED.
 general charge for factor k: $c = 1 + k/(k-1)$ $k=2 \rightarrow 3$ $k=1.5 \rightarrow 4$
 amortised != average. adversary CAN defeat average, CANNOT defeat amortised.
 single push worst case is still $O(n)$. `reserve()` makes it worst-case $O(1)$.

```
SHRINK - hysteresis or nothing
trigger n < cap/4, target cap/2. naive n < cap/2 -> alternating push/pop
reallocates every op -> O(m*n). Same reasoning as hash load factor / B-tree
merge threshold. std::vector NEVER shrinks: peak capacity is permanent.
release with: std::vector().swap(v);
```

```

NODE STACK                                push: head_ = new Node[v, head_];
16 B/int by sizeof (4+4pad+8ptr)         ^ next = OLD head, then move head
32 B/int as allocated (glibc chunk)      reversed order = self-cycle, infinite
~20 ns/push (malloc), 0(1) worst         loop in the DESTRUCTOR, not push
wins ONLY where one 0(n) push is fatal

```

CACHE - the two sentences that answer every array-vs-list question
 "one 64 B line delivers 16 ints from an array and 1 from a node stack"
 "the next node's address is known only after the current node arrives,
 so the prefetcher is blind" -- a dependent load chain
 measured ns/op: vec reserved 0.8-1.5 | vec grow 1.5-2.5 | deque 2-4
 NodeStack 20-45 | list 30-60 => 20-40x
 benchmark WITHOUT a volatile sink -> -O2 deletes the loop -> 0.1 ns = fake

FIXED CAPACITY - alignas(T) unsigned char buf_[N*sizeof(T)] + placement new
worst-case O(1), zero allocations, pop must call ~T() explicitly
converts a PAUSE into a REFUSAL (try_emplace -> false). Better failure.
as a local: keep under ~1 MB. FixedStack = 12 MB frame ->
segfault in the PROLOGUE, backtrace points at the opening brace.

NUMBERS

100 pushes doubling from 4: caps 4,8,16,32,64,128, 124 copies (1.24/elem)
1M ints vector: cap 1,048,576, 4.19 MB, 4.8% overshoot; 700k -> 33.2% unused
1M pops: array ~5 ms of line misses, nodes ~80 ms -> 16x before prefetch

PART THREE

Core operations

Each operation written properly, once. The exception-safety ordering inside push, the reason the standard library refuses to return a value from pop, the unsigned-underflow trap that eats an entire loop, and a test harness that finds the bugs unit tests do not.

14. push

`push` is three lines and every one of them can be wrong. Grow if needed, construct the element, then increment the count — in that order, and the order is a correctness requirement, not a style preference.

```
void push(const T& v) {
    if (n == cap) grow();      // 1. may throw bad_alloc -- stack unchanged
    new (data + n) T(v);       // 2. may throw T's copy ctor -- n not yet bumped
    ++n;                       // 3. cannot throw. Commit point.
}
```

Read it as a transaction. Steps 1 and 2 can fail; step 3 cannot. Because the count is bumped only after construction succeeds, a throwing copy constructor leaves the stack exactly as it was: same size, same elements, no half-built object in slot `n`. That is the **strong exception guarantee**, and it is free — it costs one line reordering.

Where the guarantee is lost

```
void push_wrong(const T& v) {
    if (n == cap) grow();
    ++n;                       // committed before the object exists
    new (data + n - 1) T(v);    // if this throws, slot n-1 holds garbage
}                               // and size() now counts a non-existent element
```

The damage from `push_wrong` shows up in the destructor, not at the push. The destructor runs `~T()` over `data[0..n-1]`, including the slot that was never constructed, and calls a destructor on uninitialised memory. For `std::string` that means `free()` on a garbage pointer.

push vs emplace

```
std::stack<std::pair<int, std::string>> s;
s.push({1, "hello"});           // build a pair, then move it in -- 1 extra move
s.emplace(1, "hello");          // construct in place from the arguments
```

For trivially copyable types the two compile to identical code. For types with allocation, `emplace` saves one move construction and one destructor call per push — roughly 8-15 ns for a small `std::string`, which matters only in a loop of millions but costs nothing to prefer.

push transaction, in order

```
state: n=3, cap=4      +----+----+----+----+
                       | a | b | c | .. |
                       +----+----+----+----+
                           ^ data+n
```

1. grow? `n != cap`, skip.
2. construct at `data+3`. If this `THROWS` -> `n` is still 3 -> stack intact.
3. `++n`. Commit. `n=4`. Now the element is visible.

reverse 2 and 3 and the throw leaves `n=4` with slot 3 uninitialised:
the `DESTRUCTOR` later calls `~T()` on garbage. Crash is far from the cause.

THE KEY INSIGHT

Put the operation that cannot throw last. Every strong-guarantee container in C++ is built on that single rule: do all the fallible work first, then commit with a `noexcept` operation. Reordering three lines is the difference between a container that is safe under exceptions and one that corrupts on the first `bad_alloc`.

THE TRAP

`grow()` that uses `memcpy` on a non-trivially-copyable `T`. It compiles, and for `std::string` it even appears to work — until two strings share a heap buffer and both destructors free it. Symptom: `double free or corruption (!prev)` at a point unrelated to the stack. Use `std::uninitialized_move`, or `std::is_trivially_copyable_v<T>` to select the fast path.

push – 0(1) amortised, 0(n) on the growing push · strong exception guarantee if `++n` is last ·
emplace saves one move+dtor (~8–15 ns for a small string)

15. pop and the value-return problem

Every beginner writes `T pop()`. Every standard library writes `void pop()` and makes you call `top()` first. The reason is the most-cited argument in C++ container design and it is a genuinely good interview question.

```
// The interface everyone wants:
T pop() {
    T result = data[n - 1];    // 1. copy out
    --n;                      // 2. remove
    return result;            // 3. copy again into the caller's slot <-- may throw
}
```

Step 3 constructs the return value in the caller's storage. If `T`'s copy constructor throws there — a `std::string` running out of memory, a class with a validating constructor — the element has *already* been removed at step 2 and the copy that would have delivered it has failed. The value is gone. There is no way for the caller to retry, because the stack no longer contains it. Tom Cargill's 1994 paper is the canonical statement of the problem, and it is why the standard splits the operation.

```
// The standard's answer:
if (!s.empty()) {
    T v = s.top();    // may throw -- stack still intact, retry is possible
    s.pop();          // cannot throw -- removal only
}
```

Four resolutions, and who uses each

Approach	Signature	Property
Split	<code>const T& top(); void pop();</code>	C++ standard. Safe, two calls, easy to misuse by forgetting one
Move-out	<code>T pop(); // T move-noexcept</code>	Safe <i>iff</i> <code>is_nothrow_move_constructible_v<T></code>
Out-parameter	<code>bool pop(T& out);</code>	Assigns into an existing object; no construction to fail
Optional	<code>std::optional<T> pop();</code>	Rust's <code>Vec::pop</code> ; still needs a nothrow move

```
// The modern safe version: enabled only when the move cannot throw.
template <class U = T,
        class = std::enable_if_t<std::is_nothrow_move_constructible_v<U>>>
T pop_value() {
    T v = std::move(data[n - 1]); // noexcept by the constraint above
    data[n - 1].~T();
    --n;
    return v;                      // guaranteed elision / nothrow move
}
```

THE KEY INSIGHT

The problem is not that returning a value is slow — it is that *removal and delivery are two operations that can fail independently*, and a single function cannot roll back the first when the second fails. C++11 move semantics fixed it: if moving out is `noexcept`, delivery cannot fail, so a value-returning pop becomes safe. That is the full answer to “why does `std::stack::pop` return void?” — and the follow-up is “because the interface predates the fix.”

THE TRAP

`int v = s.top(); s.pop();` written as `s.pop(); int v = s.top();`. It compiles, runs, and silently returns the wrong element every time — the one below the one you wanted — and on a one-element stack it is undefined behaviour rather than an error. The symptom is an off-by-one in the *result* with no crash. This is the most common stack bug in interview code; write the two lines as a unit and read them back.

pop – O(1), --n plus one destructor call · `void pop()` exists because remove-then-deliver cannot roll back · value-returning pop is safe only when the move is `noexcept`

16. top and reference invalidation

`top()` returns a *reference* into the buffer, which is why it is free. It is also why the reference has a lifetime, and the lifetime rules are the ones people get wrong.

```
std::stack<int> s;
s.push(1);
int& r = s.top(); // r points into the deque's current block
s.push(2);        // may reallocate (vector) or add a block (deque)
r = 99;           // vector backing: DANGLING WRITE. deque: still valid.
```

Backing	push invalidates <code>top()</code> ?	Why
<code>std::vector</code>	Yes , on reallocation	The whole buffer moves
<code>std::deque</code> (the default)	No	Existing blocks never move; only the block index is reallocated
<code>std::list</code>	No	Nodes never move
Your <code>FixedStack</code>	No	Storage never moves

This table is a trap generator. Code written against the default `std::deque` backing holds references across pushes and works perfectly. Someone later changes the backing to `std::vector` for

speed — a one-word change, measured, 2× faster — and every held reference becomes a use-after-free that only fires on reallocation boundaries.

vector backing, push causes reallocation

```

before:  r ---> +-----+
              | 1 | .. |      buffer at 0x1000
              +-----+
push(2) -> grow: new buffer at 0x2000, copy, FREE 0x1000

after:    r ---> 0x1000 (freed)      +-----+-----+-----+
              ^^^^^^^^^^^^^^      | 1 | 2 | .. | .. | at 0x2000
              dangling              +-----+-----+-----+

```

deque backing, push adds a block

```

before:  r ---> block A [1, ...]
push(2) -> block A has room, or a NEW block B is added.
          Block A is never moved.  r stays valid.

```

THE KEY INSIGHT

The safe habit is to treat `top()`'s result as valid only until the next mutation of the stack — regardless of backing. Copy it if you need it longer. That rule costs one `int` copy and makes your code correct under every backing, which means the future speed change is safe.

THE TRAP

`auto& x = s.top(); s.pop(); use(x);` — the reference outlives the element under *every* backing, because `pop` destroys the object. With `int` the memory is usually still readable and the old value is still there, so the bug passes all tests; with `std::string` it is a use-after-free that ASan catches immediately. Symptom: works in the integer test suite, crashes in production on strings. Use `auto x = s.top();` — by value.

`top` — $O(1)$, returns a reference, no copy · valid until the next mutation · vector backing invalidates on realloc, deque and list do not · copy by value unless you are in the same statement

17. size, empty, and the unsigned-underflow trap

`size()` and `empty()` are the two total operations, and they are where the most spectacular stack bug in C++ lives. It is not a stack bug strictly speaking — it is an unsigned arithmetic bug — but it appears in stack code more than anywhere else because stack loops are written in terms of `size`.

```

std::stack<int> s;                // empty. size() == 0
for (std::size_t i = 0; i < s.size() - 1; ++i) { /* ... */ }
//                               ^^^^^^^^^^^^^^
// 0 - 1 in size_t arithmetic is 18446744073709551615, not -1.
// The loop runs 1.8 x 10^19 times, indexing into nothing.

```

The value is $2^{64} - 1$. The loop does not fail fast — it runs until it segfaults on some unrelated memory, typically after several seconds, with a backtrace pointing far from the subtraction. Under `-fsanitize=undefined` nothing is reported either, because unsigned overflow is *defined* to wrap; it is not UB, it is just wrong.

Expression, with <code>size() == 0</code>	Value	Safe?
<code>s.size() - 1</code>	18446744073709551615	No
<code>s.size() >= 1</code>	false	Yes
<code>!s.empty()</code>	false	Yes — prefer this
<code>(int)s.size() - 1</code>	-1	Yes below 2 ³¹ elements
<code>s.size() > i + 1</code>	false	Yes — move the term, do not subtract

THE KEY INSIGHT

Never subtract from an unsigned size. Move the term to the other side: `i < s.size() - 1` becomes `i + 1 < s.size()`. The two are algebraically identical and only one of them is correct on an empty container. This single habit removes an entire bug class from your code.

THE TRAP

`while (s.size() >= 0)` as a drain loop. `size()` is unsigned, so the condition is **always true** — a compiler will usually warn with `-Wtype-limits`, but this appears in real code guarded by an inner `break` that a later refactor removes. Symptom: an infinite loop calling `pop()` on an empty stack, which under `std::stack` is UB and typically corrupts the container's internal size counter before crashing. Use `while (!s.empty())`.

size/empty — both O(1), both total · `size_t` is unsigned: `0 - 1 == 1.8×1019` · prefer `!empty()` · never subtract, move the term

18. clear, swap, and destructor cost

Removing every element is not free, and how expensive it is depends entirely on whether `T` has a destructor. This is the point where the “pop is just a decrement” simplification from Topic 2 stops being true.

```
void clear() {
    if constexpr (!std::is_trivially_destructible_v<T>)
        for (std::size_t i = n; i-- > 0; ) data[i].~T(); // note the idiom
    n = 0; // capacity retained -- see Topic 10
}
```

The loop header `for (std::size_t i = n; i-- > 0; )` is the standard way to count down over an unsigned type without hitting Topic 17: it tests `i > 0` and then decrements, so it stops at 0 and never wraps. Written as `for (std::size_t i = n - 1; i >= 0; --i)` it never terminates.

Element type	clear() of 1,000,000	Work done
int	~2 ns	One store to <code>n</code>
std::string (short, SS0)	~1-2 ms	1M destructor calls, no frees
std::string (long, heap)	~30-60 ms	1M <code>free()</code> calls
NodeStack<int>	~20-40 ms	1M <code>delete</code> plus 1M cache misses

swap: the O(1) operation people write as O(n)

```
void swap(Stack& o) noexcept {  
    std::swap(data, o.data);    // three pointer/integer exchanges.  
    std::swap(n, o.n);          // No element is touched.  
    std::swap(cap, o.cap);  
}
```

Swapping two million-element stacks costs 24 bytes of movement and zero element operations, because the elements never move — only the ownership of the buffer does. This is why `std::vector<int>().swap(v)` is the capacity-release idiom from Topic 10: the temporary takes ownership of the big buffer and destroys it at the end of the statement.

THE KEY INSIGHT

`swap` is O(1) and `noexcept` for every array-backed container, which makes it the standard tool for implementing the strong exception guarantee: build a complete new object off to the side, then swap it in with an operation that cannot fail. Copy-and-swap assignment (Topic 20) is exactly that pattern.

THE TRAP

`clear()` that only sets `n = 0` on a type with a destructor. Every element leaks: 1,000,000 `std::strings` with 40-byte buffers is **40 MB gone per clear**, and Valgrind reports it as “definitely lost” only if the stack itself is destroyed — otherwise it looks like ordinary growth. Symptom: memory rising in a loop that “clears everything each iteration.” The `is_trivially_destructible_v` branch above is not an optimisation, it is the correctness condition for skipping the loop.

clear – O(1) for trivial T, O(n) destructor calls otherwise · capacity retained · swap O(1)
noexcept, 24 bytes, elements never move · 1M heap strings cost 30–60 ms to destroy

19. Making it generic

Turning `IntStack` into `Stack<T>` is mostly mechanical, but three requirements on `T` emerge that were invisible while the element type was `int`, and naming them is a common interview follow-up.

```

template <class T, class Container = std::vector<T>>
class Stack {
    Container c_;
public:
    using value_type = typename Container::value_type;
    using size_type = typename Container::size_type;

    bool      empty() const { return c_.empty(); }
    size_type size()  const { return c_.size(); }
    T&        top()   { return c_.back(); }
    const T&   top()   const { return c_.back(); }

    void push(const T& v) { c_.push_back(v); }
    void push(T&& v)      { c_.push_back(std::move(v)); }
    template <class... A>
    decltype(auto) emplace(A&&... a) {
        return c_.emplace_back(std::forward<A>(a)...);
    }
    void pop() { c_.pop_back(); }
};

```

That is the **container adaptor** pattern, and it is exactly what `std::stack` is: not a container at all, but a wrapper that narrows another container's interface to five operations. The adaptor holds the container by value in a protected member named `c` — a detail the standard actually specifies, which is why you can derive from `std::stack` to reach the underlying container.

What the adaptor requires of its container

Requirement	Satisfied by
<code>back()</code>	vector, deque, list — <i>not</i> forward_list
<code>push_back()</code>	vector, deque, list
<code>pop_back()</code>	vector, deque, list
<code>empty()</code> , <code>size()</code>	all of them

Note what is *not* required: no iterators, no `operator[]`, no contiguity. That is why the adaptor can sit on a `std::list` at all, and why swapping backings is a one-word change.

THE KEY INSIGHT

Writing the stack as an adaptor rather than a container means you inherit someone else's growth policy, exception safety, allocator support and move semantics for free — roughly 300 lines of subtle code you no longer have to get right. Hand-roll the storage only when you need something the standard containers do not offer, which in practice means fixed capacity (Topic 13) or lock-freedom (Topic 69).

THE TRAP

Forgetting the `T&&` overload of `push`. With only `push(const T&)`, every `s.push(std::move(big))` silently copies instead of moving. There is no error and no warning; the symptom is a profile showing time in `memcpy` and an allocation count double what you expected. For a stack of 1M 100-byte strings that is **100 MB of avoidable copying**.

adaptor – all operations forward, O(1) · requires only back/push_back/pop_back/empty/size · inherits growth, exception safety and allocators from the container · missing the && overload silently copies

20. Rule of five for the array stack

If you own a raw buffer, the compiler's generated copy operations are wrong: they copy the pointer, so two stacks own the same memory and both free it. Writing the five special members correctly is a standard interview exercise, and the copy-and-swap idiom collapses two of them into one line.

```
Stack(const Stack& o)                                // 1. copy ctor
: data(static_cast<T*>(::operator new(o.cap * sizeof(T)))),
  n(0), cap(o.cap) {
  try {
    for (; n < o.n; ++n) new (data + n) T(o.data[n]); // may throw
  } catch (...) {
    while (n) data[--n].~T();                          // unwind what we built
    ::operator delete(data);
    throw;
  }
}

Stack(Stack&& o) noexcept                             // 2. move ctor
: data(o.data), n(o.n), cap(o.cap) {
  o.data = nullptr; o.n = 0; o.cap = 0;                // leave o destructible
}

Stack& operator=(Stack o) noexcept {                  // 3+4. by VALUE
  swap(o);                                             // copy-and-swap
  return *this;                                       // o's dtor frees the old
}

~Stack() {                                           // 5. destructor
  clear();
  ::operator delete(data);
}
```

Taking the assignment parameter **by value** is the trick. The caller's argument is copied or moved into `o` by the ordinary overload rules, so one function handles both copy-assign and move-assign; the body is a `noexcept` swap; and self-assignment works without a check because `o` is a separate object.

copy-and-swap on assignment: `a = b`

1. `b` is COPIED into the parameter `o`. If this throws, `a` is untouched.
a: [x y z] b: [p q] o: [p q]
2. `swap(o)` Cannot throw. Commit.
a: [p q] b: [p q] o: [x y z]
3. `o`'s destructor runs at the closing brace, freeing the OLD contents of `a`.
a: [p q] b: [p q]

Strong guarantee, self-assignment safe, one function for two operators.

THE KEY INSIGHT

Mark the move constructor `noexcept`. If you do not, `std::vector` reallocation refuses to move your objects and copies them instead — because it must preserve the strong guarantee and a throwing move cannot be rolled back. The symptom is a container that is inexplicably 10× slower on growth with no code change nearby. One keyword.

THE TRAP

A move constructor that steals the pointer but does not null the source. The moved-from object still holds `data`, so both destructors free the same buffer. Symptom: `free(): double free detected in tcache 2` at scope exit, with a backtrace that names neither the move nor the original owner. The rule is that a moved-from object must be left in a *valid, destructible* state — not merely unused.

rule of five – copy $O(n)$ with unwind-on-throw · move $O(1)$, must be `noexcept` or vector will copy instead · copy-and-swap gives both assignments, self-assignment safety and the strong guarantee in three lines

21. Testing: invariants, fuzzing, instrumentation

Unit tests on a stack find the bugs you thought of. The bugs that survive are in operation *sequences* — a push at a capacity boundary followed by a pop that shrinks, an exception during a copy during a growth. Three techniques find those, and all three are short.

1. The invariant checker

```
void check() const {
    assert(n <= cap);
    assert((cap == 0) == (data == nullptr));
    assert(cap == 0 || cap >= 4);           // growth policy floor
}
// Call it at the top and bottom of every public operation in debug builds.
```

2. The model-based fuzz loop

```
// Run the real stack and a std::vector "model" through the same random
// operations and compare after every step. Any divergence is a bug, and
// the operation log is the reproducer.
std::mt19937 rng(12345);                     // FIXED seed -- reproducible
Stack<int> s; std::vector<int> model;
for (int step = 0; step < 1'000'000; ++step) {
    int op = rng() % 10;                      // 70% push, 30% pop
    if (op < 7) { int v = rng(); s.push(v); model.push_back(v); }
    else if (!model.empty()) {
        assert(s.top() == model.back());
        s.pop(); model.pop_back();
    }
    assert(s.size() == model.size());
    s.check();
}
```

3. The throwing-type harness

```
// Constructs successfully N times, then throws. Sweep N from 0 upward
// and assert the stack is still consistent after every failure point.
struct Bomb {
    static inline int countdown = -1;
    int v;
    Bomb(int x) : v(x) { if (countdown >= 0 && countdown-- == 0) throw 1; }
    Bomb(const Bomb& o) : v(o.v) { if (countdown >= 0 && countdown-- == 0) throw 1; }
};
```

Sweeping `Bomb::countdown` from 0 to 200 and copying a stack after each setting exercises every possible throw point in the copy constructor. This is how you actually verify the unwind loop in Topic 20, and it is the technique libstdc++'s own test suite uses.

THE KEY INSIGHT

A model-based fuzz test is about fifteen lines and finds more container bugs than a hundred hand-written unit tests, because it explores operation *sequences* rather than operations. Fix the seed so failures reproduce, and print the operation index on assertion failure so you can bisect.

THE TRAP

`assert` compiled out by `-DNDEBUG` in the build you actually test. Release builds define it by default in most CMake configurations, so `check()` becomes an empty function and your entire invariant harness silently disappears. Symptom: a test suite that passes in Release and fails in Debug — or worse, one that passes in both because it is checking nothing. Verify by making one assertion deliberately false and confirming the build you ship actually aborts.

testing – invariant check $O(1)$ per call · model fuzz 10^6 steps in ~ 1 s · Bomb sweep is $O(\text{throw points}) \times O(n)$ per run · verify `NDEBUG` has not deleted your assertions

PRACTICE SET — Topics 14-21

A. Conceptual

1. Write the three lines of `push` in the correct order and state which line is the commit point. What property of that line makes it the commit point?
2. What exactly goes wrong if `++n` precedes the placement new, and *where* does the resulting crash appear?
3. Explain the value-return problem in `T pop()` in terms of two operations that can fail independently. Why does C++11 partially fix it?
4. Give the constraint under which a value-returning `pop` is safe, as a type trait.
5. Under which backings does `push` invalidate a reference previously returned by `top()`? Explain the deque case specifically.
6. Why is `auto& x = s.top(); s.pop(); use(x);` wrong under *every* backing, and why does it nonetheless pass a test suite that uses `int`?
7. Compute `std::size_t(0) - 1` and explain why `-fsanitize=undefined` does not report it.
8. Rewrite `i < s.size() - 1` so that it is correct on an empty stack, without a cast.
9. Why does `for (size_t i = n - 1; i >= 0; --i)` never terminate? Give the idiom that does.
10. Under what condition may `clear()` skip the destructor loop entirely? Name the trait.
11. Why is `swap` on two million-element stacks $O(1)$? How many bytes move?
12. State the exact requirements `std::stack` places on its container. Which standard container fails them, and on which requirement?
13. What happens to `std::vector`'s growth path if your move constructor is not marked `noexcept`, and why is that the standard's choice rather than a bug?
14. Explain why copy-and-swap assignment handles self-assignment correctly without an `if (this == &o)` check.
15. Why does a model-based fuzz test find bugs that unit tests miss? Answer in terms of what is being explored.

B. Tracing and analysis

1. A stack of `std::string` has `n = 5`, `cap = 8`. The copy constructor of the sixth string throws. Give the values of `n` and `cap` afterwards, and the number of live objects, under (a) the correct push order and (b) the wrong order.
2. Count the constructor and destructor calls for `s.push(std::pair<int, std::string>{1, "hello"})` versus `s.emplace(1, "hello")`, assuming no reallocation.
3. Given `std::size_t` is 64-bit, write out `s.size() - 1` in decimal for an empty stack, and state how many iterations the loop in Topic 17 performs before it touches unmapped memory, assuming it indexes 4 bytes per step into a 4 KB page mapping starting at the buffer.
4. For a stack of 1,000,000 elements, estimate `clear()` time for `int`, short `std::string`, long `std::string`, and `NodeStack<int>`. Give the ratio of the slowest to the fastest.
5. Trace copy-and-swap for `a = b` where `a` holds `[x,y,z]` and `b` holds `[p,q]`. Give the contents of `a`, `b` and the parameter at each of the three steps, and say which destructor frees which buffer.
6. In the `Bomb` harness, a stack holds 100 elements and you are copying it. How many distinct values of `countdown` must you sweep to cover every throw point in the copy constructor, and what should the invariant check report at each?

7. The move constructor steals the pointer but forgets `o.data = nullptr`. Both objects go out of scope at the end of a function. State exactly what happens and what glibc prints.
8. A profile shows 1,000,000 calls to `std::string`'s copy constructor in a loop that only ever calls `s.push(std::move(str))`. Give the single missing declaration and the bytes copied for 100-byte strings.

C. Code tasks

1. **Build.** Implement `Stack<T>` with the full rule of five from Topic 20, plus `check()` from Topic 21. Verify with `Stack<std::string>`: push 5, copy the stack, pop from the copy, confirm the original is unchanged.
2. **Break it — the commit-order bug.** Swap the `++n` and placement-new lines. Then push a type whose copy constructor throws on the third element. Run under `-fsanitize=address` and record where the error is reported. Predict first: is it reported at the push, or at the destructor?
3. **Break it — the dangling top.** Write a loop that holds `int& r = s.top();`, then pushes 10,000 elements, writing `r = i;` each iteration. Run with `std::vector` backing and then with `std::deque` backing. Report which one ASan flags and the error class. Explain the difference using the Topic 16 table.
4. **Break it — unsigned underflow.** Write the `for (size_t i = 0; i < s.size() - 1; ++i)` loop over an empty stack. Time how long it runs before crashing, and record whether `-fsanitize=undefined` reports anything. Then fix it by moving the term and confirm zero iterations.
5. **Break it — the leaking clear.** Implement `clear()` as `n = 0;` with no destructor loop. Push 1,000,000 long `std::strings`, `clear`, and repeat ten times while watching RSS. Report the memory growth and what `valgrind --leak-check=full` says while the stack object is still alive versus after it is destroyed.
6. **Break it — the missing noexcept.** Remove `noexcept` from your move constructor. Push 1,000,000 `Stack<std::string>` objects into a `std::vector` and time it. Restore `noexcept` and time it again. Report both figures and the ratio.
7. **Fix and verify.** Run the model-based fuzz loop from Topic 21 for 10,000,000 steps against your fixed implementation. Confirm zero assertion failures, then deliberately reintroduce the Topic 15 `pop`-before-`top` ordering bug and report the step index at which the fuzzer catches it.
8. **Verify your assertions exist.** Make `check()` assert something false. Build in your project's default Release configuration and confirm the program actually aborts. If it does not, report which flag deleted your test harness.

ANSWER KEY — Topics 14-21

A. Conceptual

1. `if (n == cap) grow(); / new (data + n) T(v); / ++n;`. The commit point is `++n`, because it **cannot throw** — incrementing an integer has no failure mode, so once it executes the transaction is irrevocably complete.
2. The count claims an element that was never constructed. Nothing goes wrong at the push. The crash appears **in the destructor** (or in `clear`), which runs `~T()` over `data[0..n-1]` including the uninitialised slot — for `std::string` that is `free()` on a garbage pointer.
3. **Removal** (decrementing the count) and **delivery** (constructing the return value in the caller's storage) can each fail, and a single function cannot undo the first when the second fails. C++11 fixes it because a `noexcept` move makes delivery infallible, so there is nothing left to roll back.
4. `std::is_nothrow_move_constructible_v<T>`.
5. Only `std::vector`, and only on reallocation. `std::deque` never moves an existing block — growth adds a new block and reallocates only the *block index*, which holds pointers, not elements — so references to elements stay valid.
6. Because `pop` **destroys the object** the reference names, regardless of where the storage lives. It passes an `int` test suite because destroying an `int` is a no-op and the memory is still mapped and still holds the old bit pattern — the read is UB but returns the expected value.
7. **18446744073709551615** ($2^{64} - 1$). UBSan does not report it because unsigned overflow is **defined** to wrap modulo 2^n by the standard; it is not undefined behaviour, merely wrong. `-fsanitize=unsigned-integer-overflow` in Clang catches it, but it is not part of the default `undefined` group.
8. `i + 1 < s.size()`.
9. Because `i` is unsigned, so `i >= 0` is always true; after `i == 0` the decrement wraps to 18446744073709551615. Idiom: `for (size_t i = n; i-- > 0; )`.
10. When `std::is_trivially_destructible_v<T>` is true.
11. Because only the three header fields are exchanged — a pointer and two `size_t`s. On x86-64 that is **24 bytes**. No element is read, written, constructed or destroyed; ownership of the buffer moves, the buffer does not.
12. `back()`, `push_back()`, `pop_back()`, `empty()`, `size()`. `std::forward_list` fails — it is singly linked, so it has no `back()` and no `push_back()/pop_back()`.
13. Reallocation falls back to **copying** every element instead of moving. It is not a bug: `vector` promises the strong guarantee on growth, and a throwing move halfway through cannot be undone, whereas a throwing copy can be — the originals are still intact.
14. Because the parameter `o` is a **separate object** constructed from the argument before the body runs. On `a = a`, the copy is made first, then swapped in; nothing is destroyed before it is copied.
15. Because it explores **operation sequences and their interleavings**, not individual operations. Bugs at capacity boundaries, in shrink hysteresis, and in exception unwinding only appear in particular sequences that nobody writes by hand.

B. Tracing and analysis

1. (a) Correct order: `n = 5`, `cap = 8`, **5 live objects** — the stack is exactly as before the failed push. (b) Wrong order: `n = 6`, `cap = 8`, **5 live objects** — the count claims a sixth that does not exist, and the destructor will call `~string()` on raw memory.

2. `push` with a braced temporary: **1 pair construction (which constructs 1 string) + 1 move construction of the pair + 1 destructor** for the temporary. `emplace`: **1 pair construction, no move, no destructor**. Net saving: one move and one destructor per push.
3. **18446744073709551615**. Indexing 4 bytes per step from the buffer, a 4096-byte mapping is exhausted after **1024 iterations** — but in practice the heap is surrounded by other mapped pages, so it typically runs for millions of iterations, often seconds, before reaching an unmapped page. That distance between cause and crash is the lesson.
4. `int` **~2 ns**; short string **~1-2 ms**; long string **~30-60 ms**; `NodeStack<int>` **~20-40 ms**. Slowest to fastest ratio is roughly **10⁷×** — seven orders of magnitude between clearing ints and clearing heap strings.
5. Step 1: `a=[x,y,z]`, `b=[p,q]`, `o=[p,q]` (copy). Step 2 after swap: `a=[p,q]`, `b=[p,q]`, `o=[x,y,z]`. Step 3: `o`'s destructor runs at the closing brace and frees **the buffer that used to belong to a**. `a` now owns the buffer built in step 1.
6. **101 values** — `countdown` from 0 to 100 inclusive, covering a throw at each of the 100 copy constructions plus the no-throw case. At every setting the invariant check on the *source* stack must report **unchanged: n = 100, cap unchanged**, and the partially-built destination must have been fully unwound and its buffer freed.
7. Both objects hold the same `data` pointer, so both destructors call `::operator delete` on it. `glibc` prints `free(): double free detected in tcache 2` and aborts. The backtrace names the second destructor, which is neither the move site nor the original construction site.
8. Missing: `void push(T&& v) { c_.push_back(std::move(v)); }`. Without it the rvalue binds to `const T&` and is copied. At 100 bytes per string that is **100 MB** of avoidable `memcpy` plus 1,000,000 avoidable allocations.

C. Code tasks — what to verify

1. The copy must be independent: popping from the copy leaves the original at size 5 with identical contents. If the original also shrinks, you have the compiler-generated shallow copy and are heading for a double free.
2. Prediction: **reported at the destructor, not the push**. Expected ASan output class is `heap-use-after-free` or `attempting free on address which was not malloc()-ed`, with frame #0 inside `std::string`'s destructor and frame #1 in your `clear()`. The push itself completes silently. Distance between cause and symptom is the whole lesson of Topic 14.
3. `std::vector` backing: ASan reports `heap-use-after-free, WRITE of size 4`, with `freed by thread T0 here` pointing at the reallocation inside `push_back`. `std::deque` backing: **no error** — existing blocks never move. The pair of runs is the demonstration that this bug is a property of the *backing choice*, not of your code, which is exactly why the safe habit must not depend on the backing.
4. Typical: it runs for **hundreds of milliseconds to several seconds** before segfaulting, and `-fsanitize=undefined` reports **nothing at all**. After the fix, **zero iterations**. Two lessons: the sanitiser you trust does not cover this, and the crash site is unrelated to the bug site.
5. Expect RSS to climb by roughly **the full payload each round** — for 1M strings of ~40 heap bytes, about **40 MB per clear**, so ~400 MB after ten rounds. Valgrind while the stack is alive reports the blocks as **“still reachable”** (because `data` still points at the buffer) and only calls them **“definitely lost”** once the stack is destroyed. A leak that hides behind “still reachable” is the hardest kind to see in a report.
6. Without `noexcept` the vector **copies** on every growth; with it, it **moves**. Expect a ratio between **5×** and **30×** depending on string length, with the allocation count differing by roughly the number of elements copied across all growth steps.

7. Ten million steps should complete in around a second with zero failures. With the `pop-before-top` bug reintroduced, the fuzzer should fail on the **first pop it performs** — typically within the first ten steps — at the `assert(s.top() == model.back())` line. That immediacy is the argument for model-based testing.
8. If the program does not abort, the flag is `-DNDEBUG`, which CMake adds to `Release` and `RelWithDebInfo` by default. The fix is either to test in a configuration without it or to replace `assert` with a macro that survives, such as one that calls `std::abort()` unconditionally in the builds you actually ship.

HANDNOTE SUMMARY CARD

PART 3 - CORE OPERATIONS

Topics 14-21

PUSH - three lines, order is a CORRECTNESS requirement

```
if (n == cap) grow();      // may throw bad_alloc -> stack unchanged
new (data + n) T(v);      // may throw T's ctor  -> n not yet bumped
++n;                      // CANNOT throw      -> COMMIT POINT
reversed 2/3: n counts a non-existent object; ~T() on garbage LATER,
in the destructor, far from the cause. Rule: the nothrow op goes LAST.
emplace saves 1 move + 1 dtor per push (~8-15 ns for a small string).
```

POP - why the standard returns void (Cargill)

```
T pop() { T r = data[n-1]; --n; return r; }  <- step 3 may throw AFTER
removal committed; the value is lost and cannot be retrieved.
safe form requires: std::is_nothrow_move_constructible_v
THE COMMON BUG: s.pop(); v = s.top(); returns the element BELOW.
write "v = s.top(); s.pop();" as one unit.
```

TOP - a reference with a lifetime

backing	push invalidates top()?	why
vector	YES on realloc	whole buffer moves
deque (default)	NO	blocks never move, only the index
list / Fixed	NO	storage never moves

auto& x = s.top(); s.pop(); use(x); <- wrong under EVERY backing
habit: valid only until the next mutation. copy by value.

SIZE / EMPTY - the unsigned trap

```
size_t(0) - 1 == 18446744073709551615    UBSan reports NOTHING (defined wrap)
i < s.size() - 1    -> i + 1 < s.size()  <- move the term, never subtract
while (s.size() >= 0) is always true
countdown idiom: for (size_t i = n; i-- > 0; )
```

CLEAR / SWAP

```
clear: skip the dtor loop only if is_trivially_destructible_v
      1M: int ~2 ns | short str ~1-2 ms | heap str 30-60 ms | nodes 20-40 ms
      capacity is RETAINED (T10)
swap : 0(1) noexcept, 24 bytes, elements never move
      basis of the strong guarantee: build aside, swap in with a nothrow op
      release capacity: std::vector().swap(v);
```

ADAPTOR - std::stack is NOT a container

```
requires back(), push_back(), pop_back(), empty(), size()
forward_list FAILS (no back/push_back/pop_back)
no iterators, no operator[], no contiguity required
MISSING push(T&&) -> every std::move silently COPIES. no warning.
```

RULE OF FIVE

```
copy ctor  0(n), unwind partially-built elements in catch, rethrow
move ctor  0(1), MUST be noexcept or vector copies on growth (5-30x slower)
           MUST null the source or both dtors free the same buffer
operator=(Stack o) by VALUE + swap -> both assignments, self-assign safe,
           strong guarantee, three lines
dtor       clear() then ::operator delete
```

TESTING

1. check(): n<=cap; (cap==0)==(data==nullptr); assert at entry AND exit
2. model fuzz vs std::vector, FIXED seed, 10^6 steps ~1 s - finds sequence bugs
3. Bomb type: throws on the Nth construction; sweep N over every throw point

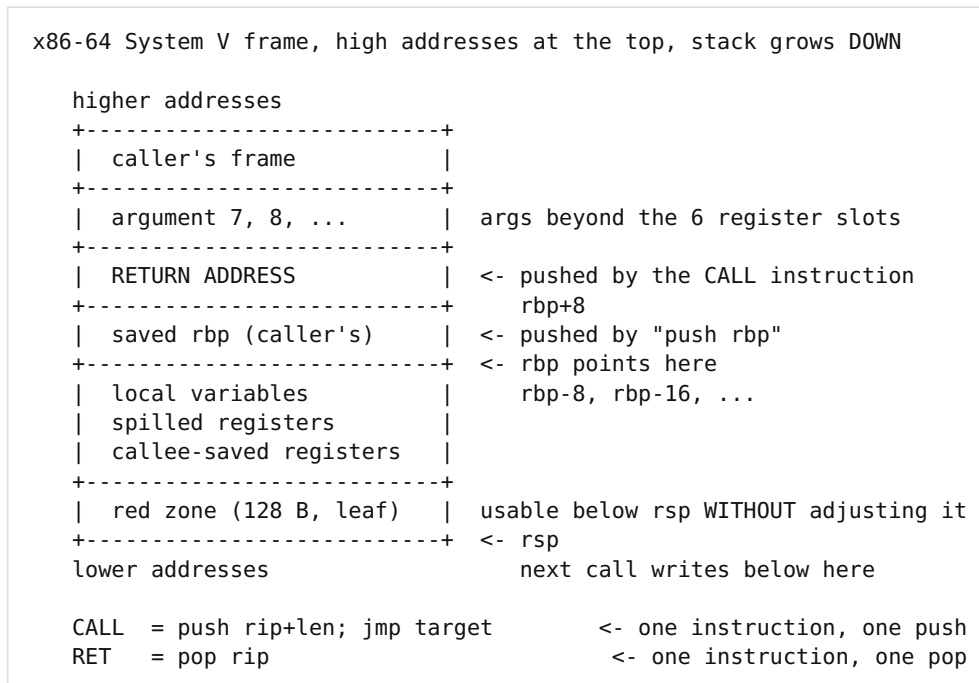
VERIFY -DNDEBUG has not deleted your assertions: make one false, ship it, run.

The call stack

The other stack — the one the CPU manages, that you did not write and cannot grow. Frame layout in real disassembly, the mechanical recipe for converting recursion into an explicit stack, and the arithmetic that tells you exactly how deep you can go before the guard page kills you.

22. Frame anatomy

Every function call pushes a **stack frame**: a contiguous block holding the return address, the caller's saved registers, the function's local variables, and space for arguments to functions it will call. On x86-64 the frame is delimited by two registers — **rsp** (stack pointer, the current top) and optionally **rbp** (base pointer, a fixed handle on the frame's start).



Read the diagram once and the phrase “a function call is a push” stops being a metaphor. The **call** instruction literally pushes the return address onto the stack pointed to by **rsp**, and **ret** literally pops it into the instruction pointer. LIFO is enforced by hardware, and the correctness of every program depends on it.

What decides the frame size

Contributor	Typical bytes
Return address	8
Saved rbp (omitted at -02)	0 or 8
Each long /pointer local that must live in memory	8
Callee-saved registers actually used (rbx , r12–r15)	8 each
Local arrays	their full size
Alignment to 16 bytes at the call boundary	0–15

THE KEY INSIGHT

Locals that stay in registers cost *nothing* on the stack. A recursive function with four **int** locals may have a 16-byte frame at **-02** and a 64-byte frame at **-00**, which changes your maximum recursion depth by 4×. This is why “how deep can I recurse?” has no answer without naming the optimisation level.

THE TRAP

Taking the address of a local (`&x`, or passing it by reference) forces it out of a register and into the frame. A recursive helper that takes `std::vector<int>& path` plus a couple of local references can quietly triple its frame size relative to the version that passes by value in registers. Symptom: a stack overflow that appears after a refactor that “only changed a parameter to a reference to avoid a copy.”

frame – CALL/RET are one push and one pop, hardware-enforced · 16–64 bytes typical, dominated by memory-resident locals · -O0 frames are 2–4× -O2 frames · red zone 128 B for leaf functions

23. Reading a frame in disassembly

You do not need to write assembly to benefit from reading it. Twenty seconds with `objdump` tells you a function's exact frame size, which is the input to every calculation in Topic 28.

```
// frame.cpp
int sum(int n) {
    int acc = 0;
    char pad[64];           // forces a real frame
    pad[0] = (char)n;
    for (int i = 0; i <= n; ++i) acc += i;
    return acc + pad[0];
}
// g++ -O2 -c frame.cpp && objdump -d --no-show-raw-insn frame.o
```

```
0000000000000000 <_Z3sumi>:
    0:  sub    $0x58,%rsp        <-- THE FRAME SIZE: 0x58 = 88 bytes
    4:  mov     %edi,%eax
    6:  mov     %dil,(%rsp)
    ...
   2a:  add     $0x58,%rsp        <-- restore
   2e:  ret
```

The single number you want is the operand of the `sub` on `%rsp` in the prologue. Add 8 for the return address pushed by the caller's `call`, and 8 more if you see `push %rbp`. Here: $88 + 8 = 96$ **bytes per frame**.

Two faster ways to get the same number

```
# 1. Ask the compiler directly -- prints a warning per function over the limit
g++ -O2 -Wframe-larger-than=64 frame.cpp
#   warning: the frame size of 88 bytes is larger than 64 bytes

# 2. Ask at runtime -- the difference between two frames' addresses
void probe(int d) {
    char here;
    static char* prev = nullptr;
    if (prev) printf("frame = %td bytes\n", prev - &here);
    prev = &here;
    if (d) probe(d - 1);
}
```

THE KEY INSIGHT

`-Wframe-larger-than=N` turns “is this function safe to recurse?” into a compile-time check. The Linux kernel builds with `-Wframe-larger-than=1024` for exactly this reason: its threads have 8 or 16 KB stacks, so a 2 KB frame is a bug. Set it in any project that recurses over user-controlled data.

THE TRAP

Measuring frame size at `-O0` and planning depth from it, then shipping `-O2` — or the reverse, which is worse. Debug builds have frames 2–4× larger, so an algorithm tuned to just fit at `-O2` overflows the moment someone builds it in Debug to investigate a bug. Symptom: “it only crashes in the debugger.” Compute your depth budget from the *largest* frame across your build configurations.

measuring frames — `objdump`: read the `sub $N,%rsp`, add 8 for the return address · `-Wframe-larger-than=N` at compile time · address-difference probe at runtime · `-O0` is 2–4× `-O2`

24. Recursion is a stack you did not write

Recursion has no independent existence. It is a stack — specifically, the call stack — with the pushes and pops written by the compiler instead of by you. Once that is concrete, converting between the two forms becomes mechanical rather than clever.

```
int fact(int n) {
    if (n <= 1) return 1;      // base case
    return n * fact(n - 1);    // the multiply happens AFTER the call returns
}
```

fact(4): what the call stack holds at maximum depth

frame	n	pending work after the callee returns	
fact(1)	1	(base case, returns 1 immediately)	<- top, rsp here
fact(2)	2	multiply the result by 2	
fact(3)	3	multiply the result by 3	
fact(4)	4	multiply the result by 4	<- bottom
main		use the result	

Unwinding, bottom of the diagram last:

```
fact(1) -> 1
fact(2) -> 2 * 1 = 2
fact(3) -> 3 * 2 = 6
fact(4) -> 4 * 6 = 24
```

The “pending work” column is exactly what an explicit stack must store. Four frames, ~32 bytes each at `-O2` = ~128 bytes of call stack for `n=4`.

Read the pending-work column carefully, because it is the whole conversion recipe. Recursion is not storing the numbers 4, 3, 2, 1 — it is storing *four suspended multiplications*, each waiting for a value that does not exist yet. Any explicit-stack version must store the same suspended work.

THE KEY INSIGHT

The question “what does my explicit stack need to hold?” has one answer: **everything the frame held that is still needed after the recursive call returns**. Not the parameters you passed — the parameters you will still need. In `fact` that is `n`; in a tree traversal it is the node pointer and which child you were on (Topic 26).

THE TRAP

Assuming a recursive function's memory cost is $O(1)$ because it declares no containers. `fact(1000000)` allocates a million frames — with a 32-byte frame that is 32 MB against an 8 MB limit, so it dies at roughly depth 262,000. The symptom is a `SIGSEGV` with a backtrace showing hundreds of identical frames, which gdb often truncates with `(More stack frames follow...)`. Recursion depth is space complexity: $O(\text{depth})$.

recursion — Time $O(n)$ · Space $O(\text{depth})$ on the *call* stack, ~32 bytes per frame · the stack stores suspended work, not values · depth is a hard limit, not a soft one

25. Recursion to explicit stack: the recipe

The conversion is four mechanical steps. Learn it once and it applies to tree traversal, DFS, quicksort, backtracking and expression parsing — every recursive algorithm in Parts 5 to 9.

Step	What to do
1	Define a frame struct holding every variable still live after the recursive call
2	Push the initial frame; loop <code>while (!st.empty())</code>
3	Replace “call myself” with “push a new frame and <code>continue</code> ”
4	Replace “return v” with “pop, deliver v to the frame now on top”

The simple case: one recursive call in tail position

```
// Recursive
void printDown(int n) {
    if (n == 0) return;
    std::cout << n << ' ';
    printDown(n - 1);          // tail position: nothing pending afterwards
}

// Explicit -- degenerates to a loop, because the frame holds nothing
void printDown_iter(int n) {
    while (n != 0) { std::cout << n << ' '; n = n - 1; }
}
```

The general case: work pending after the call

```
// Recursive
int fact(int n) { return n <= 1 ? 1 : n * fact(n - 1); }

// Explicit: the pending multiplications become stack entries
int fact_iter(int n) {
    std::stack<int> pending;           // the "multiply by k" jobs
    while (n > 1) { pending.push(n); --n; }
    int result = 1;                   // the base case's return value
    while (!pending.empty()) {        // unwind, applying pending work
        result *= pending.top();
        pending.pop();
    }
    return result;
}
```

Compare the two loops against the diagram in Topic 24: the first loop builds the pending-work column top-down, the second consumes it bottom-up. That is exactly what the hardware was doing, made visible.

THE KEY INSIGHT

If the frame struct comes out empty, the recursion was a loop in disguise — convert it to a `while` and stop. If it holds one field, you get a single `std::stack<T>`. If it holds a resumption point as well, you have the general case and need Topic 26. Deciding which of the three you are in is the first thing to do, and it takes ten seconds.

THE TRAP

Converting to an explicit stack and then declaring it as a local array: `int pending[1000000];` inside the function. You have moved the data from the call stack to... the call stack. The whole benefit is that `std::stack` allocates on the heap. Symptom: the identical segfault you were trying to eliminate, now in a function that looks iterative. See Topic 13.

conversion — Time unchanged · Space moves from the 8 MB call stack to the heap · empty frame → plain loop · one field → one stack · resumption point → Topic 26

26. The resumption-point problem

With one recursive call, the explicit stack holds data. With *two*, it must also hold *where in the function to resume* — because after the first callee returns you must run the code between the two calls, and after the second you must run the code after both. The call stack encodes this in the return address; an explicit stack has to encode it as a field.

```
// Recursive: three distinct resumption points
void inorder(Node* r) {
    if (!r) return;
    inorder(r->left);           // resume at (A) when this returns
    visit(r);                   // (A)
    inorder(r->right);          // resume at (B) when this returns
}                               // (B) -- done
```

```
// Explicit, general form: a state field IS the return address
struct Frame { Node* node; int stage; }; // stage: 0 = entered,
                                         //         1 = left done,
                                         //         2 = right done

void inorder_iter(Node* root) {
    std::stack<Frame> st;
    if (root) st.push({root, 0});
    while (!st.empty()) {
        Frame& f = st.top(); // reference: we mutate stage in place
        if (f.stage == 0) {
            f.stage = 1;
            if (f.node->left) st.push({f.node->left, 0});
        } else if (f.stage == 1) {
            visit(f.node);
            f.stage = 2;
            if (f.node->right) st.push({f.node->right, 0});
        } else {
            st.pop();
        }
    }
}
```

The stage field is the program counter, made into data

recursive	explicit
-----	-----
return address -> just after the first recursive call	stage == 1 -> "left child is done, now visit and go right"
return address -> just after the second recursive call	stage == 2 -> "both children done, pop me"
Number of stages needed = number of recursive call sites + 1.	
1 call -> 2 stages, often collapsible to no field at all	
2 calls -> 3 stages (inorder, quicksort, merge sort)	
k calls -> k+1 stages	

This general form always works and is worth knowing, but for binary tree traversal there are shorter specialised versions — Topic 63 gives the classic pointer-walking inorder that needs no stage field, at the cost of being much harder to derive from scratch. Under interview pressure the general form is the safer answer, because you can produce it mechanically.

THE KEY INSIGHT

A return address is a program counter value. When you make the call stack explicit you must make the program counter explicit too, and the cheapest encoding is a small integer naming which recursive call site you are returning from. Every “how do I do postorder iteratively?” question is this one observation.

THE TRAP

`Frame& f = st.top();` followed by `st.push(...)` in the same iteration. With `std::vector` backing, the push may reallocate and `f` dangles — and the very next loop iteration reads it. This is Topic 16 in its most dangerous setting, because the code looks obviously correct and the default `std::deque` backing hides it completely. Either copy the frame by value first, or re-fetch `st.top()` after every push.

resumption points – Time $O(n)$ · Space $O(\text{depth})$ on the heap · stages = recursive call sites + 1 · the stage field is the return address · never hold a reference to `top()` across a `push`

27. Tail calls and why they do not save you

A **tail call** is a recursive call whose result is returned directly, with no pending work. In principle the compiler can reuse the current frame instead of pushing a new one, turning recursion into a loop and making depth unbounded. In practice, whether that happens is a property of your compiler and flags, not of your language, and it is not something you may rely on.

```
// NOT a tail call: the multiply is pending after the call returns.
int fact(int n)          { return n <= 1 ? 1 : n * fact(n - 1); }

// IS a tail call: the recursive result is returned unchanged.
int fact_tail(int n, int acc = 1) {
    return n <= 1 ? acc : fact_tail(n - 1, acc * n);
}
```

Language / compiler	Eliminates tail calls?	Consequence
GCC / Clang C++ at <code>-O2</code>	Usually, not guaranteed	Works until a refactor breaks it silently
GCC / Clang at <code>-O0</code>	No	Debug builds overflow where Release does not
Python (CPython)	No , by design	<code>RecursionError</code> at depth 1000
Java (HotSpot)	No	<code>StackOverflowError</code> at ~10-20k frames
Scheme, Scala (self-calls), Lua	Yes , guaranteed	Unbounded tail recursion is idiomatic

Guido van Rossum's position on Python is worth knowing because it comes up: tail-call elimination is rejected because it destroys the stack trace, which is the most valuable debugging artefact the interpreter produces. That is a real trade-off, not an oversight — and Topic 71 shows what you lose.

THE KEY INSIGHT

In the languages most people interview in — C++, Java, Python, JavaScript — you cannot rely on tail-call elimination. So the correct answer to “how do I handle recursion depth of 10^6 ?” is never “rewrite it in tail form”; it is **“convert to an explicit heap stack”** (Topic 25). ES6 specified proper tail calls and only Safari implemented them — a useful illustration of how little you can lean on this.

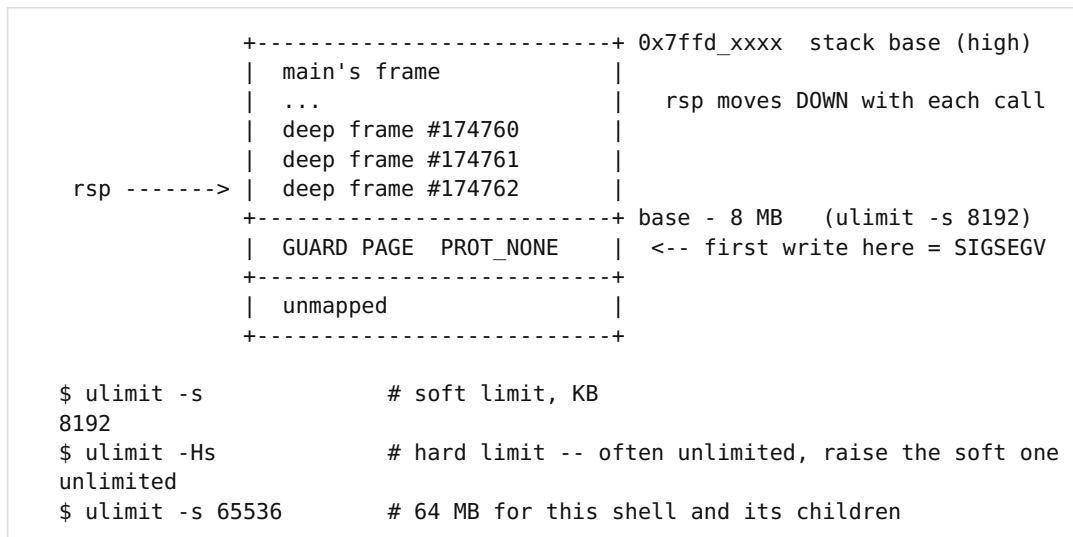
THE TRAP

Relying on tail-call elimination that the optimiser happens to be doing today. Adding a local with a destructor — a `std::string`, a `unique_ptr`, anything needing cleanup after the call — makes the call no longer a tail call, because the destructor must run *after* it returns. Symptom: a stack overflow introduced by a logging statement, with no change to the algorithm. The compiler will not warn you.

tail calls – $O(1)$ space when eliminated, $O(\text{depth})$ when not · not guaranteed in C++/Java/Python/JS · any destructor-bearing local silently disables it · use an explicit stack instead

28. Stack overflow: guard page, ulimit, symptom

Running out of call stack is not an exception you can catch in C++. The kernel places an unmapped **guard page** immediately below the stack region; the first write into it raises a page fault that the kernel converts to **SIGSEGV**. Your process dies with no cleanup, no destructors, and no error message beyond “Segmentation fault”.



What each runtime does

Runtime	Limit	What you see
C / C++ (Linux)	8 MB default (<code>ulimit -s</code>)	Segmentation fault (core dumped)
pthread (non-main)	8 MB, set via <code>pthread_attr_setstacksize</code>	Same, and the main thread's limit does not apply
Python	1000 frames (<code>sys.getrecursionlimit()</code>)	<code>RecursionError</code> — catchable
Java	~1 MB per thread (<code>-Xss</code>)	<code>StackOverflowError</code> — ~10–20k frames
Node.js	~1 MB default	<code>RangeError: Maximum call stack size exceeded</code>

Note the asymmetry: Python and Java give you a *catchable* error because they count frames or check a limit before recursing. C++ gives you a signal, because checking would cost a comparison on every call. Python's limit of 1000 is deliberately conservative — it is a guard against the interpreter's own C stack overflowing, not against your Python frames.

THE KEY INSIGHT

Raising `ulimit -s` buys a constant factor, never a change of class. Going from 8 MB to 64 MB takes you from ~175,000 frames to ~1.4 million — useful for a one-off batch job, useless for anything with adversarial or unbounded input. An explicit heap stack scales with RAM instead, which is three or four orders of magnitude more headroom.

THE TRAP

Catching `SIGSEGV` to “handle” stack overflow. The signal handler itself needs a frame, and there is no stack left to put it on, so the handler faults and the kernel kills the process outright. The only correct approach is `sigaltstack()` with a separate signal stack, and even then you can only report and abort, never resume — the frames are already gone. Symptom of getting it wrong: your handler never runs, and the process dies exactly as it would have without it.

stack overflow — 8 MB default, guard page, `SIGSEGV`, no destructors run · Python 1000 frames (catchable), Java ~10–20k, Node ~1 MB · raising the limit is a constant factor, not a fix

29. Depth arithmetic and the recursive destructor crash

Two numbers give you your maximum recursion depth: the stack limit and the frame size. Divide. This is the calculation to do *before* writing a recursive function over user-controlled input, and it takes fifteen seconds.

```
max depth = stack_bytes / frame_bytes

8 MB, 48-byte frame      8388608 / 48    = 174,762
8 MB, 96-byte frame      8388608 / 96    = 87,381
8 MB, 4144-byte frame     8388608 / 4144  = 2,024   <- one char buf[4096]
64 MB, 48-byte frame     67108864 / 48    = 1,398,101
1 MB (Java default), 100 B 1048576 / 100  = 10,486

Rule of thumb for C++ at -O2 with a handful of locals: ~200,000 frames.
Rule of thumb for anything recursing over user input: assume the input
is a straight line of length n and check n against that number FIRST.
```

The crash that ties this book to Volume I

A singly linked list with a compiler-generated destructor destroys itself recursively: destroying the head destroys its `unique_ptr` member, which destroys the next node, and so on. Each destruction is a nested call. A 1,000,000-node list therefore needs a million frames to destroy, and dies.

```
struct Node {                                // looks completely innocent
    int value;
    std::unique_ptr<Node> next;               // ~Node() destroys *next recursively
};
// Building 1,000,000 nodes: fine.
// Letting the head go out of scope: SIGSEGV, ~1,000,000 frames deep.

// THE FIX: flatten the chain iteratively before the recursion can start.
~List() {
    while (head) {
        std::unique_ptr<Node> next = std::move(head->next);
        head = std::move(next);           // each node's next is already null
    }                                     // when its destructor runs
}
```

This is the single most instructive bug in both volumes. The recursion is invisible — there is no recursive function anywhere in the source. It is generated by the compiler from a member declaration. And it is *depth-dependent*, so it passes every test with 1000 nodes and kills production at 300,000.

THE KEY INSIGHT

Any data structure containing an owning pointer to another instance of itself has a recursive destructor whether you wrote one or not. Linked lists, trees, JSON documents, expression ASTs, and parser nodes are all in this category. If the depth can be large or attacker-controlled, write an iterative destructor. The same applies to recursive *copy* constructors.

THE TRAP

The symptom points nowhere useful. The backtrace is 1,000,000 identical frames of `std::unique_ptr<Node>::~~unique_ptr`, gdb truncates it, and the crash occurs at a closing brace — often in code that did not build the list and does not mention it. Meanwhile the sanitizers are quiet, because nothing is leaked or overrun. If you see a segfault in a destructor with a monotonous backtrace, this is it.

depth – max = stack_bytes / frame_bytes, ~200k at -O2 with an 8 MB stack · recursive destructors are generated, not written · iterative flattening is $O(n)$ time, $O(1)$ call-stack space

PRACTICE SET — Topics 22-29

A. Conceptual

1. Name the two things the `call` instruction does and the one thing `ret` does. In what sense is the hardware enforcing LIFO?
2. Which locals cost nothing in a stack frame, and what single action forces a local out of a register and into the frame?
3. Why is “how deep can I recurse?” unanswerable without naming the optimisation level? Give the typical ratio.
4. Where in a disassembly do you read the frame size, and what must you add to it to get the true per-call cost?
5. Explain what the “pending work” column of a recursion trace corresponds to in an explicit-stack implementation.
6. State the four steps of the recursion-to-explicit-stack recipe.
7. If the frame struct you derive in step 1 turns out to be empty, what does that tell you about the original function?
8. Why does an explicit stack need a stage field when there are two recursive call sites but not when there is one? What does the stage field correspond to in the recursive version?
9. How many stages does a function with three recursive call sites need?
10. Define a tail call. Why does adding a local `std::string` to a tail-recursive function silently destroy the optimisation?
11. Give the reason CPython refuses to implement tail-call elimination, and say what is gained by refusing.
12. Why can C++ not turn stack overflow into a catchable exception, when Python and Java both can?
13. Explain why installing a `SIGSEGV` handler to catch stack overflow does not work, and name the mechanism that partially does.
14. A struct holds `std::unique_ptr<Node> next`. Where is the recursion, given that no recursive function appears in the source?
15. Why does the recursive-destructor bug pass all unit tests and fail in production? Name the property that makes it invisible to testing.

B. Tracing and analysis

1. Draw the call stack at maximum depth for `fact(5)`, listing the frame, the value of `n`, and the pending work in each. Then give the unwinding sequence with each intermediate result.
2. A function's prologue reads `sub $0x38,%rsp` and there is no `push %rbp`. Give the total bytes consumed per call.
3. Using that frame size, compute the maximum recursion depth on an 8 MB stack, then on the Java default of 1 MB.
4. Recompute the 8 MB depth if a `char buf[1024]` local is added. Give the reduction factor.
5. Trace `inorder_iter` from Topic 26 on the tree with root 2, left child 1, right child 3. List every stack state as `(node, stage)` pairs, bottom to top, after each loop iteration, and mark where `visit` fires.
6. For each, state whether it is a tail call: (a) `return f(n-1);` (b) `return 1 + f(n-1);` (c) `return f(n-1), 0;` (d) `{ std::string s; return f(n-1); }` (e) `return n ? f(n-1) : 0;`

7. A Linux thread has `ulimit -s 8192`. A recursive parser frame measures 176 bytes. Compute maximum depth, then state the JSON nesting depth at which an attacker crashes your service.
8. Compute the maximum depth for a 64 MB stack with the same 176-byte frame, and state whether raising the limit closes the attack. Justify in one sentence.
9. A 300,000-node linked list with a `unique_ptr next` member is destroyed. Given a 40-byte destructor frame, compute the bytes required and state whether it survives on an 8 MB stack. Repeat for 1,000,000 nodes.

C. Code tasks

1. **Build.** Write the runtime frame probe from Topic 23 and report your compiler's actual frame size for a recursive function with (a) two `int` locals, (b) two `int` locals plus `char pad[256]`, at both `-O0` and `-O2`. Give all four numbers.
2. **Break it — overflow on purpose.** Write `int boom(int d) { return boom(d + 1); }` and a variant with a `char pad[1024]` local. Run both, then run under `ulimit -s 1024` and `ulimit -s 65536`. Report the depth reached in all six combinations by printing `d` before recursing, and confirm it matches your arithmetic within 5%.
3. **Observe.** Run the overflow under `gdb`. Report what `bt` prints, how `gdb` truncates it, and what `info registers rsp` shows compared with the stack base from `cat /proc/self/maps`.
4. **Break it — the recursive destructor.** Build the `Node` with `unique_ptr next` from Topic 29. Construct lists of 10,000, 100,000 and 1,000,000 nodes and let each go out of scope. Report the first size that crashes. Confirm ASan and Valgrind both report **nothing** before the crash, and say why.
5. **Fix.** Write the iterative destructor. Confirm 10,000,000 nodes now destroy cleanly, and time it.
6. **Break it — the dangling frame reference.** In `inorder_iter`, hold `Frame& f = st.top();` across the `st.push(...)`. Run it with `std::stack<Frame, std::deque<Frame>>` and then with `std::stack<Frame, std::vector<Frame>>` on a tree of depth 20. Report which one ASan flags and what the other one does.
7. **Convert.** Take the recursive `inorder` and produce the explicit version by applying the four-step recipe on paper before writing code. Verify both produce identical output on a 100,000-node *degenerate* (left-spine) tree, and report which one crashes.
8. **Measure.** Compare wall-clock time for recursive versus explicit-stack inorder on a balanced tree of 10,000,000 nodes. Report both times and explain any difference in terms of what the hardware does for `call/ret` that your loop does not.

ANSWER KEY — Topics 22-29

A. Conceptual

1. `call` pushes the return address and jumps; `ret` pops it into the instruction pointer. It is LIFO enforcement because the return address that was pushed last is necessarily the one popped first — the CPU offers no way to return to any other frame.
2. Locals that **stay in registers** cost nothing. Taking a local's address — `&x`, or binding a reference, or passing it by pointer — forces it into memory.
3. Because `-00` spills every local to the frame while `-02` keeps most in registers. Typical ratio: **2-4x**, so the same code can recurse 4x deeper in Release than in Debug.
4. The operand of `sub $N,%rsp` in the prologue. Add **8 bytes** for the return address pushed by the caller's `call`, and another 8 if you see `push %rbp`.
5. It is exactly the contents of the explicit stack: **everything the frame held that is still needed after the recursive call returns**. In `fact` it is the pending multiplications, not the values.
6. (1) Define a frame struct of the still-live variables. (2) Push the initial frame and loop while the stack is non-empty. (3) Replace a recursive call with a push and `continue`. (4) Replace a return with a pop and delivery to the new top frame.
7. The recursion was **a loop in disguise** — there was no pending work, so it should be a plain `while` with no stack at all.
8. With one call site there is only one place to resume, so it is implicit. With two, the code must know whether it is returning from the first or the second. The stage field corresponds to the **return address** — a program counter value made into data.
9. **Four** — call sites plus one.
10. A tail call is a recursive call whose result is returned directly, with no pending work. A local `std::string` has a destructor that must run **after the callee returns**, so there is now pending work and the call is no longer in tail position.
11. Because tail-call elimination **destroys the stack trace**, and CPython treats the traceback as its most valuable debugging artefact. What is gained is that every Python exception shows the full chain of calls that produced it.
12. Because a catchable error requires checking remaining stack space **on every call**, which Python and Java can afford (they already have per-call interpreter or JIT overhead) and C++ cannot — a comparison per call would tax every function in the language.
13. Because the handler **needs a frame** and there is no stack left for it, so the handler itself faults and the kernel kills the process. The mechanism that partially works is `sigaltstack()`, which gives the handler a separate stack — but you can only report and abort, never resume.
14. In the **compiler-generated destructor**: `~Node()` destroys its `next` member, whose destructor destroys the following node, and so on. The recursion is generated from a member declaration.
15. Because it is **depth-dependent**: it needs roughly 200,000 nodes to fail, and tests use tens or hundreds. Nothing about the code changes between the passing and failing cases — only the data size.

B. Tracing and analysis

1.

```
frame    n    pending work
fact(1)  1    (base: returns 1)      <- top
fact(2)  2    multiply result by 2
fact(3)  3    multiply result by 3
fact(4)  4    multiply result by 4
fact(5)  5    multiply result by 5      <- bottom

unwind: 1 -> 2*1=2 -> 3*2=6 -> 4*6=24 -> 5*24=120
```

2. $0x38 = 56$, plus 8 for the return address = **64 bytes per call**.

3. $8388608 / 64 = \mathbf{131,072 \text{ frames}}$. Java 1 MB: $1048576 / 64 = \mathbf{16,384 \text{ frames}}$.

4. Frame becomes $64 + 1024 = 1088$ bytes. $8388608 / 1088 = \mathbf{7710 \text{ frames}}$ — a reduction of **17x**.

5.

```
iter  stack (bottom -> top)      action
1     (2,0)                      stage 0 -> 1, push left
2     (2,1) (1,0)                stage 0 -> 1, node 1 has no left
3     (2,1) (1,1)                VISIT 1, stage -> 2, no right
4     (2,1) (1,2)                pop 1
5     (2,1)                      VISIT 2, stage -> 2, push right
6     (2,2) (3,0)                stage 0 -> 1, no left
7     (2,2) (3,1)                VISIT 3, stage -> 2, no right
8     (2,2) (3,2)                pop 3
9     (2,2)                      pop 2 -- empty, done

visit order: 1, 2, 3 (correct inorder)
```

6. (a) **Yes**. (b) **No** — the addition is pending. (c) **No** — the comma operator discards the result and returns 0 afterwards. (d) **No** — `s`'s destructor must run after the call. (e) **Yes** — both branches are in tail position.

7. $8388608 / 176 = \mathbf{47,662 \text{ frames}}$. An attacker crashes the service with roughly **47,000 levels of JSON nesting** — a payload of about 94 KB consisting of nothing but opening brackets.

8. $67108864 / 176 = \mathbf{381,300 \text{ frames}}$. It does **not** close the attack: the attacker simply sends 400,000 brackets instead of 48,000, costing them 800 KB of bandwidth. A constant-factor defence against an input the attacker chooses freely is not a defence — the fix is an explicit depth limit (Topic 43).

9. $300,000 \times 40 = \mathbf{12,000,000 \text{ bytes} = 12 \text{ MB}}$ — **crashes** on an 8 MB stack. $1,000,000 \times 40 = \mathbf{40 \text{ MB}}$ — crashes comfortably. The survival threshold is $8388608 / 40 = \mathbf{209,715 \text{ nodes}}$, which is why a 100,000-node test passes and a 300,000-node production list does not.

C. Code tasks — what to verify

1. Expected shape: (a) **-O0 ~48 bytes, -O2 ~16-32 bytes** (often 0 extra if fully inlined into a loop — add `__attribute__((noinline))` to prevent that). (b) **-O0 ~304 bytes, -O2 ~272-288 bytes**. The array dominates and the optimisation level barely matters once a large local exists — that is the point of the exercise.

2. Expected depths: 8 MB with a small frame **~130,000-260,000**; 8 MB with `pad[1024]` **~7,700**; 1 MB small frame **~16,000-32,000**; 1 MB padded **~960**; 64 MB small **~1-2 million**; 64 MB padded **~61,000**. Note that at **-O2** the compiler may turn `boom` into an infinite loop with no frames at all and never crash — if that happens, add `__attribute__((noinline))` and a use of a local, and say so in your notes. That non-crash is itself a demonstration of Topic 27.

3. `bt` prints a long run of identical frames and `gdb` truncates with **(More stack frames follow...)**; use `bt -20` for the innermost or `set backtrace limit`. `info registers rsp` should show a value within one page of the **low end of the [stack] mapping** in `/proc/<pid>/maps` — that page difference is the guard page.
4. 10,000 and 100,000 succeed; **1,000,000 crashes**, and the threshold sits near 200,000 for a 40-byte frame. ASan and Valgrind report nothing because **no memory is leaked, overrun, or used after free** — the program is entirely well-behaved right up to the moment it runs out of a resource neither tool tracks. That silence is the lesson.
5. 10,000,000 nodes destroy cleanly. Time is dominated by 10M `free()` calls: expect **0.3-1.0 s**, with the call-stack depth constant at 1 throughout.
6. `std::vector` backing: ASan reports **heap-use-after-free** on the read of `f` after the push that reallocated. `std::deque` backing: **runs correctly and reports nothing**. Same source, different backing, one is a security bug — which is why the rule from Topic 16 must not be conditioned on the backing.
7. Both produce identical inorder output on a balanced tree. On the **100,000-node left-spine tree the recursive version crashes** (100,000 nested frames) while the explicit version completes, because its frames are on the heap. A degenerate tree is the standard adversarial input for anything recursive over a tree.
8. Expect the recursive version to be **equal or slightly faster** — often 5-20% — on inputs shallow enough to survive. The hardware has a **return-address predictor** (a dedicated stack of recent return targets) that makes `ret` essentially free, whereas your explicit loop's branch on the `stage` field is data-dependent and mispredicts. The honest conclusion: convert to an explicit stack for *depth safety*, not for speed.

HANDNOTE SUMMARY CARD

PART 4 - THE CALL STACK

Topics 22-29

FRAME LAYOUT (x86-64 SysV, grows DOWN)

```
[caller frame][args 7+][RETURN ADDR][saved rbp]<-rbp [locals][spills]<-rsp
CALL = push return address + jmp      RET = pop into rip
LIFO is enforced by hardware; there is no instruction to return elsewhere.
frame size = operand of "sub $N,%rsp" + 8 (return addr) + 8 if "push %rbp"
register locals cost ZERO. taking &x forces the local into the frame.
-00 frames are 2-4x -02 frames -> depth differs 4x between build configs
measure: objdump -d | -Wframe-larger-than=N | runtime address-diff probe
```

RECURSION = A STACK YOU DID NOT WRITE

the stack stores SUSPENDED WORK, not values.
recursion depth IS space complexity: O(depth) on the 8 MB call stack.

CONVERSION RECIPE (works for tree/DFS/quicksort/backtracking/parsing)

1. frame struct = every var still live AFTER the recursive call
2. push initial frame; while (!st.empty())
3. "call myself" -> push a frame, continue
4. "return v" -> pop, deliver v to the new top

empty frame -> it was a loop in disguise, write a while
one field -> one std::stack
2+ call sites-> need a STAGE field = the return address as data
 stages = call sites + 1 (inorder: 3)
NEVER hold Frame& f = st.top() across a push (vector backing dangles, T16)
do NOT allocate the explicit stack as a LOCAL ARRAY - same stack you fled

TAIL CALLS - do not rely on them

tail call = result returned directly, nothing pending
GCC/Clang -02 usually | -00 never | Python never (keeps tracebacks)
Java never | Node never | Scheme/Lua guaranteed
ANY destructor-bearing local silently un-tails the call. No warning.
correct answer to "depth 10^6" is always: EXPLICIT HEAP STACK.

STACK OVERFLOW

guard page PROT_NONE below the stack; first write -> SIGSEGV
no destructors run, no cleanup, no catch (C++). Signal handler needs a
frame it cannot get -> use sigaltstack() to REPORT, never to resume.
C/C++ 8 MB (ulimit -s 8192) -> Segmentation fault
Python 1000 frames -> RecursionError (catchable)
Java ~1 MB (-Xss) -> StackOverflowError, ~10-20k frames
Node ~1 MB -> RangeError: Maximum call stack size

DEPTH ARITHMETIC max depth = stack_bytes / frame_bytes

8 MB / 48 B = 174,762	8 MB / 96 B = 87,381
8 MB / 4144 B = 2,024	8 MB / 176 B = 47,662 (JSON parser)
64 MB / 48 B = 1,398,101	1 MB / 100 B = 10,486 (Java default)

raising ulimit is a CONSTANT FACTOR, never a fix against chosen input.

THE RECURSIVE DESTRUCTOR (ties to Volume I)

```
struct Node { int v; std::unique_ptr next; };
~Node() destroys *next recursively - you never wrote a recursive function.
40 B frame -> dies above 209,715 nodes. Passes every test at 10k.
ASan and Valgrind report NOTHING: nothing is leaked, overrun or reused.
backtrace is 1M identical ~unique_ptr frames, truncated, at a closing brace.
FIX: iterative flatten - while (head) head = std::move(head->next);
applies to: lists, trees, JSON docs, ASTs, parser nodes, recursive COPY too
```


PART FIVE

Expression parsing

The application that put stacks in every textbook, done properly: why postfix exists at all, shunting-yard with a precedence table you can defend, the unary-minus case everyone omits, and a calculator that survives the inputs interviewers actually try.

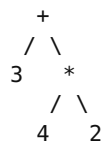
30. Infix, prefix, postfix

Infix is what humans write: $3 + 4 * 2$. It is ambiguous without extra rules — precedence, associativity, and parentheses — because the operator sits between its operands and nothing in the text says which multiplication binds to which addition. Prefix and postfix put the operator outside the operands, which removes the ambiguity entirely and therefore removes the need for parentheses.

Notation	Form	$3 + 4 * 2$	Parentheses needed?
Infix	$a \oplus b$	$3 + 4 * 2$	Yes
Prefix (Polish)	$\oplus a b$	$+ 3 * 4 2$	No
Postfix (RPN)	$a b \oplus$	$3 4 2 * +$	No

The reason parentheses become unnecessary is worth stating precisely: in postfix, an operator's operands are *the two most recently completed subexpressions*. That is a LIFO rule, which is why a stack evaluates postfix in one pass with no lookahead and no backtracking. Infix cannot be evaluated that way because when you meet $+$ you do not yet know whether its right operand is 4 or $4 * 2$.

The same expression as a tree -- the three notations are three walks of it



```
preorder (root, left, right) -> + 3 * 4 2      = PREFIX
inorder  (left, root, right) -> 3 + 4 * 2      = INFIX (ambiguous!)
postorder (left, right, root) -> 3 4 2 * +      = POSTFIX
```

Infix is the only one that loses information, because the tree structure is not recoverable from the sequence without extra rules. This is why Topic 36 can rebuild the tree from postfix but not from infix.

THE KEY INSIGHT

The three notations are the three depth-first traversals of the same expression tree. Postfix is postorder, prefix is preorder, infix is inorder. Once you see that, every conversion in this part is a tree walk, and the reason postfix evaluates with a single stack is the same reason postorder traversal needs a stack (Topic 64) — it is the same operation.

THE TRAP

Assuming postfix is unambiguous for *any* operator set. It is only unambiguous when every operator's arity is fixed and known. Add a unary minus that shares the token $-$ with binary subtraction and $3 4 - -$ becomes ambiguous. Real RPN systems solve this by giving unary minus a distinct token — HP calculators use CHS , and Topic 34 uses $\sim$. If you skip this, your evaluator silently mis-evaluates negative numbers.

notation — conversion $O(n)$ · postfix/prefix need no parentheses because arity plus order determines grouping · the three notations are pre/in/postorder walks of one tree

31. Postfix evaluation

The whole algorithm is nine lines and it is the cleanest illustration of a stack in the book. Scan left to right. An operand is pushed. An operator pops two operands, applies itself, and pushes the result. At the end the stack holds exactly one value: the answer.

```
#include <stack>
#include <string>
#include <stdexcept>

long evalPostfix(const std::vector<std::string>& tok) {
    std::stack<long> st;
    for (const std::string& t : tok) {
        if (isOperator(t)) {
            if (st.size() < 2) throw std::runtime_error("too few operands");
            long b = st.top(); st.pop();      // RIGHT operand first
            long a = st.top(); st.pop();      // then LEFT
            st.push(apply(t, a, b));
        } else {
            st.push(std::stol(t));
        }
    }
    if (st.size() != 1) throw std::runtime_error("malformed expression");
    return st.top();
}
```

Trace: 5 1 2 + 4 * + 3 - (infix: 5 + ((1+2)*4) - 3)

token	action	stack (bottom -> top)
5	push 5	[5]
1	push 1	[5, 1]
2	push 2	[5, 1, 2]
+	b=2, a=1, push 1+2=3	[5, 3]
4	push 4	[5, 3, 4]
*	b=4, a=3, push 3*4=12	[5, 12]
+	b=12, a=5, push 5+12=17	[17]
3	push 3	[17, 3]
-	b=3, a=17, push 17-3=14	[14]

result 14, stack size 1 -> well formed

Max stack depth reached: 3. For n tokens, depth <= (n+1)/2.

THE KEY INSIGHT

The first pop is the **right** operand and the second is the **left**. For **+** and ***** the mistake is invisible; for **-** and **/** it silently computes **b - a** instead of **a - b**. If a test with only addition passes and one with subtraction fails, this is the line.

THE TRAP

Not validating the final stack size. **3 4 5 +** leaves **[3, 9]** — two values — and returning **top()** gives 9, silently dropping the 3. Likewise **3 +** underflows. LeetCode-style problems promise well-formed input, so this check is often omitted in practice and then the code is copied into a system that parses user input. Symptom: user-supplied expressions returning plausible wrong numbers rather than errors.

postfix eval – Time $O(n)$, one pass, no lookahead · Space $O(n)$ worst case, max depth $(n+1)/2$ · validity check: exactly one value remains · pop order is right-then-left

32. Prefix evaluation

Prefix is postfix with the direction reversed. Scan **right to left**; push operands; when you meet an operator, pop two and push the result. The only change from Topic 31 is the scan direction and, as a consequence, the pop order.

```
long evalPrefix(const std::vector<std::string>& tok) {
    std::stack<long> st;
    for (auto it = tok.rbegin(); it != tok.rend(); ++it) {    // RIGHT to LEFT
        if (isOperator(*it)) {
            long a = st.top(); st.pop();           // LEFT operand first now
            long b = st.top(); st.pop();           // then RIGHT
            st.push(apply(*it, a, b));
        } else {
            st.push(std::stol(*it));
        }
    }
    return st.top();
}
```

Trace: - + 5 * + 1 2 4 3 (same expression as Topic 31)

scanning RIGHT to LEFT

token	action	stack (bottom -> top)
3	push 3	[3]
4	push 4	[3, 4]
2	push 2	[3, 4, 2]
1	push 1	[3, 4, 2, 1]
+	a=1, b=2, push 1+2=3	[3, 4, 3]
*	a=3, b=4, push 3*4=12	[3, 12]
5	push 5	[3, 12, 5]
+	a=5, b=12, push 5+12=17	[3, 17]
-	a=17, b=3, push 17-3=14	[14]

result 14 -- identical to the postfix evaluation, as it must be

An alternative that avoids reverse iteration: reverse the token sequence, then run a modified postfix evaluator with the pop order swapped. This is worth knowing because it generalises — the same reverse-and-reuse trick converts infix to prefix in Topic 33's framework without writing a second algorithm.

THE KEY INSIGHT

Prefix and postfix evaluation are the same algorithm parameterised by scan direction, and the pop order flips with it. Knowing that means you write one evaluator and a flag, not two evaluators — and it means that if you can derive one at a whiteboard you can derive the other in ten seconds.

THE TRAP

Copying the postfix pop order into the prefix evaluator. Because the scan is reversed, the *first* pop in prefix is the left operand, not the right. Getting it wrong produces correct results for `+` and `*` and reversed results for `-` and `/` — the identical symptom as Topic 31's trap, from the opposite cause, which makes it doubly easy to mis-diagnose.

prefix eval — Time $O(n)$ · Space $O(n)$ · scan right-to-left, pop left-then-right · same algorithm as postfix with the direction flipped

33. Shunting-yard

Dijkstra's shunting-yard algorithm converts infix to postfix in one pass with two stacks — conceptually an output queue and an operator stack. It is the single most-asked stack algorithm after balanced brackets, and it is short enough to write from memory once you hold the invariant.

The invariant: the operator stack always holds operators in non-decreasing precedence from bottom to top. An incoming operator pops everything of greater or equal precedence before being pushed, because those operators' operands are now complete.

```
std::vector<std::string> toPostfix(const std::vector<std::string>& tok) {
    std::vector<std::string> out;
    std::stack<std::string> ops;
    for (const auto& t : tok) {
        if (isNumber(t)) {
            out.push_back(t);
        } else if (t == "(") {
            ops.push(t);
        } else if (t == ")") {
            while (!ops.empty() && ops.top() != "(") {
                out.push_back(ops.top()); ops.pop();
            }
            if (ops.empty()) throw std::runtime_error("unmatched ");
            ops.pop(); // discard the "("
        } else { // an operator
            while (!ops.empty() && ops.top() != "(" &&
                (prec(ops.top()) > prec(t) ||
                 (prec(ops.top()) == prec(t) && leftAssoc(t)))) {
                out.push_back(ops.top()); ops.pop();
            }
            ops.push(t);
        }
    }
    while (!ops.empty()) {
        if (ops.top() == "(") throw std::runtime_error("unmatched (");
        out.push_back(ops.top()); ops.pop();
    }
    return out;
}
```

Trace: 3 + 4 * 2 / (1 - 5)

token	operator stack (bot->top)	output so far
3	[]	3
+	[+]	3
4	[+]	3 4
*	[+, *]	3 4
2	[+, *]	3 4 2
/	[+, /]	3 4 2 *
(	[+, /, (]	3 4 2 *
1	[+, /, (]	3 4 2 * 1
-	[+, /, (, -]	3 4 2 * 1
5	[+, /, (, -]	3 4 2 * 1 5
)	[+, /]	3 4 2 * 1 5 -
drain	[]	3 4 2 * 1 5 - / +

result: 3 4 2 * 1 5 - / + infix check: 3 + ((4*2) / (1-5)) = 3 + (8/-4) = 1

THE KEY INSIGHT

(is pushed with the lowest possible precedence so that nothing pops it accidentally, and it is removed only by its matching) or by the end-of-input error check. Treating (as an ordinary operator with a real precedence is the single most common way to get shunting-yard wrong; it is a *barrier*, not an operator.

THE TRAP

Omitting the two unmatched-parenthesis checks. Without them, 3 +) pops the operator stack empty and then calls `ops.top()` on it — undefined behaviour, typically returning a garbage string that is then written to the output. And (3 + 4 silently emits (into the post-fix output, which the evaluator in Topic 31 will treat as an operand and crash on inside `stol`. Both inputs are the first things an interviewer will try.

shunting-yard — Time $O(n)$, each token pushed and popped at most once · Space $O(n)$ for the operator stack · invariant: precedence non-decreasing bottom to top · (is a barrier, not an operator

34. Precedence, associativity, unary minus

Shunting-yard is only as correct as its precedence table. Here is one that handles everything a calculator interview will ask for, including the two cases that are almost always omitted: right associativity and unary minus.

Operator	Precedence	Associativity	Note
()	—	—	Barrier; never compared
+ -	1	Left	Binary
* / %	2	Left	Binary
~ (unary -)	3	Right	Distinct token — see below
^	4	Right	$2^3^2 = 2^9 = 512$, not 64

Why associativity changes one comparison

For a **left**-associative operator, an incoming operator of equal precedence must pop the one on the stack: $8 / 4 / 2$ must group as $(8/4)/2 = 1$. For a **right**-associative one it must *not*: $2 ^ 3 ^ 2$ must group as $2^(3^2) = 512$. That is the entire content of the `prec == prec && leftAssoc` clause in Topic 33 — one condition, two grouping behaviours.

Unary minus: the token-disambiguation pass

```
// A minus is UNARY iff it is the first token, or follows an operator or "(".
// Rewrite it to a distinct token so precedence and arity are unambiguous.
std::vector<std::string> markUnary(std::vector<std::string> t) {
    for (std::size_t i = 0; i < t.size(); ++i) {
        if (t[i] != "-") continue;
        bool unary = (i == 0) || t[i-1] == "(" || isOperator(t[i-1]);
        if (unary) t[i] = "~";          // arity 1, precedence 3, right-assoc
    }
    return t;
}

// "-3 + 4"      -> ~ 3 + 4      -> postfix: 3 ~ 4 +      = 1
// "2 * -3"      -> 2 * ~ 3      -> postfix: 2 3 ~ *      = -6
// "-(2 + 3)"    -> ~ ( 2 + 3 )  -> postfix: 2 3 + ~      = -5
// "5 - -3"      -> 5 - ~ 3      -> postfix: 5 3 ~ -      = 8
```

The evaluator then needs one extra branch: `~` pops *one* operand and pushes its negation. That single-operand case is why the unary token must be distinct — an evaluator cannot infer arity from `-` alone.

THE KEY INSIGHT

Unary minus is a *tokenisation* problem, not a parsing problem. Solve it in a pre-pass that rewrites the token to a distinct symbol, and both shunting-yard and the evaluator stay unchanged apart from one arity branch. Trying to handle it inside the shunting-yard loop tangles arity with precedence and is where hand-written calculators go wrong.

THE TRAP

Making `^` left-associative by using the default comparison. 2^3^2 then evaluates to $(2^3)^2 = 64$ instead of **512**. It is a single missing clause, every other test passes, and the wrong answer is a plausible number — so it survives review. Test $2^3^2 == 512$ and $8/4/2 == 1$ together; those two inputs cover both associativity directions.

precedence – table lookup $O(1)$ · left-assoc pops on equal precedence, right-assoc does not ·
unary minus is a retokenisation to `~`, arity 1, prec 3, right-assoc

35. Two-stack direct infix evaluation

You can evaluate infix without converting to postfix at all, using two stacks: one for values and one for operators. The control flow is shunting-yard with the output queue replaced by immediate application. It is shorter, it avoids materialising an intermediate array, and it is the version to write when an interviewer says “evaluate this expression” rather than “convert it.”

```

long evalInfix(const std::vector<std::string>& tok) {
    std::stack<long>      vals;
    std::stack<std::string> ops;

    auto applyTop = [&] {
        std::string op = ops.top(); ops.pop();
        if (op == "~") { long a = vals.top(); vals.pop(); vals.push(-a); return; }
        long b = vals.top(); vals.pop();
        long a = vals.top(); vals.pop();
        vals.push(apply(op, a, b));
    };

    for (const auto& t : tok) {
        if (isNumber(t))      vals.push(std::stol(t));
        else if (t == "(")    ops.push(t);
        else if (t == ")") {
            while (!ops.empty() && ops.top() != "(") applyTop();
            if (ops.empty()) throw std::runtime_error("unmatched ");
            ops.pop();
        } else {
            while (!ops.empty() && ops.top() != "(" &&
                (prec(ops.top()) > prec(t) ||
                 (prec(ops.top()) == prec(t) && leftAssoc(t)))) applyTop();
            ops.push(t);
        }
    }
    while (!ops.empty()) applyTop();
    if (vals.size() != 1) throw std::runtime_error("malformed expression");
    return vals.top();
}

```

Trace: 3 + 4 * 2

token	vals	ops	action
3	[3]	[]	push value
+	[3]	[+]	stack empty, push
4	[3,4]	[+]	push value
*	[3,4]	[+,*]	prec(*) > prec(+), do NOT apply, push
2	[3,4,2]	[+,*]	push value
drain	[3,8]	[+]	applyTop: 4*2 = 8
drain	[11]	[]	applyTop: 3+8 = 11

answer 11. Note the multiplication happened FIRST despite appearing second in the text -- that is precedence, enforced by the pop condition.

THE KEY INSIGHT

Shunting-yard and two-stack evaluation are one algorithm. The only difference is whether popping an operator *emits* it or *applies* it. If you know one, write the other by changing three lines — and say so in an interview, because it demonstrates you understand the structure rather than having memorised two snippets.

THE TRAP

Forgetting that `applyTop` must handle unary operators separately. With `~` in the operator stack and only one value available, the two-pop path calls `vals.top()` on an empty stack — undefined behaviour in `std::stack`, so it reads garbage and produces a wildly wrong number rather than crashing. Symptom: `-3 + 4` returning something like 1717986921. Branch on arity before popping.

two-stack eval – Time $O(n)$ · Space $O(n)$ across two stacks · identical control flow to shunting-yard, apply instead of emit · no intermediate postfix array

36. Expression tree from postfix

Replace “push the computed value” with “push a node” and the postfix evaluator becomes a parser. This is how real compilers get from tokens to an AST, and it is the bridge between this part and Part 9's tree traversals.

```
struct ExprNode {
    std::string tok;
    ExprNode* l = nullptr;
    ExprNode* r = nullptr;
};

ExprNode* buildTree(const std::vector<std::string>& postfix) {
    std::stack<ExprNode*> st;
    for (const auto& t : postfix) {
        if (!isOperator(t)) {
            st.push(new ExprNode{t});
        } else {
            ExprNode* n = new ExprNode{t};
            n->r = st.top(); st.pop();           // RIGHT child popped first
            n->l = st.top(); st.pop();
            st.push(n);
        }
    }
    return st.top();                          // the root
}
```

Building from 3 4 2 * +

token	stack of subtree roots (bottom -> top)
-------	--

3	[3]
4	[3, 4]
2	[3, 4, 2]
*	[3, (*)]

where (*) = *

/ \

4 2

+	[(+)]
---	-------

where (+) = +

/ \

3 *

/ \

4 2

Once built, the three walks recover the three notations (Topic 30):

preorder -> + 3 * 4 2

inorder -> 3 + 4 * 2 (needs parentheses re-inserted to be exact)

postorder -> 3 4 2 * + (the input, unchanged)

The inorder walk is where the information loss from Topic 30 becomes concrete. Printing inorder without parentheses gives a string that may re-parse to a different tree. To round-trip, emit a parenthesis pair around every operator node whose precedence is lower than its parent's — or, more simply and more safely, around every internal node.

THE KEY INSIGHT

Evaluation and parsing are the same traversal with different actions at the operator: one computes $a \circ b$, the other allocates a node with children a and b . Recognising that means the tree version costs you no new algorithm — and it explains why an interpreter can either evaluate directly or build an AST from the same parser.

THE TRAP

Assigning the children in the order they are popped: `n->l = st.top()` first. The tree is then mirrored at every operator, so `+` and `*` still evaluate correctly and `-` and `/` do not — the same signature as Topic 31's trap, now hidden one level deeper inside a tree you must draw to debug. Pop right first.

tree build — Time $O(n)$ · Space $O(n)$ nodes plus $O(\text{depth})$ stack · same algorithm as evaluation, node allocation replaces arithmetic · pop right child first

37. A working calculator

Everything in this part assembled into one program that handles multi-digit numbers, all five binary operators, parentheses, unary minus, and right-associative exponentiation — and rejects malformed input rather than guessing. This is the artefact to have written before a calculator interview question.

```
// tokenise: multi-digit integers, operators, parentheses. Skips whitespace.
std::vector<std::string> lex(const std::string& s) {
    std::vector<std::string> out;
    for (std::size_t i = 0; i < s.size(); ) {
        if (std::isspace((unsigned char)s[i])) { ++i; continue; }
        if (std::isdigit((unsigned char)s[i])) {
            std::size_t j = i;
            while (j < s.size() && std::isdigit((unsigned char)s[j])) ++j;
            out.push_back(s.substr(i, j - i)); // the whole number, not one digit
            i = j;
        } else if (std::strchr("+-*/%^()", s[i])) {
            out.push_back(std::string(1, s[i]));
            ++i;
        } else {
            throw std::runtime_error(std::string("bad character: ") + s[i]);
        }
    }
    return out;
}

long calculate(const std::string& s) {
    return evalInfix(markUnary(lex(s))); // lex -> mark -> evaluate
}
```

The test set that actually finds bugs

Input	Expected	What it catches
3+4*2	11	Precedence
8/4/2	1	Left associativity
2^3^2	512	Right associativity
-3+4	1	Leading unary minus
5- -3	8	Unary after a binary operator
-(2+3)	-5	Unary before a parenthesis
2*(3+(4-1))	12	Nesting
100/10	10	Multi-digit lexing
3+)	throws	Unmatched close
(3+4	throws	Unmatched open
3 4	throws	Two values, no operator
1/0	throws	Division by zero

THE KEY INSIGHT

The pipeline is **lex** → **mark unary** → **evaluate**, three independent stages with clean interfaces. Each is testable alone, and each failure mode belongs to exactly one stage: **100** parsed as **1, 0, 0** is a lexer bug, **-3** mis-evaluated is a marking bug, and **2^3^2 == 64** is an evaluator bug. Interview calculators that fail usually do so because all three stages were written as one loop.

THE TRAP

Lexing one character at a time, so **100/10** becomes **1, 0, 0, /, 1, 0** and evaluates to something arbitrary. Every single-digit test passes. LeetCode's "Basic Calculator" family supplies multi-digit numbers precisely to catch this. The **while (isdigit)** loop in **lex** is the fix, and it is three lines.

calculator – Time $O(n)$ total across all three stages · Space $O(n)$ · lex/mark/evaluate are independently testable · 12 test inputs cover every failure mode in this part

PRACTICE SET — Topics 30-37

A. Conceptual

1. Why does postfix need no parentheses? Answer in terms of where an operator's operands are located.
2. Name the tree traversal corresponding to each of the three notations. Which one loses information, and what exactly is lost?
3. Under what condition does postfix stop being unambiguous? Give a concrete token sequence that becomes ambiguous.
4. In postfix evaluation, which pop gives the left operand? For which operators does getting it wrong go unnoticed?
5. What must be true of the stack at the end of a postfix evaluation, and what does violating it in each direction mean?
6. State the difference between the prefix and postfix evaluators in exactly two respects.
7. State the shunting-yard invariant about the operator stack.
8. Why is `(` pushed with no comparable precedence, and what goes wrong if you give it a real one?
9. Which single clause in the shunting-yard pop condition implements associativity? Explain both branches with a concrete expression.
10. Why is unary minus solved by retokenisation rather than inside the parser?
11. Give the exact rule for deciding whether a `-` is unary.
12. How do shunting-yard and two-stack infix evaluation differ? Answer in one sentence.
13. Why must `applyTop` branch on arity before popping?
14. How does the postfix evaluator become a parser? State the one substitution.
15. Name the three stages of the calculator pipeline and assign each of these failures to exactly one stage: `100` read as three tokens; `2^3^2 == 64`; `-3+4 == 7`.

B. Tracing and analysis

1. Evaluate the postfix `7 2 3 * - 4 +` by hand. Give the stack contents after every token and the final result.
2. Evaluate the prefix `* + 2 3 - 9 4`. Show the right-to-left trace and the result.
3. Convert `a + b * c - d / e` to postfix by hand with shunting-yard. Give the operator stack and output after each token.
4. Convert `( 1 + 2 ) * ( 3 - 4 ) ^ 2 ^ 3` to postfix. State the value if all variables are as written, and confirm the exponent associativity.
5. Give the postfix and the value of `8 / 4 / 2` and of `2 ^ 3 ^ 2`. State which associativity each tests.
6. Apply `markUnary` by hand to the token sequences for `-5 * -(3 - -2)`. Give the marked sequence, the postfix, and the value.
7. For the postfix `5 1 2 + 4 * + 3 -`, give the maximum stack depth reached and the token at which it occurs. Then state the general bound on depth for n tokens and construct a 9-token input that achieves it.
8. Draw the expression tree built from `3 4 2 * +` and give all three traversals. Then state what the inorder walk would re-parse to without parentheses, and whether that differs from the original.

9. For each malformed input, state exactly where the failure is detected — lexer, shunting-yard, or evaluator: `3+); (3+4; 3 4; 3 & 4; 3 +.`

C. Code tasks

1. **Build.** Implement `lex`, `markUnary`, `toPostfix`, `evalPostfix` and `evalInfix`. Run all twelve rows of the Topic 37 test table and confirm every expected value and every expected throw.
2. **Break it — the operand order.** Swap the two pops in `evalPostfix` so `a` is popped first. Run the full test table. Report exactly which rows still pass, which fail, and the wrong value for each failure. Explain why the passing rows pass.
3. **Break it — associativity.** Remove the `&& leftAssoc(t)` clause so equal precedence always pops. Run `2^3^2` and `8/4/2`. Report both results and state which one is now wrong and by how much.
4. **Break it — single-character lexing.** Replace the `while (isdigit)` loop with a single-character push. Run `100/10`, `12+3` and `3+4`. Report which produce right answers and explain why the last one hides the bug.
5. **Break it — the missing paren checks.** Delete both unmatched-parenthesis throws. Run `3+)` and `(3+4` under `-fsanitize=address`. Record what ASan reports for each and where the error actually originates.
6. **Fix and verify.** Restore all four defects. Re-run the twelve test rows and confirm all pass, then add `1/0` and confirm it throws rather than raising `SIGFPE`.
7. **Extend.** Add a `buildTree` stage and a `printInorder` that emits parentheses around every internal node. Feed its output back through `calculate` for all twelve test inputs and assert the values round-trip.
8. **Measure.** Generate a 1,000,000-token expression of the form `1+1+1+...` and time `evalInfix` against `toPostfix` followed by `evalPostfix`. Report both times and the peak operator-stack depth for each. Then generate `(((((...1...))))` with 500,000 nesting levels and report the peak stack depth and the memory used.

ANSWER KEY — Topics 30-37

A. Conceptual

1. Because an operator's operands are always **the two most recently completed subexpressions** — a LIFO rule that fixes grouping without any additional notation.
2. Prefix = **preorder**, infix = **inorder**, postfix = **postorder**. Infix loses information: **the tree structure is not recoverable from the sequence** without precedence rules or parentheses.
3. When operator **arity is not fixed per token**. Example: `3 4 - -` — the second `-` could be binary (giving `3 - (4-3)` ... ill-formed) or unary negation of the result.
4. The **second** pop is the left operand. Getting it wrong is invisible for **+** and *****, which are commutative.
5. Exactly **one value** must remain. More than one means **too few operators** (e.g. `3 4 5 +` leaves two); zero or an underflow during the scan means **too few operands**.
6. **Scan direction** (right-to-left instead of left-to-right) and **pop order** (left operand first instead of right).
7. The operator stack holds operators in **non-decreasing precedence from bottom to top**.
8. Because it is a **barrier**, not an operator: it must never be popped by a precedence comparison, only by its matching `)`. Giving it a real precedence lets an incoming operator pop it, which destroys the grouping and emits `(` into the output.
9. `prec(ops.top()) == prec(t) && leftAssoc(t)`. Left: `8/4/2` must pop on the second `/` so it groups as `(8/4)/2 = 1`. Right: `2^3^2` must not pop, so it groups as `2^(3^2) = 512`.
10. Because the ambiguity is about **which token this is**, not about how to parse it. Rewriting to a distinct token fixes arity and precedence at once and leaves both the parser and the evaluator otherwise untouched.
11. A `-` is unary iff it is **the first token, or follows an operator, or follows `(`**.
12. Popping an operator **emits** it in shunting-yard and **applies** it in two-stack evaluation.
13. Because a unary operator has only one operand available; the two-pop path would call `top()` on an empty stack, which is **undefined behaviour** in `std::stack` and returns garbage rather than crashing.
14. Replace **“push the computed value”** with **“push a newly allocated node whose children are the two popped subtrees”**.
15. Stages: **lex, mark unary, evaluate**. `100` as three tokens = **lexer**. `2^3^2 == 64` = **evaluator** (associativity in the pop condition). `-3+4 == 7` = **marking** (the unary minus was treated as binary or dropped).

B. Tracing and analysis

1.

token	action	stack
7	push	[7]
2	push	[7,2]
3	push	[7,2,3]
*	b=3 a=2 -> 6	[7,6]
-	b=6 a=7 -> 1	[1]
4	push	[1,4]
+	b=4 a=1 -> 5	[5]
result 5		

2. Scanning right to left: `4, 9, -(a=9,b=4)→5, 3, 2, +(a=2,b=3)→5, *(a=5,b=5)→25`. **Result 25**.
Check: `(2+3) * (9-4) = 25`.

3.

token	ops	output
a	[]	a
+	[+]	a
b	[+]	a b
*	[+,*]	a b
c	[+,*]	a b c
-	[-]	a b c * + (* and + both pop: prec >= and left-assoc)
d	[-]	a b c * + d
/	[-,/]	a b c * + d
e	[-,/]	a b c * + d e
drain	[]	a b c * + d e / -

- Postfix: `1 2 + 3 4 - 2 3 ^ ^ *`. The two `^` tokens do **not** pop each other (right associative), so they drain in reverse order giving `2 3 ^` then `^` — i.e. $(3-4)^{(2^3)} = (-1)^8 = 1$, times $(1+2) = 3$. **Value 3**. Left-associating `^` would give $((-1)^2)^3 = 1$ too here — use `2^3^2` as the discriminating test instead.
- `8 4 / 2 / = 1`, tests **left** associativity. `2 3 2 ^ ^ = 512`, tests **right** associativity.
- Tokens: `- 5 * - ( 3 - - 2 )`. Marked: `~ 5 * ~ ( 3 - ~ 2 )`. Postfix: `5 ~ 3 2 ~ - ~ *`. Value: `~2 = -2`; `3 - (-2) = 5`; `~5 = -5`; `-5 * -5 = 25`.
- Maximum depth **3**, reached after the token **4** (stack `[5,3,4]`). General bound: $(n+1)/2$. A 9-token input achieving it: `1 2 3 4 5 + + + +` — five operands pushed before any operator, giving depth **5**.
- Tree: root `+`, left `3`, right `*` with children `4` and `2`. Preorder `+ 3 * 4 2`; inorder `3 + 4 * 2`; postorder `3 4 2 * +`. The unadorned inorder re-parses to **the same tree** here, by luck — precedence happens to agree. Swap the operators `(3 * (4 + 2))` and the inorder `3 * 4 + 2` re-parses to a **different** tree, which is the general case.
- `3+)`: **shunting-yard** (or the two-stack evaluator) — the drain-to-`(` loop empties the stack. `(3+4)`: **shunting-yard** final drain, which sees `(` remaining. `3 4`: **evaluator** — two values remain at the end. `3 & 4`: **lexer** — bad character. `3 +`: **evaluator** — underflow when the operator is applied.

C. Code tasks — what to verify

- All twelve rows must match, including the four that throw. If `1/0` raises `SIGFPE` instead of throwing, your `apply` is missing the zero check — integer division by zero is UB in C++, not an exception.
- Still passing: `3+4*2`, `2*(3+(4-1))` (all-commutative paths), and all four throwing rows. Failing: `8/4/2` gives **4** instead of 1; `100/10` gives **0** (`10/100` in integers); `5- -3` gives **-8**; `-(2+3)` path shifts. The commutative rows pass because `a+b == b+a` and `a*b == b*a` — the bug is invisible to exactly the operators most tests use.
- `2^3^2` gives **64** instead of 512 — wrong by a factor of 8. `8/4/2` still gives **1**, correctly, because left associativity is what the unconditional pop implements. One test detects the bug, the other cannot — which is why both are in the table.
- `3+4` gives **7**, correct. `12+3` gives **4** (tokens `1,2,+,3` → evaluator sees two leading values). `100/10` throws or returns garbage. The single-digit case hides the bug because **a one-character number is indistinguishable from a correctly lexed one** — every test written by hand tends to use small numbers.
- `3+)`: `ops.top()` on an empty `std::stack<std::string>`. Under ASan expect a **heap-buffer-over-flow READ** inside `std::deque`'s element access, or a crash inside `std::string`'s copy constructor reading a garbage pointer — the reported frame is in the standard library, *not* in your parser. `(3+4`: usually **no sanitiser error at all**; the `(` is emitted into the postfix output

and `std::stoi("")` throws `std::invalid_argument` from the evaluator. Two different-looking failures, one missing check — and neither points at the line that is wrong.

6. All twelve pass. `1/0` must throw your own exception; if you see `Floating point exception (core dumped)` the check is still missing.
7. Round-tripping must give identical values for all twelve inputs. Parenthesising every internal node produces ugly output like `((3)+((4)*(2)))` but is **always correct**; the precedence-aware version is prettier and is where round-trip bugs hide.
8. Both should run in **tens of milliseconds** for 1M tokens, with the two-pass version measurably slower — expect **1.3-2×** — because it materialises a 1,000,000-element intermediate vector. Peak operator-stack depth for `1+1+1+...` is **1**, because each `+` pops its predecessor. For 500,000 nested parentheses the peak depth is **500,000**, at roughly 32 bytes per `std::string` entry = **~16 MB** of heap — which would have been a stack overflow in a recursive-descent parser (Topic 43) and is merely a large allocation here.

HANDNOTE SUMMARY CARD

PART 5 - EXPRESSION PARSING

Topics 30-37

THE THREE NOTATIONS ARE THREE WALKS OF ONE TREE

```
prefix = preorder    + 3 * 4 2    no parens needed
infix  = inorder      3 + 4 * 2    AMBIGUOUS - structure not recoverable
postfix = postorder   3 4 2 * +    no parens needed
postfix works because an operator's operands are the two most recently
completed subexpressions -- a LIFO rule. Ambiguous only if arity varies.
```

POSTFIX EVAL - scan LEFT to RIGHT

```
operand -> push
operator-> b = pop(); a = pop(); push(apply(op, a, b));
          ^ RIGHT first, then LEFT. wrong order: + and * still pass,
          - and / silently reversed.

end: exactly ONE value must remain. >1 = too few operators, underflow =
too few operands. max depth (n+1)/2.
```

PREFIX EVAL - scan RIGHT to LEFT, pop LEFT first then RIGHT

same algorithm, two changes: direction and pop order.

SHUNTING-YARD (infix -> postfix) invariant: ops stack precedence is
number -> output non-decreasing bottom to top

```
"(" -> push (a BARRIER, never compared, never popped by precedence)
")" -> pop to "(" , discard it; empty stack => unmatched )
op   -> while top != "(" and (prec(top) > prec(t) or
      (prec(top) == prec(t) and leftAssoc(t)))
      emit top
      push t
drain -> emit all; a remaining "(" => unmatched (
BOTH paren checks are mandatory: 3+) and (3+4 are the first inputs tried.
```

PRECEDENCE TABLE

```
( )    barrier    + -    1 left    * / %    2 left
~ unary 3 RIGHT    ^      4 RIGHT
the ONE clause "prec==prec && leftAssoc" gives both behaviours:
8/4/2 = 1 (left: pop on equal)      2^3^2 = 512 (right: do not pop)
test BOTH; each is invisible to the other.
```

UNARY MINUS = A TOKENISATION PROBLEM, NOT A PARSING ONE

"-" is unary iff: first token, or follows an operator, or follows "("
rewrite to "~" (arity 1, prec 3, right-assoc); evaluator adds ONE branch
-3+4 -> ~ 3 + 4 -> 3 ~ 4 + 5- -3 -> 5 - ~ 3 -> 5 3 ~ -

TWO-STACK INFIX EVAL = shunting-yard where popping APPLIES instead of EMITS
one sentence difference. applyTop MUST branch on arity before popping,
or "~" pops twice from a one-element stack (UB, garbage, no crash).

EXPRESSION TREE = the evaluator with "push a node" instead of "push a value"
n->r = pop(); n->l = pop(); <- RIGHT child first, same trap as eval
evaluation and parsing are one traversal with different actions.

CALCULATOR PIPELINE: lex -> markUnary -> evaluate (3 testable stages)

```
"100" as 1,0,0      -> LEXER (missing the while(isdigit) loop)
2^3^2 == 64          -> EVALUATOR (associativity clause)
-3+4 == 7           -> MARKING

TEST SET: 3+4*2=11 | 8/4/2=1 | 2^3^2=512 | -3+4=1 | 5- -3=8 | -(2+3)=-5
          2*(3+(4-1))=12 | 100/10=10 | 3+ 3+4 "3 4" 1/0 all THROW
1/0 is UB in C++ (SIGFPE), not an exception -- check it yourself.
```


PART SIX

Matching and nesting

The interview warm-up and everything it grows into. Index stacks instead of value stacks, the sentinel that makes longest-valid-parentheses work, nested-structure unwinding, and the parser depth limit that stops a 200-byte request killing your service.

38. Balanced brackets

The canonical stack problem, asked in some form in a large fraction of first technical screens. Given a string of `()[]{}` , decide whether every opener has a matching closer in the correct order. Three lines of logic, and three distinct failure modes to close.

```
bool isBalanced(const std::string& s) {
    static const std::unordered_map<char, char> pair = {
        {'}', '{'}, {'}', '['}, {'}', '('}, {'}', '['}, {'}', '('}, {'}', '('}
    };
    std::stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);
        } else if (auto it = pair.find(c); it != pair.end()) {
            if (st.empty()) return false; // closer with nothing open
            if (st.top() != it->second) return false; // wrong type
            st.pop();
        }
        // any other character is ignored
    }
    return st.empty(); // openers left unclosed
}
```

Trace: "{[()]}"

c	stack	note
-	-----	----
{	[{]	
[	[{,[	
(	[{,[,(	
)	[{,[,	matches top
]	[{,[	matches top
}	[{	matches top
end empty	-	-> TRUE

Trace: "([)]" -- the case that needs a stack

c	stack	note
-	-----	----
(	[(	
[	[(,[	
)	[(,[	top is '[' not '(' -> FALSE detected at the 3rd char

A counter cannot detect this: the count of open brackets is 2 and balanced. Only a stack knows the ORDER.

The three failure modes, and the line that catches each

Input	Failure	Caught by
"())"	Closer with empty stack	<code>if (st.empty()) return false;</code>
"([)]"	Wrong closer type	<code>if (st.top() != ...) return false;</code>
"(()"	Unclosed opener at end	<code>return st.empty();</code>

THE KEY INSIGHT

A counter suffices for a single bracket type; a stack is required the moment there are two or more, because only a stack records *which* opener is innermost. If an interviewer asks “could you do this with $O(1)$ space?”, the answer is **yes for one type, no for several** — and the proof is `()[]`, which every counter accepts.

THE TRAP

Returning `true` at the end of the loop instead of `st.empty()`. `"(("` then passes: no closer ever mismatched, so no early return fired. Every test containing a closing bracket passes, which is every test people write. It is the single most common bug in this problem and the first input a good interviewer supplies.

balanced brackets – Time $O(n)$, one pass · Space $O(n)$ worst case (all openers), $O(1)$ for a single bracket type using a counter · three failure modes, three lines

39. Longest valid parentheses

Given a string of only `(` and `)`, find the length of the longest contiguous valid substring. The jump from Topic 38 is that we now need *positions*, not just balance — so the stack holds **indices, not characters**. That substitution is the single most transferable idea in this part and it reappears throughout Part 7.

```
int longestValidParentheses(const std::string& s) {
    std::stack<int> st;
    st.push(-1); // SENTINEL: base for the first valid run
    int best = 0;
    for (int i = 0; i < (int)s.size(); ++i) {
        if (s[i] == '(') {
            st.push(i);
        } else {
            st.pop(); // consume a matching '(' -- or the base
            if (st.empty())
                st.push(i); // no match: i becomes the new base
            else
                best = std::max(best, i - st.top());
        }
    }
    return best;
}
```

The stack holds indices of unmatched characters. Its top is always the index immediately *before* the current valid run, so `i - st.top()` is that run's length with no arithmetic to get wrong. The `-1` pushed at the start is what makes the very first run work: without it, a string starting `"(")` would pop an empty stack.

Trace: `"()()())"` indices 0..5

i	c	stack (bottom->top)	action	best
-	-	-----	-----	----
-	-	[-1]	sentinel	
0	)	[0]	pop -1, empty -> push 0	0
1	(	[0,1]	push index	0
2	)	[0]	pop 1, top=0, 2-0=2	2
3	(	[0,3]	push index	2
4	)	[0]	pop 3, top=0, 4-0=4	4
5	)	[5]	pop 0, empty -> push 5	4
-	-	-----	-----	----

answer 4 -- the substring `"()()"` at indices 1..4

the stack always holds indices of UNMATCHED characters; its top is the boundary just before the current run, so `i - top()` is the run length.

THE KEY INSIGHT

When a problem asks for a *length*, a *distance*, or a *span*, push indices. When it asks only whether something is valid, push values. This one decision determines whether the arithmetic at the end is trivial or impossible, and it is the same decision you will make twelve more times in Part 7.

THE TRAP

Omitting the `-1` sentinel and special-casing the empty stack instead. It can be made to work, but the off-by-one moves into the length expression, where `i - st.top()` becomes `i - st.top() + 1` in some branches and not others. Symptom: correct answers on strings starting with `)` and answers one too small on strings starting with `(`. Push the sentinel; it costs one line and removes the branch.

longest valid parens – Time $O(n)$, one pass · Space $O(n)$ · index stack with a `-1` sentinel · $O(1)$ -space alternative exists via two counter passes, left-to-right and right-to-left

40. Minimum insertions and score of parentheses

Two variations that show the same stack carrying different payloads. Both are common follow-ups to Topic 38 because they test whether you understood the structure or memorised the code.

Minimum additions to make valid

You do not need a stack at all — with one bracket type, two counters suffice, which is worth saying out loud because it demonstrates the Topic 38 insight.

```
int minAddToMakeValid(const std::string& s) {
    int open = 0, needed = 0;           // open: unmatched '(' so far
    for (char c : s) {                  // needed: ')' that had no partner
        if (c == '(') ++open;
        else if (open > 0) --open;
        else ++needed;
    }
    return open + needed;               // insert one bracket for each
}
```

Score of parentheses

Score `"()"` as 1, `"AB"` as `A + B`, and `"(A)"` as `2 × A`. Now the stack carries *accumulated scores*, one per nesting level, and the pattern generalises directly to Topic 42.

```
int scoreOfParentheses(const std::string& s) {
    std::stack<int> st;
    st.push(0); // score accumulator for the outermost level
    for (char c : s) {
        if (c == '(') {
            st.push(0); // open a new level with score 0
        } else {
            int inner = st.top(); st.pop();
            int add = inner ? 2 * inner : 1; // "()" scores 1, "(A)" scores 2A
            st.top() += add; // fold into the parent level
        }
    }
    return st.top();
}
```

Trace: "((()()))"

c	stack (bottom -> top)	action
(	[0, 0]	open level 1
(	[0, 0, 0]	open level 2
)	[0, 1]	inner=0 -> add 1 to level 1
(	[0, 1, 0]	open level 2 again
(	[0, 1, 0, 0]	open level 3
)	[0, 1, 1]	inner=0 -> add 1
)	[0, 1, 2]	inner=1 -> add 2*1 = 2
)	[6]	inner=3 -> add 2*3 = 6

answer 6. check: (() (())) = 2 * (1 + 2*1) = 2*3 = 6		

THE KEY INSIGHT

Pushing an accumulator per nesting level and folding it into the parent on close is the general shape of every “evaluate a nested structure” problem: this, decode-string (Topic 42), JSON parsing, and directory sizes. The stack depth is the nesting depth, and `st.top()` `+= add` is the fold. Recognise the shape and these stop being separate problems.

THE TRAP

Forgetting the initial `st.push(0)`. The final `)` then folds into an empty stack — `st.top()` on empty is undefined behaviour, so the result is garbage rather than a crash, and on many runs it will be a plausible small integer. The base accumulator plays exactly the role of the `-1` sentinel in Topic 39: it gives the outermost close somewhere to fold into.

min-add $O(n)$ time, **$O(1)$ space** with two counters · score-of-parens $O(n)$ time, $O(\text{depth})$ space · one accumulator per level, folded into the parent on close

41. Path canonicalisation

Turn a Unix path such as `/a/./b/../../c/` into its canonical form `/c`. The stack holds directory *names*, `..` pops, `.` and empty components are skipped. It is a genuine systems routine, not just an exercise — every filesystem API does some version of it, and getting it wrong is a directory-traversal vulnerability.

```

std::string simplifyPath(const std::string& path) {
    std::vector<std::string> st;           // vector: we must iterate at the end
    std::stringstream ss(path);
    std::string part;
    while (std::getline(ss, part, '/')) {
        if (part.empty() || part == ".") continue;    // "/" and "/"
        if (part == "..") {
            if (!st.empty()) st.pop_back();           // ".." at root is a no-op
        } else {
            st.push_back(part);
        }
    }
    std::string out;
    for (const auto& d : st) { out += '/'; out += d; }
    return out.empty() ? "/" : out;                 // root special case
}

```

Input	Output	What it tests
/home/	/home	Trailing slash
/../	/	.. above root
/home//foo/	/home/foo	Empty component
/a/./b/../../c/	/c	Mixed
/...	/...	Three dots is a normal name
/a/../../b	/b	Over-popping at root

THE KEY INSIGHT

This is the first problem in the book where the natural implementation is a `std::vector` used as a stack rather than a `std::stack` — because the final step iterates from the bottom to build the output string. When the answer requires reading the stack in order, use a vector and say why. Reaching for `std::stack` and then popping into a second stack to reverse is two extra allocations and a worse answer.

THE TRAP

`if (part == "..") st.pop_back();` without the emptiness check. On `/../` that is `pop_back()` on an empty vector — undefined behaviour, which in practice sets the size to `(size_t)-1` and turns the subsequent loop into the Topic 17 disaster. In a real filesystem service, the equivalent bug is a **directory traversal vulnerability**: `../../../../etc/passwd` escapes the intended root. Absorbing `..` at the root is both the correct semantics and the security fix.

path canonicalisation — Time $O(n)$ · Space $O(n)$ · use a vector, not `std::stack`, when the output is read bottom-up · `..` at root must be absorbed, not popped

42. Decode string

Expand `3[a2[c]]` into `accaccacc`. Two parallel stacks — one for repeat counts, one for the partially built string of each enclosing level. This is the general nested-structure unwinding pattern from Topic 40, now carrying strings instead of integers.


```

std::string decodeString(const std::string& s) {
    std::stack<int> counts;
    std::stack<std::string> parts;
    std::string cur;
    int num = 0;

    for (char c : s) {
        if (std::isdigit((unsigned char)c)) {
            num = num * 10 + (c - '0'); // MULTI-DIGIT: 12[a] is 12, not 1,2
        } else if (c == '[') {
            counts.push(num); // save the count for this level
            parts.push(cur); // save the parent's text so far
            num = 0;
            cur.clear();
        } else if (c == ']') {
            std::string repeated;
            repeated.reserve(cur.size() * counts.top());
            for (int i = 0; i < counts.top(); ++i) repeated += cur;
            counts.pop();
            cur = parts.top() + repeated; // fold into the parent
            parts.pop();
        } else {
            cur += c;
        }
    }
    return cur;
}

```

Trace: 3[a2[c]]

c	num	cur	counts	parts	action
-	-	-	-	-	-
3	3	""	[]	[]	accumulate digit
[	0	""	[3]	[""]	open level, save parent ""
a	0	"a"	[3]	[""]	append
2	2	"a"	[3]	[""]	accumulate digit
[	0	""	[3,2]	["", "a"]	open level, save parent "a"
c	0	"c"	[3,2]	["", "a"]	append
]	0	"acc"	[3]	[""]	"c"*2 = "cc"; cur = "a" + "cc"
]	0	"acc..."	[]	[]	"acc"*3 = "accaccacc"
-	-	-	-	-	-

answer "accaccacc" (9 chars)

the two stacks always have equal height = current nesting depth

THE KEY INSIGHT

On `[`, save the parent's state and start fresh; on `]`, finish the child and fold it into the restored parent. That is the resumption-point pattern of Topic 26 written without a stage field, because there is only one recursive call site. Every nested-format decoder — this, JSON, XML, nested CSV quoting — is this loop.

THE TRAP

Reading the digit as `num = c - '0'` instead of accumulating with `num * 10 + (c - '0')`. `12[a]` then produces `aa` instead of twelve `as` — and every single-digit test passes, which is the identical failure shape as the lexer trap in Topic 37. A second, more expensive trap: the output can be exponentially larger than the input. `10[10[10[10[10[10[a]]]]]]` is 26 characters and expands to **one million characters**; nest nine deep and you have a gigabyte from a 40-byte input. Bound the output length if the input is untrusted.

decode string – Time $O(\text{output length})$, which can be exponential in input length · Space $O(\text{depth})$ stacks + $O(\text{output})$ · `10[10[10[a]]]` = 26 bytes in, 1000 chars out; nine levels is 1 GB

43. Recursive descent and deep-nesting DoS

A recursive-descent parser is a stack you did not write — the call stack again (Topic 24). For nested formats such as JSON, the nesting depth of the input becomes the recursion depth of the parser, which means **the attacker chooses your stack depth**. That is a denial-of-service vector present in a surprising number of hand-written parsers.

```
// A recursive-descent JSON value parser, abbreviated. Each nested array or
// object costs one frame. NOTHING here limits how many.
void parseValue(Lexer& lx) {
    if (lx.peek() == '[') {
        lx.next();
        while (lx.peek() != ']') { parseValue(lx); if (lx.peek() == ',') lx.next(); }
        lx.next();
    } else if (lx.peek() == '{') {
        /* ... same shape ... */
    } else {
        lx.scalar();
    }
}
```

The attack is one line of shell:

```
$ python3 -c "print([' '*100000 + '']*100000)" > payload.json
$ curl -X POST --data-binary @payload.json http://service/api
```

200 KB of payload -> 100,000 nested frames.
frame ~176 bytes -> 17.6 MB of call stack against an 8 MB limit.
SIGSEGV, no stack trace, no log line, whole process dies -- taking every other in-flight request with it.

Compare with the explicit-stack parser: 100,000 entries on the HEAP, ~3 MB, completes normally. Depth limit then rejects it politely.

The three defences, in order of preference

Defence	Cost	Effect
Explicit depth limit	One counter, one comparison	Rejects with a 400, process survives — do this
Explicit heap stack	Rewrite the parser	Raises the ceiling by ~1000x; still needs a limit
Raise <code>ulimit -s</code>	One line of config	Constant factor only — the attacker just sends more

```
void parseValue(Lexer& lx, int depth = 0) {  
    if (depth > 200) throw ParseError("nesting too deep");    // the whole fix  
    // ... recurse with depth + 1  
}
```

Two hundred is not arbitrary: `serde_json` defaults to 128, Python's `json` module inherits the 1000-frame recursion limit, and Go's `encoding/json` caps at 10,000. Any of those is fine; having no limit is not.

THE KEY INSIGHT

Whenever recursion depth is a function of untrusted input, you have given the attacker a write primitive against your call stack. The fix is not a bigger stack — it is a limit, checked before recursing. Ask this question of every recursive parser, tree walker, or deserialiser that touches the network, including the ones inside libraries you depend on.

THE TRAP

Assuming your language runtime protects you. Python raises a catchable `RecursionError` at 1000 frames — but if the code has called `sys.setrecursionlimit(100000)` to handle “legitimate deep data”, the C stack underneath will overflow first and the process dies with a hard segfault instead of an exception. Raising the recursion limit converts a catchable error into an uncatchable crash. The symptom is a worker that dies silently with no traceback in the logs.

deep-nesting DoS – attack cost $O(n)$ bytes for $O(n)$ frames · 200 KB payload → 17.6 MB of stack → SIGSEGV · fix is one counter and one comparison · typical limits: `serde_json` 128, Go 10,000

PRACTICE SET — Topics 38-43

A. Conceptual

1. For which bracket alphabets can balance be decided in $O(1)$ space? Give the smallest input that proves a counter is insufficient for the general case.
2. Name the three failure modes of bracket matching and the line of code that catches each.
3. Why does the longest-valid-parentheses solution push indices rather than characters? State the general rule this exemplifies.
4. What does the top of the index stack represent at any moment in Topic 39, and why does that make the length expression `i - st.top()` rather than something with a `+1`?
5. What is the role of the `-1` sentinel, and what specifically breaks without it?
6. Why does “minimum additions to make valid” need no stack at all? Under what change to the problem would it need one?
7. Describe the accumulator-per-level pattern in one sentence, and name three problems outside this part that have the same shape.
8. In `simplifyPath`, why is a `std::vector` the right choice over `std::stack`?
9. What are the semantics of `..` at the filesystem root, and what vulnerability class does getting it wrong create?
10. In `decodeString`, what is saved on `[` and what is restored on `]`? Relate this to the resumption-point discussion of Topic 26.
11. Why do the two stacks in `decodeString` always have equal height?
12. Explain why the output of `decodeString` can be exponential in the input length, and give the shortest input you can construct that produces more than one million characters.
13. In what sense does a recursive-descent parser hand control of the call stack to the attacker?
14. Rank the three defences against deep-nesting DoS and justify the ranking in one sentence each.
15. Why can calling `sys.setrecursionlimit(100000)` make a Python service *less* safe?

B. Tracing and analysis

1. Trace `isBalanced("[{()}])"` and `isBalanced("([)])"`, giving the stack after each character and the index at which the second one returns false.
2. Trace `longestValidParentheses("()())"`. Give the stack, the current best, and the index at each step, plus the final answer and the substring it corresponds to.
3. Trace the same function on `"))())"`. Give the final answer and identify every index at which the stack was pushed as a new base.
4. Compute `scoreOfParentheses` by hand for `"((()())"` and for `"()())"`. Show the stack after every character.
5. Run `simplifyPath` by hand on `/a/.b/../../../../c///.../d/`. Give the vector contents after each component and the final output.
6. Give the outputs for `/...`, `/...hidden`, `/a/../../../../b` and `/.`. State which of these a naive implementation gets wrong.
7. Trace `decodeString("2[ab3[c]]")`. Give `num`, `cur`, and both stacks after every character, plus the final string and its length.
8. Compute the output length of `3[2[2[ab]]]` and of `10[10[10[10[a]]]]`. Give the input length in bytes for each and the expansion ratio.

9. A JSON parser has a 176-byte frame and runs on a thread with `ulimit -s 8192`. Compute the payload size in bytes needed to crash it with nested arrays. Then compute it for a 64 MB stack and state the conclusion.

C. Code tasks

1. **Build.** Implement all five algorithms in this part. Verify `isBalanced` on `"{[()]}"`, `"([)]"`, `"((("`, `"())"` and `""`; verify `simplifyPath` against all six rows of the Topic 41 table.
2. **Break it — the missing final check.** Change `isBalanced` to `return true;` at the end. Run your five test strings and report which now give the wrong answer. Explain why a randomly generated test suite would still likely pass.
3. **Break it — the missing sentinel.** Remove `st.push(-1)` from `longestValidParentheses`. Run `"()"`, `"()()())"` and `"()"` under `-fsanitize=address`. Report the ASan error class for the first input and the wrong answers for the others.
4. **Break it — the root pop.** Remove the `if (!st.empty())` guard from `simplifyPath`. Run `/../` and `/a/../../../../etc/passwd`. Report what `st.size()` becomes, what the output string is, and what ASan says. Then state, in one sentence, what this bug would be called in a security report.
5. **Break it — single-digit counts.** Change `decodeString`'s digit handling to `num = c - '0'`. Run `3[a]`, `12[a]` and `2[b3[c]]`. Report which are correct and explain the pattern in which tests hide the bug.
6. **Break it — the DoS.** Write the recursive-descent bracket parser from Topic 43 (it need only count nesting). Generate payloads of 1,000, 10,000, 100,000 and 1,000,000 opening brackets. Report the first depth that segfaults and compare it with the arithmetic from Topic 29. Confirm no exception is thrown and nothing is logged.
7. **Fix, two ways.** First add a depth limit of 200 and confirm the 1,000,000-bracket payload is rejected in microseconds with an exception. Then rewrite the parser with an explicit heap stack, remove the limit, and report the depth at which *that* version fails and what the failure looks like.
8. **Measure.** For the explicit-stack parser, report peak RSS while parsing 1,000,000 nested brackets, and compare it with the 8 MB call-stack budget the recursive version had. State the ratio of achievable depths.

ANSWER KEY — Topics 38-43

A. Conceptual

1. Only a **single** bracket type. Smallest counterexample: `([)]` — four characters, perfectly balanced by count for both types, but invalidly interleaved.
2. Closer on an empty stack → `if (st.empty()) return false;`. Wrong closer type → `if (st.top() != it->second) return false;`. Unclosed openers at the end → `return st.empty();`.
3. Because the answer is a **length**, which requires positions. General rule: **if the problem asks for a length, distance or span, push indices; if it asks only for validity, push values.**
4. The top is the index of the **last unmatched character**, i.e. the boundary immediately before the current valid run. Since it is *excluded* from the run, the count of characters from `st.top()+1` to `i` inclusive is exactly `i - st.top()` — no adjustment needed.
5. It provides a base index before position 0. Without it, a string beginning `"()` pops the stack empty and then reads `st.top()` on an empty stack — undefined behaviour.
6. Because with one bracket type, **counters capture all the state** — there is no ordering question. It would need a stack if the alphabet grew to two or more bracket types.
7. Push a fresh accumulator on open, fold it into the parent's accumulator on close. Same shape: **decode-string, JSON/XML parsing, and directory size computation** (also expression trees and nested transaction savepoints).
8. Because the final step must **iterate the contents bottom-to-top** to build the output string, and `std::stack` deliberately has no iterators. Using `std::stack` forces a second reversing stack — two extra allocations for no benefit.
9. `..` at the root is a **no-op** — it must be absorbed. Getting it wrong creates a **directory traversal vulnerability** (path traversal), letting `../../../../etc/passwd` escape the intended root.
10. On `[`: the repeat **count** and the **parent's partially built string**. On `]`: the parent string is restored and the repeated child is appended to it. It is the resumption-point pattern with **no stage field needed, because there is only one recursive call site**.
11. Because both are pushed exactly once per `[` and popped exactly once per `]`. Their common height is the current nesting depth.
12. Because each nesting level **multiplies** the length by its count. Shortest input over one million characters: `10[10[10[10[10[10[a]]]]]]` — 25 characters producing $10^6 = 1,000,000$. `(9[9[9[9[9[9[a]]]]]])` gives 4,782,969 in 29 characters; the base-10 form is shorter per level.)
13. Because the parser's recursion depth equals the input's nesting depth, and the input is chosen by the attacker. They control how many frames you push, and therefore whether you reach the guard page.
14. (1) **Explicit depth limit** — the only defence that turns a crash into a rejection at $O(1)$ cost. (2) **Explicit heap stack** — raises the ceiling $\sim 1000\times$ but does not remove it, so a limit is still needed. (3) **Raise `ulimit -s`** — a constant factor against an input the attacker sizes freely, so it is not a defence at all.
15. Because the 1000-frame limit is a guard on **CPython's own C stack**. Raising it past what the C stack can support converts a **catchable `RecursionError`** into an **uncatchable `segfault`** — the worker dies with no traceback.

B. Tracing and analysis

1. "{[()]}" : {[}, {[, [], {[, [, (], {[, [], {[}, [] → **true**. "([])" : [(], [(, [], then at **index 2** the closer) finds top [→ **false**.

2.

i	c	stack	best	note
-	-	[-1]	0	
0	(	[-1,0]	0	
1	)	[-1]	2	1-(-1) = 2
2	(	[-1,2]	2	
3	(	[-1,2,3]	2	
4	)	[-1,2]	2	4-2 = 2, not better
5	)	[-1]	6	5-(-1) = 6
answer 6, the whole string "()((()))"				

3. ")))(": bases pushed at **index 0** and **index 1** (both) with nothing to match). Then (at 2, (at 3,) at 4 pops index 3 leaving top = 2, giving 4 - 2 = 2. **Answer 2**.
4. "((()((())))" : as traced in Topic 42's neighbour — **6**. "()" : [0] → [0,0] → [1] → [1,0] → [2] → **2**.

5.

component	vector	
a	[a]	
.	[a]	skipped
b	[a,b]	
..	[a]	
..	[]	
c	[c]	
(empty)	[c]	from "/"
.	[c]	skipped
..	[]	
d	[d]	
output: /d		

6. /... → /... (three dots is an ordinary name). ../hidden → ../hidden. /a/.../b → /b. / → /. A naive implementation typically gets /... **wrong** (by testing `part.starts_with("..")` instead of `part == ".."`) and /a/.../b **wrong** (by popping an empty container).

7.

c	num	cur	counts	parts	note
2	2	""	[]	[]	
[	0	""	[2]	[""]	
a	0	"a"	[2]	[""]	
b	0	"ab"	[2]	[""]	
3	3	"ab"	[2]	[""]	
[	0	""	[2,3]	["", "ab"]	
c	0	"c"	[2,3]	["", "ab"]	
]	0	"abccc"	[2]	[""]	"c"*3, cur = "ab"+"ccc"
]	0	...	[]	[]	"abccc"*2
result "abcccabccc", length 10					

8. 3[2[2[ab]]]: 2×2×3 = 12 repetitions of ab = **24 characters** from **11 bytes**, ratio **2.2×**. 10[10[10[10[a]]]]: 10⁴ = **10,000 characters** from **17 bytes**, ratio **588×**.
9. 8,388,608 / 176 = 47,662 frames. Each nesting level costs **2 bytes** (one [and one]), so the payload is about **95 KB**. For 64 MB: 381,300 frames → about **763 KB**. Conclusion: **an 8× larger stack costs the attacker 8× the bandwidth — roughly nothing**. Constant-factor defences fail against attacker-sized input.

C. Code tasks — what to verify

1. Expected: **true, false, false, false, true**. The empty string returning **true** is the case people forget to test and it is correct — vacuously balanced.

2. Only `"((("` changes, from `false` to `true`. A random test suite passes because random bracket strings almost always contain a mismatch that triggers an early `return false` before the end of the loop is ever reached; you must deliberately generate *openers-only* strings to catch it.
3. `"()"`: ASan reports a read on an empty container — with `std::deque` backing it is often silent, so also run with `-D_GLIBCXX_DEBUG`, which aborts with `attempt to access an element in an empty container`. `"()()())"` gives **2 instead of 4** and `"(())"` gives **0 instead of 2** — both under-report, because the missing base makes every run measure from the wrong origin.
4. `st.size()` becomes **18446744073709551615** (Topic 17 again). The output string is garbage or the program hangs building it. ASan reports `heap-buffer-overflow READ` in the output loop. In a security report this is a **path traversal / directory traversal vulnerability (CWE-22)**.
5. `3[a]` **correct** (`aaa`); `12[a]` **wrong** (`aa` instead of twelve); `2[b3[c]]` **correct**. The pattern: **every count below 10 passes**, and hand-written tests almost always use small counts — the same failure shape as single-character lexing in Topic 37.
6. With a ~176-byte frame the segfault should appear between roughly **40,000 and 60,000** nested brackets, so 1,000 and 10,000 succeed and 100,000 crashes. Confirm: **no exception, no log line, exit code 139 (SIGSEGV)**. Compare against 8,388,608 / frame-size from your own `objdump` measurement — it should agree within about 20%.
7. With the limit: the 1,000,000-bracket payload is rejected after **201 frames**, in **microseconds**, with your exception — and note that it fails *faster* than the crashing version did. The explicit-stack version handles **1,000,000 levels comfortably**; it fails only when heap allocation fails, typically in the **tens of millions**, and the failure is a catchable `std::bad_alloc` rather than a signal.
8. 1,000,000 stack entries at a few bytes each is roughly **4-16 MB of RSS**, against an 8 MB call-stack budget that supported only ~47,000 levels. Achievable-depth ratio is on the order of **20-1000×** depending on entry size — and, unlike the stack, it degrades gracefully.

HANDNOTE SUMMARY CARD

PART 6 - MATCHING AND NESTING

Topics 38-43

BALANCED BRACKETS - three failure modes, three lines

```
closer on empty stack -> if (st.empty()) return false;      "()"
wrong closer type     -> if (st.top() != match) return false; "[]"
openers left at end   -> return st.empty();                 "((("
<-- forgetting the LAST one is the #1 bug; every test with a ')' passes
ONE bracket type -> a counter is enough, 0(1) space.
TWO or more        -> stack required. Proof: "[()]" balances by count.
```

INDEX STACKS - the rule that carries into Part 7

asks for VALIDITY -> push values
asks for LENGTH / DISTANCE / SPAN -> push INDICES

LONGEST VALID PARENTHESES

```
st.push(-1);                                <- SENTINEL: base before position 0
'(' -> push i
')' -> pop; if empty push i (new base) else best = max(best, i - st.top())
top() is the last UNMATCHED index, excluded from the run -> no +1 needed
without the sentinel: UB on "()" and every answer measured from the
wrong origin ("()())" gives 2 not 4)
```

MIN ADD TO MAKE VALID - no stack at all, 0(1) space

```
open = 0, needed = 0; '(' -> ++open; ')' -> open ? --open : ++needed;
answer = open + needed
```

SCORE OF PARENTHESES - accumulator per level

```
st.push(0);                                <- base accumulator, same role as -1
'(' -> st.push(0)
')' -> inner = pop(); st.top() += (inner ? 2*inner : 1)
THE GENERAL SHAPE of every nested-structure evaluation:
  push a fresh accumulator on open, FOLD into the parent on close
  (score, decode-string, JSON, XML, directory sizes, nested savepoints)
```

SIMPLIFY PATH - use std::vector, NOT std::stack (output is read bottom-up)

```
split on '/'; skip "" and "."; ".." pops IF NON-EMPTY; else push
".." at root is a NO-OP - absorbing it is the security fix
missing the empty check = path traversal, CWE-22
/... -> /... (three dots is a normal name: compare ==, not starts_with)
/a/../../b -> /b      /../ -> /      /home// -> /home
```

DECODE STRING - two stacks, equal height = nesting depth

```
digit -> num = num*10 + (c-'0')             <- MULTI-DIGIT or 12[a] gives "aa"
'[' -> counts.push(num); parts.push(cur); num=0; cur.clear();
']' -> cur = parts.top() + repeat(cur, counts.top()); pop both
other -> cur += c
OUTPUT IS EXPONENTIAL: 10[10[10[10[10[10[a]]]]]] = 25 bytes -> 1,000,000
bound the output if the input is untrusted.
```

DEEP-NESTING DoS - the attacker chooses your stack depth

```
recursive descent: input nesting depth == parser recursion depth
176 B frame, 8 MB -> 47,662 levels -> a 95 KB payload kills the process
SIGSEGV, no exception, no log, takes every in-flight request with it
DEFENCES, in order:
  1. depth limit (one counter)           -> 400 instead of a crash. DO THIS.
  2. explicit heap stack                  -> ~1000x ceiling, still needs a limit
  3. raise ulimit -s                     -> constant factor. NOT a defence.
real limits: serde_json 128, Go encoding/json 10000, Python 1000 frames
sys.setrecursionlimit(100000) makes it WORSE: catchable error -> segfault
```


PART SEVEN

The monotonic stack

The densest interview material in the subject. One invariant, twelve problems. Learn what “unresolved” means and the four-way direction table, and next-greater, daily temperatures, histogram, rain water and subarray minimums stop being separate problems.

44. The invariant and what “unresolved” means

A **monotonic stack** is an ordinary stack maintained so its contents are always increasing or always decreasing. That is the mechanical description and it is not the useful one. The useful description is:

The stack holds the elements whose answer is not yet known, and the element that resolves them is the one that pops them.

Consider the next-greater-element problem. Scanning left to right, when you meet a value you cannot yet say what its next greater element is — it lies somewhere to the right, unseen. So you set it aside. Later, some value arrives that *is* greater; it resolves every set-aside element smaller than itself, and those are exactly the ones at the top of the stack. The monotonicity is not the goal; it is a consequence of always resolving from the top.

Why the stack is automatically monotonic

If x is pushed while y is on top and $x > y$, then y would have been resolved by x -- so y was popped first. Therefore anything remaining below x is $\geq x$. The order maintains itself; you never sort anything.

Scanning [2, 1, 5] for next greater, stack holds INDICES:

$i=0, v=2$	stack empty	push 0	stack: [2]
$i=1, v=1$	$1 < 2$, resolves nothing	push 1	stack: [2, 1]
$i=2, v=5$	$5 > 1 \rightarrow$ pop, ans[1]=5		
	$5 > 2 \rightarrow$ pop, ans[0]=5		
	push 2		stack: [5]

Total pushes 3, total pops 2. Each index enters and leaves ONCE.
That is the amortised $O(n)$ argument, in full.

The complexity argument, which interviewers ask for

The inner **while** loop looks like it makes the algorithm $O(n^2)$. It does not, and the proof is one sentence: **every index is pushed exactly once and popped at most once, so the total number of pop operations across the entire run is at most n .** The loop can be long on one iteration only because it was short on many others. This is the same amortisation reasoning as Topic 9, in a different costume.

```
// The universal skeleton. Everything in Part 7 is this with two decisions
// filled in: the comparison, and what to compute when a pop happens.
std::stack<int> st; // indices, almost always
for (int i = 0; i < n; ++i) {
    while (!st.empty() && COMPARE(a[st.top()], a[i])) {
        int j = st.top(); st.pop();
        RESOLVE(j, i); // a[i] is j's answer;
    } // st.top() is j's other boundary
    st.push(i);
}
// anything still on the stack has NO answer -- handle the default here
```

THE KEY INSIGHT

Do not memorise “increasing stack for X, decreasing for Y.” Ask instead: *what is each element waiting for, and what arrival resolves it?* The comparison then writes itself, and you will get it right under pressure instead of recalling the wrong half of a table.

THE TRAP

Forgetting the elements left on the stack at the end. They are precisely the ones with no answer — no greater element to the right, no smaller bar to the right — and every problem needs an explicit default for them: -1 for next-greater, or the array end as a boundary for histogram problems. Symptom: correct answers everywhere except the suffix of the array, typically the last one to three positions on a decreasing input.

monotonic stack — Time $O(n)$: each index pushed once, popped at most once · Space $O(n)$ worst case (sorted input) · the stack holds unresolved elements, not sorted data

45. Next greater element and the four-way table

Four closely related questions arise constantly, and they differ in exactly two bits: the scan direction and the comparison operator. Getting the table straight once saves you deriving it four times.

Want	Scan	Pop while	Stack is
Next greater to the right	Left → right	$a[\text{top}] < a[i]$	Decreasing
Next smaller to the right	Left → right	$a[\text{top}] > a[i]$	Increasing
Previous greater to the left	Left → right	$a[\text{top}] \leq a[i]$	Decreasing — read <code>st.top()</code> before pushing
Previous smaller to the left	Left → right	$a[\text{top}] \geq a[i]$	Increasing — read <code>st.top()</code> before pushing

Note the structural difference between the two halves. For *next* questions the answer is delivered **when an element is popped** — the arriving element is the answer. For *previous* questions the answer is read **just before pushing** — whatever survives on the stack is the nearest qualifying element to the left. Same loop, two different reading points.

```
// Next greater element to the right; -1 where none exists.
std::vector<int> nextGreater(const std::vector<int>& a) {
    int n = a.size();
    std::vector<int> ans(n, -1);          // default for the never-resolved
    std::stack<int> st;                  // indices, decreasing values
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[st.top()] < a[i]) {
            ans[st.top()] = a[i];        // a[i] resolves this index
            st.pop();
        }
        st.push(i);
    }
    return ans;                          // leftovers keep -1
}

// Previous smaller element to the left; -1 where none exists.
std::vector<int> prevSmaller(const std::vector<int>& a) {
    int n = a.size();
    std::vector<int> ans(n, -1);
    std::stack<int> st;                  // indices, increasing values
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[st.top()] >= a[i]) st.pop();
        if (!st.empty()) ans[i] = a[st.top()]; // READ before pushing
        st.push(i);
    }
    return ans;
}
```

a = [2, 1, 2, 4, 3]		next greater to the right		
i	a[i]	pops	stack (indices)	ans so far
0	2	-	[0]	[-1, -1, -1, -1, -1]
1	1	- (1 < 2)	[0, 1]	[-1, -1, -1, -1, -1]
2	2	pop 1 -> ans[1]=2	[0, 2]	[-1, 2, -1, -1, -1]
3	4	pop 2 -> ans[2]=4		
		pop 0 -> ans[0]=4	[3]	[4, 2, 4, -1, -1]
4	3	- (3 < 4)	[3, 4]	[4, 2, 4, -1, -1]
end leftovers 3 and 4 keep -1				=> [4, 2, 4, -1, -1]

THE KEY INSIGHT

Next answers are written at pop time; *previous* answers are read at push time. If you find yourself needing a second pass or a reversed scan to answer a “previous” question, you have missed this — both directions come out of a single left-to-right pass over the same stack.

THE TRAP

Storing values instead of indices. It works for plain next-greater and fails for everything else in this part, because distances, widths and boundaries all need positions. Converting later means rewriting the loop. Symptom: you solve next-greater in five minutes, are asked for daily temperatures, and have to start over. **Always push indices**; read `a[st.top()]` when you need the value.

next/previous greater/smaller – Time $O(n)$ · Space $O(n)$ · next → write at pop; previous → read before push · direction and comparison are the only two variables

46. Duplicates: > vs >=

The single most consequential character in Part 7 is whether the pop condition uses a strict or non-strict comparison. On distinct inputs it makes no difference, which is why it survives testing; on inputs with equal values it decides whether you count some ranges twice or drop them entirely.

```
while (!st.empty() && a[st.top()] < a[i]) { ... } // equals STAY on the stack
while (!st.empty() && a[st.top()] <= a[i]) { ... } // equals are POPPED
```

Problem	Correct condition	What the choice controls
Next strictly greater	<code>a[top] < a[i]</code>	Equal values must not resolve each other
Next greater or equal	<code>a[top] <= a[i]</code>	Equal values do resolve
Largest rectangle (Topic 50)	<code>a[top] >= a[i]</code>	Either works — equal bars pop each other and the widest wins
Sum of subarray minimums (Topic 53)	strict one side, non-strict the other	Prevents double counting — see below

The asymmetric rule for contribution problems

When each element contributes over the range where it is the minimum (or maximum), equal values create overlapping ranges. In `[2, 2]`, if both 2s claim “I am the minimum of `[2,2]`”, that subarray is counted twice. The fix is to break the tie in one direction only:

```
a = [2, 2]      subarrays: [2], [2], [2,2]  -> sum of minimums = 2+2+2 = 6

SYMMETRIC (both strict): each 2 claims the range covering both positions
left 2 : ranges [0,0], [0,1]  -> 2 subarrays
right 2: ranges [1,1], [0,1]  -> 2 subarrays
[0,1] counted TWICE -> total 8.  WRONG.

ASYMMETRIC (previous strictly smaller, next smaller-OR-EQUAL):
left 2 : previous < 2 none (left bound -1)
        next <= 2 is index 1  -> ranges [0,0]          = 1
right 2: previous < 2 none (left bound -1)
        next <= 2 none (right bound n) -> [1,1], [0,1] = 2
total 3 subarrays x value 2 = 6.  CORRECT.

RULE: make one side strict and the other non-strict. Which side is
arbitrary, as long as you are consistent -- ties then belong to
exactly one owner.
```

THE KEY INSIGHT

For contribution problems, ties must have exactly one owner. Enforce it by making one boundary strict and the other non-strict — conventionally “previous strictly smaller, next smaller or equal.” Say that phrase out loud in an interview and you have shown you understand the failure mode, not just the template.

THE TRAP

Testing only on arrays of distinct values. Every random test with distinct integers passes regardless of which comparison you chose, so the bug ships. The minimum test set for any Part 7 problem must include `[2,2]`, `[3,3,3]`, and `[1,2,2,1]`. If your answer for `[2,2]` in Topic 53 is 8 rather than 6, this is why.

duplicates – no complexity change · strict vs non-strict decides tie ownership · contribution problems need one strict side and one non-strict · always test `[2,2]`

47. Daily temperatures

Given daily temperatures, for each day report how many days you must wait for a warmer one. It is next-greater-element with the answer expressed as a *distance* rather than a value — which is precisely why the stack must hold indices, as Topic 45 warned.

```
std::vector<int> dailyTemperatures(const std::vector<int>& t) {
    int n = t.size();
    std::vector<int> ans(n, 0);          // 0 = no warmer day ever comes
    std::stack<int> st;                  // indices, decreasing temperatures
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && t[st.top()] < t[i]) {
            int j = st.top(); st.pop();
            ans[j] = i - j;              // THE DISTANCE, not t[i]
        }
        st.push(i);
    }
    return ans;
}
```

t = [73, 74, 75, 71, 69, 72, 76, 73]

i	t[i]	pops (j -> ans[j] = i-j)	stack (indices)
-	-	-	-
0	73	-	[0]
1	74	0 -> ans[0] = 1-0 = 1	[1]
2	75	1 -> ans[1] = 2-1 = 1	[2]
3	71	-	[2,3]
4	69	-	[2,3,4]
5	72	4 -> ans[4] = 5-4 = 1 3 -> ans[3] = 5-3 = 2	[2,5]
6	76	5 -> ans[5] = 6-5 = 1 2 -> ans[2] = 6-2 = 4	[6]
7	73	-	[6,7]

end: indices 6 and 7 unresolved -> ans stays 0

ans = [1, 1, 4, 2, 1, 1, 0, 0]
total pushes 8, total pops 6 -- never more than n of each

THE KEY INSIGHT

Three problems, one loop: next-greater writes `a[i]`, daily temperatures writes `i - j`, stock span writes `i - st.top()` after popping. The only thing that changes is the expression inside the pop. Once you see that, the whole family collapses to one thing you can hold in your head.

THE TRAP

Initialising the answer array to -1 out of habit from next-greater. The problem asks for a *number of days*, and the required default for “never gets warmer” is **0**. Every mid-array answer is correct and only the trailing decreasing run is wrong — typically the last one or two entries, which a quick eyeball of a passing-looking output will miss.

daily temperatures — Time $O(n)$ · Space $O(n)$, worst case a strictly decreasing input holding all n indices · write $i - j$ at pop · default 0, not -1

48. Circular arrays

When the array wraps — the next greater element may be found by continuing past the end and around to the beginning — the fix is to scan $2n$ positions with modulo indexing, pushing only during the first pass.

```
std::vector<int> nextGreaterCircular(const std::vector<int>& a) {
    int n = a.size();
    std::vector<int> ans(n, -1);
    std::stack<int> st;
    for (int i = 0; i < 2 * n; ++i) {
        int idx = i % n;
        while (!st.empty() && a[st.top()] < a[idx]) {
            ans[st.top()] = a[idx];
            st.pop();
        }
        if (i < n) st.push(idx);      // PUSH ONLY IN THE FIRST PASS
    }
    return ans;
}
```

The `if (i < n)` is the whole algorithm. The second pass exists only to *resolve* indices still waiting; pushing during it would create duplicate entries and, on a strictly increasing input, could double the work and leave stale indices unresolved.

`a = [1, 2, 1]` `n = 3, scanning i = 0..5`

i	idx	a[idx]	action	stack
0	0	1	push (first pass)	[0]
1	1	2	pop 0 -> ans[0]=2; push 1	[1]
2	2	1	push (first pass)	[1,2]
3	0	1	1 < 1 no pop; NO push	[1,2]
4	1	2	pop 2 -> ans[2]=2; NO push	[1]
5	2	1	1 < 2 no pop; NO push	[1]

end: index 1 (value 2) unresolved -> ans[1] = -1

`ans = [2, -1, 2]` the global maximum is always the one left over

THE KEY INSIGHT

Two passes always suffice, never three, because any element that has no greater value in a full wrap-around has none at all — it is a global maximum. That one-sentence justification is what an interviewer is checking when they ask “why $2n$ and not $3n$?”

THE TRAP

Pushing during the second pass. On an input like `[5,4,3,2,1]` the stack accumulates a second copy of every index, so the final leftovers include stale duplicates and `ans` entries get overwritten with values from the wrong pass. Symptom: correct output on inputs with a large early maximum, wrong output on decreasing inputs. The guard is four characters.

circular next-greater – Time $O(2n) = O(n)$ · Space $O(n)$ · push only when `i < n` · leftovers are exactly the global maxima

49. Stock span

For each day, count the number of consecutive preceding days (including today) whose price was less than or equal to today's. This is a *previous greater element* question with the answer as a span, and it is the same loop again with a different expression at the read point.

```
std::vector<int> stockSpan(const std::vector<int>& p) {
    int n = p.size();
    std::vector<int> span(n);
    std::stack<int> st;                // indices, decreasing prices
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && p[st.top()] <= p[i]) st.pop();
        span[i] = st.empty() ? i + 1 : i - st.top(); // READ before pushing
        st.push(i);
    }
    return span;
}
```

`p = [100, 80, 60, 70, 60, 75, 85]`

i	p[i]	pops	stack after pops	span[i]
-	-	-	-	-
0	100	-	<code>[]</code>	empty -> <code>0+1 = 1</code>
1	80	-	<code>[0]</code>	<code>1 - 0 = 1</code>
2	60	-	<code>[0,1]</code>	<code>2 - 1 = 1</code>
3	70	pop 2	<code>[0,1]</code>	<code>3 - 1 = 2</code>
4	60	-	<code>[0,1,3]</code>	<code>4 - 3 = 1</code>
5	75	pop 4, pop 3	<code>[0,1]</code>	<code>5 - 1 = 4</code>
6	85	pop 5, pop 1	<code>[0]</code>	<code>6 - 0 = 6</code>

`span = [1, 1, 1, 2, 1, 4, 6]`

check day 5 (price 75): days 2,3,4,5 have prices 60,70,60,75 all ≤ 75 , and day 1 has $80 > 75$ -- so the span is 4. Correct.

The streaming variant

Stock span is usually posed online: prices arrive one at a time and each span must be returned immediately. The same code works unchanged — keep the stack as member state and expose a `next(price)` method. This is worth mentioning in an interview because it shows the algorithm needs no lookahead, unlike almost everything else in Part 7.

THE KEY INSIGHT

The empty-stack case (`i + 1`) is the same idea as Topic 39's `-1` sentinel: it means “the boundary is just before position 0.” You can eliminate the branch entirely by pushing a sentinel index of `-1` with value `+∞` before the loop, after which `span[i] = i - st.top()` is unconditional. Choose either, but know they are the same device.

THE TRAP

Using `<` instead of `<=` in the pop condition. The problem says “less than *or equal to*”, so equal prices must be absorbed into the span. On `[70, 70]` the strict version returns `[1, 1]` instead of `[1, 2]`. Distinct-value tests never catch it — Topic 46's trap, in its most common concrete form.

```
stock span – Time  $O(n)$  amortised,  $O(1)$  amortised per day online · Space  $O(n)$  · read i - st.top()
before pushing · <= because equal prices count
```

50. Largest rectangle in histogram

Given bar heights of width 1, find the largest axis-aligned rectangle that fits. This is the canonical hard monotonic-stack problem and the one most worth being able to write cold. The brute force is $O(n^2)$; the stack solution is $O(n)$ and the reasoning is short once framed correctly.

The framing. Every maximal rectangle has some bar as its limiting height. For bar j , the widest rectangle of height `h[j]` extends left until the first bar shorter than `h[j]` and right until the first bar shorter than `h[j]`. So the problem is: for every bar, find its previous smaller and next smaller boundaries. That is Topic 45, twice — and the stack gives both at once.

```
long largestRectangleArea(std::vector<int> h) {
    h.push_back(0); // SENTINEL: flushes the stack at the end
    std::stack<int> st; // indices, increasing heights
    long best = 0;
    for (int i = 0; i < (int)h.size(); ++i) {
        while (!st.empty() && h[st.top()] >= h[i]) {
            int height = h[st.top()]; st.pop();
            // left boundary is the new top; right boundary is i. Both exclusive.
            int width = st.empty() ? i : i - st.top() - 1;
            best = std::max(best, (long)height * width);
        }
        st.push(i);
    }
    return best;
}
```

```

h = [2, 1, 5, 6, 2, 3] + sentinel 0

      6      #
      5      # #
      4      # #
      3      # #  #
      2  #   # #  # #
      1  # # # # # #
        2 1 5 6 2 3 0
idx    0 1 2 3 4 5 (6=sentinel)

i  h[i]  pops (height, width, area)                                stack
-  - - -
0  2      -                                                            [0]
1  1      pop 0: h=2, st empty -> w=1,  area=2                        [1]
2  5      -                                                            [1,2]
3  6      -                                                            [1,2,3]
4  2      pop 3: h=6, top=2 -> w=4-2-1=1, area=6
          pop 2: h=5, top=1 -> w=4-1-1=2, area=10  <-- best
                                                    [1,4]
5  3      -                                                            [1,4,5]
6  0      pop 5: h=3, top=4 -> w=6-4-1=1, area=3
          pop 4: h=2, top=1 -> w=6-1-1=4, area=8
          pop 1: h=1, st empty -> w=6,  area=6
          -----
answer 10  (height 5, spanning bars 2 and 3 -- width 2)

```

The width formula, which is where everyone slips

After popping index j , the element now on top is the nearest bar to the left that is *strictly shorter*, and i is the nearest to the right that is shorter. Both are exclusive boundaries, so the span of bars at least as tall as $h[j]$ runs from $st.top()+1$ to $i-1$ inclusive, giving width $i - st.top() - 1$. When the stack is empty there is no left boundary, so the run starts at index 0 and the width is simply i .

THE KEY INSIGHT

The sentinel `0` appended to the array replaces an entire duplicated drain loop after the main loop. Because no bar can be shorter than 0, it pops everything. Sentinels have now appeared four times — Topic 39's `-1`, Topic 40's base accumulator, Topic 49's empty case, and here. Adding a boundary element to eliminate a special case is one of the most transferable tricks in the book.

THE TRAP

Writing `width = i - st.top()` without the `- 1`. Every rectangle is one unit too wide, so the answer is too large by roughly one bar's height — a plausible number that passes eyeball inspection. The discriminating test is a single bar, `[5]`: correct output 5, buggy output 5 as well (stack empty path). Use `[2,1,2]`: correct 3, buggy 4. Test both branches of the width formula deliberately.

largest rectangle — Time $O(n)$ vs $O(n^2)$ brute force · Space $O(n)$ · width = `i - st.top() - 1`, or `i` when empty · sentinel `0` replaces the drain loop

51. Maximal rectangle in a binary matrix

Given a matrix of 0s and 1s, find the largest all-1 rectangle. Nothing new is required — this is Topic 50 run once per row, on a heights array built incrementally. It is the clearest example in the book of composition: recognise the reduction and the problem is already solved.

```
int maximalRectangle(const std::vector<std::vector<char>>& m) {
    if (m.empty()) return 0;
    int cols = m[0].size();
    std::vector<int> h(cols, 0);           // histogram of consecutive 1s above
    long best = 0;
    for (const auto& row : m) {
        for (int c = 0; c < cols; ++c)
            h[c] = (row[c] == '1') ? h[c] + 1 : 0;    // reset on a 0
        best = std::max(best, largestRectangleArea(h)); // Topic 50, unchanged
    }
    return (int)best;
}
```

matrix	heights array after each row
1 0 1 0 0	row 0: [1, 0, 1, 0, 0] max area 1
1 0 1 1 1	row 1: [2, 0, 2, 1, 1] max area 3 (bars 2,3,4 at h=1)
1 1 1 1 1	row 2: [3, 1, 3, 2, 2] max area 6 <-- h=2, width 3
1 0 0 1 0	row 3: [4, 0, 0, 3, 0] max area 4

answer 6: rows 1-2, columns 2-4, all ones.

The 0 at row 3 column 1 RESETS h[1] to 0 rather than decrementing -- a rectangle cannot span a gap, so the run of 1s above is worthless.

THE KEY INSIGHT

Nothing new: this is the histogram algorithm plus a running heights array. When an interviewer asks for maximal rectangle, the winning move is to say “this reduces to largest-rectangle-in-histogram per row” and then write the four-line reduction. Recognising a reduction is worth more than knowing a second algorithm.

THE TRAP

`h[c]--` or leaving `h[c]` unchanged on a 0 instead of resetting to 0. The heights array then describes a histogram that does not exist, because a rectangle cannot cross a gap. Symptom: answers that are too large, appearing only on matrices with interior zeros — a full-ones test matrix passes perfectly. Also note the $O(\text{rows} \times \text{cols})$ total cost holds only because `largestRectangleArea` is $O(\text{cols})$; calling an $O(\text{cols}^2)$ version per row silently makes the whole thing cubic.

maximal rectangle — Time $O(\text{rows} \times \text{cols})$ · Space $O(\text{cols})$ · pure reduction to Topic 50 · reset to 0 on a zero cell, never decrement

52. Trapping rain water: stack vs two pointers

Given an elevation map, compute the water trapped after rain. There are two good solutions and knowing both — and which is better — is the point of this topic.

The stack solution: accumulate horizontal layers

```
long trapStack(const std::vector<int>& h) {
    std::stack<int> st;           // indices, decreasing heights
    long water = 0;
    for (int i = 0; i < (int)h.size(); ++i) {
        while (!st.empty() && h[st.top()] < h[i]) {
            int bottom = h[st.top()]; st.pop();
            if (st.empty()) break;    // no left wall -- water escapes
            int width = i - st.top() - 1;
            int height = std::min(h[st.top()], h[i]) - bottom;
            water += (long)width * height;
        }
        st.push(i);
    }
    return water;
}
```

The two-pointer solution: accumulate vertical columns

```
long trapTwoPointer(const std::vector<int>& h) {
    int l = 0, r = (int)h.size() - 1;
    int leftMax = 0, rightMax = 0;
    long water = 0;
    while (l < r) {
        if (h[l] < h[r]) {           // the smaller side is the binding one
            leftMax = std::max(leftMax, h[l]);
            water += leftMax - h[l];
            ++l;
        } else {
            rightMax = std::max(rightMax, h[r]);
            water += rightMax - h[r];
            --r;
        }
    }
    return water;
}
```

h = [0,1,0,2,1,0,1,3,2,1,2,1] answer 6

```

3           #
2           # ~ ~ #
1      # ~ # ~ ~ # # ~ # #
0  #  #  #  #  #  #  #  #  #  #
   0 1 0 2 1 0 1 3  2 1 2 1
      (~ = trapped water, 6 units total)
```

STACK: fills LAYER BY LAYER, horizontally
each pop computes one flat slab of width (i - top - 1)
and height min(h[top], h[i]) - bottom

TWO POINTERS: fills COLUMN BY COLUMN, vertically
each step computes water above one index: leftMax - h[l]

Same total, orthogonal decompositions.

cost: stack O(n) time, O(n) space
 two pointers O(n) time, O(1) space <-- strictly better here

THE KEY INSIGHT

Two pointers dominate the stack here: same time, $O(1)$ space instead of $O(n)$, and no stack discipline to get wrong. The honest interview answer is “I can do this with a monotonic stack, but two pointers is $O(1)$ space, so that is what I would ship.” A book that only argued for stacks would not tell you that — and this is one of the two places in Part 7 where the stack is not the best tool.

THE TRAP

Omitting the `if (st.empty()) break;` in the stack version. Without a left wall the water escapes, but the code computes a slab against a nonexistent boundary — reading `st.top()` on an empty stack, which is undefined behaviour and typically yields a large positive overcount. Symptom: inputs beginning with a descent, like `[3,0,1]`, return water where there is none. Correct answer: 0.

trapping rain water – stack: $O(n)$ time, $O(n)$ space, horizontal slabs · two pointers: $O(n)$ time, **$O(1)$ space**, vertical columns · prefer two pointers

53. Sum of subarray minimums

Sum `min(subarray)` over all contiguous subarrays. With $n = 30,000$ there are about 450 million subarrays, so enumeration is hopeless. The **contribution technique** inverts the question: instead of asking what each subarray's minimum is, ask *how many subarrays each element is the minimum of*.

If element j has its previous strictly smaller element at index L and its next smaller-or-equal at index R (both exclusive), then j is the minimum of exactly $(j - L) \times (R - j)$ subarrays: any left endpoint in $[L, j]$ paired with any right endpoint in $[j, R)$.

```
long sumSubarrayMins(const std::vector<int>& a) {
    const long MOD = 1000000007;
    int n = a.size();
    std::stack<int> st;                // indices, increasing values
    long total = 0;
    for (int i = 0; i <= n; ++i) {
        // sentinel at i == n behaves like -infinity, flushing the stack
        while (!st.empty() && (i == n || a[st.top()] >= a[i])) {
            int j = st.top(); st.pop();
            int L = st.empty() ? -1 : st.top();    // previous strictly smaller
            int R = i;                            // next smaller-or-equal
            total = (total + (long)a[j] % MOD * (j - L) % MOD * (R - j)) % MOD;
        }
        if (i < n) st.push(i);
    }
    return total;
}
```

```
a = [3, 1, 2, 4]      all 10 subarrays and their minimums
```

```
[3]=3  [3,1]=1  [3,1,2]=1  [3,1,2,4]=1
      [1] =1  [1,2] =1  [1,2,4] =1
                [2] =2  [2,4] =2
                        [4] =4
```

```
sum = 3+1+1+1+1+1+2+2+4 = 17
```

By contribution:

```
j=0 (3): L=-1, R=1 -> (0-(-1)) * (1-0) = 1*1 = 1 subarray -> 3
j=1 (1): L=-1, R=4 -> (1-(-1)) * (4-1) = 2*3 = 6 subarrays -> 6
j=2 (2): L=1, R=4 -> (2-1) * (4-2) = 1*2 = 2 subarrays -> 4
j=3 (4): L=2, R=4 -> (3-2) * (4-3) = 1*1 = 1 subarray -> 4
total = 3 + 6 + 2 + 1 = 12. Matches.
```

Check the counts sum to $n(n+1)/2 = 10$: $1 + 6 + 2 + 1 = 10$. <-- the self-check that catches asymmetry bugs immediately.

THE KEY INSIGHT

Contribution is the general escape from “sum something over all subarrays.” Do not enumerate the subarrays; for each element compute how many subarrays it dominates, then multiply. It applies unchanged to sum of subarray maximums, sum of subarray ranges (max minus min, run it twice), and count-of-subarrays-where-x-is-the-minimum.

THE TRAP

Symmetric comparisons on both boundaries. On `[2,2]` both 2s claim the subarray `[0,1]`, giving 8 instead of the correct 6 — and the sum of contributions comes to 4 rather than $n(n+1)/2 = 3$. Use that identity as an assertion: **the counts must sum to exactly $n(n+1)/2$** . It catches the asymmetry bug on the first run instead of after a failed submission. See Topic 46.

sum of subarray minimums – Time $O(n)$ vs $O(n^2)$ enumeration · Space $O(n)$ · contribution = $(j-L) \times (R-j)$ · counts must sum to $n(n+1)/2$ · one strict boundary, one non-strict

54. Remove K digits

Given a numeric string, remove exactly k digits to leave the smallest possible number. This is a **greedy** problem that a stack implements: whenever a digit is larger than the one that follows it, removing it reduces the number, so remove it — and a stack is exactly the structure for “undo the previous choice.”

```
std::string removeKdigits(const std::string& num, int k) {
    std::string st; // a string used as a stack
    for (char c : num) {
        while (k > 0 && !st.empty() && st.back() > c) {
            st.pop_back(); --k; // this digit was too big -- drop it
        }
        st.push_back(c);
    }
    while (k-- > 0 && !st.empty()) st.pop_back(); // still owe removals: cut the tail
    std::size_t i = 0;
    while (i < st.size() && st[i] == '0') ++i; // strip leading zeros
    std::string out = st.substr(i);
    return out.empty() ? "0" : out; // "" is not a number
}
```



```
num = "1432219", k = 3
```

c	k	action	stack after
-	----	-----	-----
1	3	push	"1"
4	3	'1' > '4'? no -> push	"14"
3	3->2	'4' > '3' -> pop; '1' > '3'? no -> push	"13"
2	2->1	'3' > '2' -> pop; '1' > '2'? no -> push	"12"
2	1	'2' > '2'? no -> push	"122"
1	1->0	'2' > '1' -> pop; k now 0 -> push	"121"
9	0	k exhausted -> push	"1219"

answer "1219"			

Three post-loop cases, all required:

k not exhausted	"112" k=1 -> nothing ever pops -> cut the TAIL -> "11"
leading zeros	"10200" k=1 -> stack "0200" -> strip -> "200"
empty result	"10" k=2 -> stack "" -> return "0"

THE KEY INSIGHT

Removing a digit that is larger than its successor always lowers the number, because the more significant position gets a smaller digit. Doing it greedily from the left is optimal, and the stack is what makes “greedily undo” efficient. The same shape solves “remove duplicate letters to get the lexicographically smallest result” and “create maximum number” — the comparison flips, the structure does not.

THE TRAP

Handling only the main loop. Three post-loop cases each break a different input: **"112"**, **k=1** never triggers a pop (the digits are non-decreasing) so you must cut from the tail; **"10200"**, **k=1** leaves **"0200"**, which must be stripped to **"200"**; and **"10"**, **k=2** leaves an empty string, which must become **"0"**. Any one of these missing gives a wrong answer on a specific, commonly-tested input.

remove K digits – Time $O(n)$, each digit pushed and popped once · Space $O(n)$ · greedy: pop while `st.back() > c` and `k > 0` · three post-loop cases are all mandatory

55. Recognising a monotonic-stack problem

Twelve topics, one structure. This is the decision procedure to run when you meet an array problem under time pressure — it takes about fifteen seconds and produces the code.

Step 1 — does the phrasing match?

Phrase in the problem	Technique
"next greater / smaller element"	Topic 45 directly
"days until", "how long before"	Next greater, write $i - j$ (Topic 47)
"span", "consecutive preceding"	Previous greater, read before push (Topic 49)
"largest rectangle", "maximal area"	Histogram (Topic 50)
"circular array", "wraps around"	$2n$ pass, push only when $i < n$ (Topic 48)
"sum over all subarrays of the min/max"	Contribution (Topic 53)
"lexicographically smallest after removing k "	Greedy stack (Topic 54)
"trapped water", "container"	Two pointers first, stack second (Topic 52)
"sliding window maximum"	Not a stack — monotonic <i>deque</i> , see below

Step 2 — answer four questions, in order

1. What is each element WAITING for?
"a taller bar to my right" / "a smaller price before me"
-> this fixes the comparison operator
 2. Is the answer known at POP time or at PUSH time?
next-* -> pop time (the arriving element is the answer)
previous-* -> push time (whatever survives is the answer)
 3. Do I need indices or values?
length / distance / width / span / boundary -> INDICES (almost always)
just the value -> either, but push indices anyway
 4. What about ties?
contribution problems -> one side strict, one non-strict
everything else -> whichever the problem statement says
TEST [2,2] REGARDLESS
- Then fill the skeleton:
- ```
while (!st.empty() && COMPARE) { j = pop; RESOLVE(j, i); } push(i);
...and handle the leftovers.
```

## When it is the wrong tool

**Sliding window maximum** is the standard confusion. It needs elements removed from *both* ends — from the front when they leave the window, from the back when a larger element arrives — so it is a monotonic **deque**, not a stack. That is Topic 6's both-ends disqualifier appearing in its most common concrete form. Likewise, if the problem needs the  $k$ -th largest or an arbitrary order statistic, it is a heap; if it needs the minimum of an arbitrary range, it is a sparse table or segment tree. Monotonic stacks answer *nearest-boundary* questions only.

### THE KEY INSIGHT

Every problem in Part 7 is the same eight-line loop with two decisions filled in: the comparison, and what to compute at the pop. If you can state what each element is waiting for, you have already written the code. That is the whole section in one sentence.

### THE TRAP

Pattern-matching on the word “maximum” and reaching for a monotonic stack when the problem is a sliding window. The stack version gives correct answers for windows that happen to end at array boundaries and wrong ones elsewhere, which is the worst kind of wrong — it looks nearly right. Check for the both-ends requirement *before* writing anything.

recognition – nearest-boundary questions → monotonic stack,  $O(n)$  · both-ends removal → monotonic deque · order statistics → heap · arbitrary range queries → segment tree

## PRACTICE SET — Topics 44-55

---

### A. Conceptual

1. State what the monotonic stack holds, in the words of Topic 44 — not “elements in sorted order”.
2. Give the one-sentence proof that the algorithm is  $O(n)$  despite the inner `while` loop.
3. Why does the stack become monotonic without anything ever being sorted?
4. What do the elements remaining on the stack at the end represent, and what must every problem do about them?
5. For “next” questions the answer is written at pop time; for “previous” questions it is read at push time. Explain why, in terms of what the stack contains at each moment.
6. Give the pop condition for each of the four cells of the direction table.
7. Why should you push indices rather than values even when the immediate problem only needs values?
8. Explain the asymmetric comparison rule for contribution problems and what happens without it.
9. What identity can you assert to detect an asymmetry bug in Topic 53 on the first run?
10. Why do exactly two passes suffice for circular next-greater? What does the set of leftover elements consist of?
11. What does the `if (i < n)` guard prevent in the circular version?
12. In the histogram, after popping index  $j$ , what do `st.top()` and `i` represent, and why does the width formula subtract 1?
13. What does the appended sentinel `0` replace in the histogram algorithm? Name three earlier sentinels in this book with the same purpose.
14. Why must the heights array in maximal-rectangle be reset to 0 on a zero cell rather than decremented?
15. For trapping rain water, which solution would you ship and why? State the comparison in both dimensions.
16. Explain the contribution technique in one sentence, and name two other problems it solves unchanged.
17. Why is removing a digit larger than its successor always an improvement?
18. Name the three post-loop cases in `removeKdigits` and an input that breaks each.
19. Sliding window maximum is not a monotonic stack problem. Which disqualifier from Topic 6 applies, and what is the correct structure?

### B. Tracing and analysis

1. Run next-greater-to-the-right on `[4, 5, 2, 25]` and on `[13, 7, 6, 12]`. Give the stack after each index and the final answer array.
2. Run previous-smaller-to-the-left on `[4, 5, 2, 10, 8]`. Give the stack and the answer at each step.
3. Run daily temperatures on `[30, 40, 50, 60]` and on `[30, 60, 90]` and on `[90, 60, 30]`. Give all three answer arrays and state the maximum stack depth reached in each.
4. Run circular next-greater on `[3, 8, 4, 1, 2]`. Show all ten iterations with the stack, and identify which index is left unresolved and why.
5. Run stock span on `[31, 27, 14, 21, 30, 32]`. Give the stack and the span at each day.

6. Run largest-rectangle on `[2, 4]`, `[2, 1, 2]` and `[6, 2, 5, 4, 5, 1, 6]`. Give the area at each pop and the final answer for all three.
7. Build the heights array row by row for the matrix `[[1,1],[1,1],[0,1]]` and give the maximum area after each row.
8. Compute trapped water for `[3, 0, 1]` and `[4,2,0,3,2,5]` by hand using the two-pointer method, showing `leftMax`, `rightMax` and the accumulated total at each step.
9. Compute sum-of-subarray-minimums for `[3, 1, 2, 4]` and for `[2, 2]` using contribution. For each, list L, R, and the count per index, and verify the counts sum to  $n(n+1)/2$ .
10. Run `removeKdigits("10200", 1)`, `("112", 1)`, `("10", 2)` and `("1432219", 3)`. Give the stack before the post-processing and the final output for each.
11. For each problem in Part 7, state the input shape that maximises stack depth and give that depth for  $n = 1000$ .

### C. Code tasks

1. **Build.** Implement the universal skeleton from Topic 44 as a template taking a comparator and a resolve lambda. Instantiate it for next-greater, next-smaller, previous-greater and previous-smaller, and verify all four against brute-force  $O(n^2)$  implementations on 10,000 random arrays of length 50.
2. **Break it — the leftovers.** Remove the default initialisation of the answer array in `nextGreater` (leave it uninitialised). Run it on `[5,4,3,2,1]` under `-fsanitize=memory` or with `valgrind`. Report what is flagged and which array positions hold garbage. Explain why an increasing input would have hidden this.
3. **Break it — duplicates.** Implement `sumSubarrayMins` with symmetric strict comparisons on both boundaries. Run it on `[2,2]`, `[3,3,3]` and `[1,2,2,1]`. Report each wrong answer, the correct answer, and the value of the “counts sum to  $n(n+1)/2$ ” assertion in each case. Then fix the asymmetry and confirm all three.
4. **Break it — the histogram width.** Change `width = i - st.top() - 1` to `i - st.top()`. Run `[5]`, `[2,1,2]` and `[2,1,5,6,2,3]`. Report which inputs still give the right answer and explain why the single-bar case cannot detect the bug.
5. **Break it — the missing left wall.** Delete `if (st.empty()) break;` from `trapStack`. Run `[3,0,1]` and `[5,4,3,2,1]` under `-fsanitize=address`. Report the water returned and whether ASan flags it under the default `std::deque` backing. Then re-run with `std::stack<int, std::vector<int>>` and compare.
6. **Break it — the circular push guard.** Remove `if (i < n)` from `nextGreaterCircular`. Run `[5,4,3,2,1]` and `[1,2,3,4,5]`. Report both outputs, the final stack size, and which one is wrong.
7. **Fix and verify.** Restore every defect above. Build a fuzz harness that generates 100,000 random arrays with values drawn from `{1,2,3}` — guaranteeing heavy duplication — and compares every Part 7 algorithm against a brute-force reference. Confirm zero mismatches.
8. **Measure.** Time `largestRectangleArea` against an  $O(n^2)$  brute force for  $n = 1,000, 10,000$  and  $100,000$  on random input, and again on strictly increasing input. Report all twelve figures and the crossover point. Then report the peak stack depth for both input shapes and explain the difference.

## ANSWER KEY — Topics 44-55

---

### A. Conceptual

1. It holds **the elements whose answer is not yet known**, and the element that resolves them is the one that pops them.
2. **Every index is pushed exactly once and popped at most once, so the total number of pops across the whole run is at most  $n$ .** A long inner loop on one iteration is paid for by short ones elsewhere.
3. Because anything that would violate the order gets popped first: if  $x$  arrives while  $y < x$  is on top,  $y$  is resolved by  $x$  and removed. Monotonicity is a **consequence** of always resolving from the top, not a maintained property.
4. They are the elements with **no answer** — no greater element to the right, no shorter bar, no warmer day. Every problem must supply an explicit default:  $-1$ ,  $0$ , or the array end as a boundary.
5. For “next”, the arriving element is the answer, and the only moment you know which elements it answers is when you pop them. For “previous”, the answer is whatever has survived on the stack — which is only correct *after* the pops and *before* the current index is pushed.
6. Next greater:  $a[\text{top}] < a[i]$ . Next smaller:  $a[\text{top}] > a[i]$ . Previous greater:  $a[\text{top}] \leq a[i]$ . Previous smaller:  $a[\text{top}] \geq a[i]$ .
7. Because **distances, widths, spans and boundaries all need positions**, and converting a value-based loop to an index-based one later means rewriting it. The value is always available as  $a[\text{st.top()}]$ .
8. Make one boundary strict and the other non-strict — conventionally “previous strictly smaller, next smaller or equal.” Without it, **equal values both claim the same subarrays** and those ranges are counted more than once.
9. **The per-element subarray counts must sum to exactly  $n(n+1)/2$** , the total number of subarrays.
10. Because any element with no greater value in a full wrap-around **has none at all** — it is a global maximum. A third pass could find nothing new. The leftovers are exactly the global maxima.
11. It prevents **duplicate indices being pushed during the second pass**, which would leave stale entries on the stack and let `ans` be overwritten from the wrong pass.
12.  $\text{st.top()}$  is the nearest index to the left with a strictly smaller height;  $i$  is the nearest to the right that is smaller. Both are **exclusive** boundaries, so the qualifying bars run from  $\text{st.top()}+1$  to  $i-1$  inclusive — a count of  $i - \text{st.top()} - 1$ .
13. It replaces the **duplicated drain loop** after the main loop. Earlier sentinels with the same purpose: the  $-1$  index in Topic 39, the base accumulator  $\text{st.push}(0)$  in Topic 40, and the empty-stack  $i + 1$  case in Topic 49.
14. Because **a rectangle cannot span a gap**. A zero cell means no all-ones column continues through that row, so the accumulated run above it is worthless and the height must restart from nothing.
15. **Two pointers**. Time is  $O(n)$  for both; space is  **$O(1)$  versus  $O(n)$** ; and there is no stack discipline to get wrong. This is one of two places in Part 7 where the stack is not the best tool.
16. Instead of asking what each subarray's minimum is, ask **how many subarrays each element is the minimum of**, then multiply by its value. It solves **sum of subarray maximums** and **sum of subarray ranges** (run it twice and subtract) unchanged.

17. Because the removed digit sits in a **more significant position** than its successor, so replacing it with a smaller digit lowers the number by more than any later change could.
18. (i) **k not exhausted** — "112", k=1, digits are non-decreasing so nothing ever pops; cut the tail. (ii) **leading zeros** — "10200", k=1 leaves "0200". (iii) **empty result** — "10", k=2 leaves "", which must become "0".
19. The **both-ends** disqualifier: elements leave from the front when they exit the window and from the back when a larger one arrives. The correct structure is a **monotonic deque**.

## B. Tracing and analysis

1. [4,5,2,25] → [5, 25, 25, -1]. [13,7,6,12] → [-1, 12, 12, -1].
2. [4,5,2,10,8] → [-1, 4, -1, 2, 2]. (At i=2 the value 2 pops both 4 and 5, leaving the stack empty.)
3. [30,40,50,60] → [1,1,1,0], max depth 1. [30,60,90] → [1,1,0], max depth 1. [90,60,30] → [0,0,0], max depth 3 — a strictly decreasing input is the worst case.
4. [3,8,4,1,2] → [8, -1, 8, 2, 3]. Index 1 (value 8) is unresolved because it is the global maximum. The wrap resolves index 4 (value 2) with the 3 at index 0, and index 3 (value 1) with the 2 at index 4.
5. [31,27,14,21,30,32] → [1, 1, 1, 2, 4, 6].
6. [2,4]: pops give areas 4 then 4 (width 2 × height 2) → 4. [2,1,2]: areas 2, then at the sentinel 2 and 3 → 3. [6,2,5,4,5,1,6]: the best is height 4 over bars 2-4, width 3 → 12.
7. Row 0 [1,1] → area 2. Row 1 [2,2] → area 4. Row 2 [0,3] → area 3. Maximum overall 4.
8. [3,0,1] → 1 (one unit above index 1, bounded by 3 on the left and 1 on the right — min is 1, so 1-0 = 1). [4,2,0,3,2,5] → 9.
9. [3,1,2,4]: j=0 L=-1 R=1 count 1; j=1 L=-1 R=4 count 6; j=2 L=1 R=4 count 2; j=3 L=2 R=4 count 1. Counts sum 10 = 4·5/2. Total 17. [2,2]: j=0 L=-1 R=1 count 1; j=1 L=-1 R=2 count 2. Counts sum 3 = 2·3/2. Total 6.
10. ("10200",1): stack "0200" → "200". ("112",1): stack "112", tail cut → "11". ("10",2): stack "" → "0". ("1432219",3): stack "1219" → "1219".
11. Next-greater and daily temperatures: **strictly decreasing** input, depth 1000. Previous-smaller and stock span: **strictly increasing**, depth 1000. Histogram: **strictly increasing**, depth 1000 just before the sentinel. Trapping (stack version): **strictly decreasing**, depth 1000. In every case the worst input is the one where nothing ever resolves, so the whole array accumulates.

## C. Code tasks — what to verify

1. All four must agree with brute force on all 10,000 arrays. If only next-greater agrees, check that your previous-\* instantiations read `st.top()` before the push rather than inside the pop loop.
2. Positions 0 through 4 — every position — hold garbage for [5,4,3,2,1], because a strictly decreasing input resolves nothing and the whole array is left at its default. Valgrind reports **Conditional jump or move depends on uninitialised value(s)** when the results are used. A strictly *increasing* input would have hidden it completely, because every element except the last gets resolved.
3. [2,2]: buggy 8, correct 6, assertion sees counts summing to 4 instead of 3. [3,3,3]: buggy 27, correct 18, counts 9 instead of 6. [1,2,2,1]: buggy 18, correct 17, counts 11 instead of 10. The assertion fires before any answer is compared, which is the point.

4. `[5]` still gives **5** — it takes the `st.empty()` branch, which the bug does not touch, so a single-bar test cannot detect it. `[2,1,2]` gives **4** instead of 3, and `[2,1,5,6,2,3]` gives **12** instead of 10. Always test both branches of a two-branch formula.
5. `[3,0,1]` returns a **large garbage value** instead of **1**; `[5,4,3,2,1]` returns garbage instead of **0**. Under the default `std::deque` backing ASan often reports **nothing**, because the read lands inside an allocated deque block. With `std::vector` backing you get **heap-buffer-over-flow READ**. Same bug, visible only under one backing — the Topic 16 lesson recurring inside an algorithm.
6. `[5,4,3,2,1]`: the correct output is **[-1,5,5,5,5]**; without the guard, indices are pushed twice and the final stack has **10 entries** instead of 1, with several `ans` values overwritten. `[1,2,3,4,5]` still gives the correct **[2,3,4,5,-1]**, because the increasing input pops everything during the first pass and the duplicates never matter. The increasing case is what makes the bug survive review.
7. Zero mismatches. Drawing values from `{1,2,3}` is the key detail: random 32-bit integers almost never collide, so a naive fuzzer exercises none of the tie-breaking logic. Small alphabets are how you fuzz comparison-sensitive code.
8. Expect the stack version at roughly **3-10 ns per element** at all three sizes, and brute force at  $n^2/2$  comparisons: about **0.3 ms** at  $n=1000$ , **30 ms** at  $n=10,000$  and **3 s** at  $n=100,000$ . The crossover is well below  $n = 1000$  — often around  $n = 100$  — so there is no regime where brute force wins. Peak stack depth is **n** on increasing input and typically **1-3** on random input; the  $O(n)$  space bound is real but almost never reached in practice.



# HANDNOTE SUMMARY CARD

## PART 7 - THE MONOTONIC STACK

Topics 44-55

WHAT IT HOLDS: the elements whose answer is NOT YET KNOWN.

the element that resolves them is the one that pops them.

monotonicity is a CONSEQUENCE of always resolving from the top, not a goal.

$O(n)$  PROOF: every index is pushed once and popped at most once. Total pops  $\leq n$ .

LEFTOVERS on the stack = the elements with NO answer. Always give them a default.

### THE UNIVERSAL SKELETON

```
for (i = 0; i < n; ++i) {
 while (!st.empty() && COMPARE(a[st.top()], a[i])) {
 j = st.top(); st.pop();
 RESOLVE(j, i); // a[i] = right bound; st.top() = left bound
 }
 st.push(i); // ALWAYS INDICES
}
// handle whatever is left on the stack
```

### FOUR-WAY TABLE (all scan left -> right)

|                  |                                     |              |                  |
|------------------|-------------------------------------|--------------|------------------|
| next greater     | pop while $a[\text{top}] < a[i]$    | write at POP | stack decreasing |
| next smaller     | pop while $a[\text{top}] > a[i]$    | write at POP | stack increasing |
| previous greater | pop while $a[\text{top}] \leq a[i]$ | read at PUSH | stack decreasing |
| previous smaller | pop while $a[\text{top}] \geq a[i]$ | read at PUSH | stack increasing |

TIES: strict vs non-strict decides OWNERSHIP.

contribution problems -> ONE side strict, the other non-strict

("previous strictly smaller, next smaller-or-equal")

symmetric -> [2,2] counted twice -> 8 instead of 6

TEST [2,2], [3,3,3], [1,2,2,1] ON EVERY PROBLEM IN THIS PART.

### THE PROBLEMS - all the same loop, different RESOLVE

|                    |                                                                                                                               |                     |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------|---------------------|
| next greater       | $\text{ans}[j] = a[i]$                                                                                                        | default -1          |
| daily temperatures | $\text{ans}[j] = i - j$                                                                                                       | default 0 <- not -1 |
| stock span         | $\text{span}[i] = i - \text{st.top()}$ (or $i+1$ ) read before push, use $\leq$                                               |                     |
| circular NGE       | $\text{scan } 2n, \text{idx} = i \% n, \text{PUSH ONLY IF } i < n$                                                            |                     |
| histogram          | $w = \text{st.empty()} ? i : i - \text{st.top()} - 1; \text{area} = h[j]*w$                                                   |                     |
| maximal rectangle  | append sentinel 0 to flush; the -1 is where people slip<br>$h[c] = \text{row}[c] == '1' ? h[c]+1 : 0;$ then histogram per row |                     |
| trapping water     | RESET on 0, never decrement. $O(\text{rows}*\text{cols})$ .<br>stack $O(n)$ space   TWO POINTERS $O(1)$ space <- SHIP THIS    |                     |
| subarray minimums  | stack version needs "if (st.empty()) break;"<br>contribution: $(j-L)*(R-j)*a[j]$                                              |                     |
| remove k digits    | ASSERT counts sum to $n(n+1)/2$ <- catches ties instantly<br>pop while $k>0$ && $\text{st.back()} > c$                        |                     |
|                    | 3 post-loop cases: cut tail, strip zeros, "" -> "0"                                                                           |                     |

### RECOGNITION - four questions, fifteen seconds

1. what is each element WAITING for? -> the comparison
2. answer known at POP or at PUSH? -> next vs previous
3. length/distance/width/span/boundary? -> indices (always anyway)
4. ties? -> one strict, one not

### PHRASE -> TECHNIQUE

|                                                          |                                               |                      |                 |
|----------------------------------------------------------|-----------------------------------------------|----------------------|-----------------|
| "next greater/smaller"                                   | -> T45                                        | "days until"         | -> T47, i-j     |
| "span", "consecutive before"                             | -> T49                                        | "largest rectangle"  | -> T50          |
| "circular", "wraps"                                      | -> T48                                        | "sum over subarrays" | -> T53          |
| "smallest after removing k"                              | -> T54                                        | "trapped water"      | -> two pointers |
| "sliding window maximum"                                 | -> NOT A STACK. monotonic DEQUE (both ends).  |                      |                 |
| k-th largest                                             | -> heap. arbitrary range min -> segment tree. |                      |                 |
| monotonic stacks answer NEAREST-BOUNDARY questions only. |                                               |                      |                 |



PART EIGHT

# Augmented designs

---

Making a stack answer questions its interface does not support: the minimum in  $O(1)$ , FIFO from two LIFOs with the amortisation proof, and undo/redo done properly — including the bounded-history case a plain stack cannot handle.

## 56. Min stack: pairs and the auxiliary stack

Design a stack that also returns its minimum in  $O(1)$ . The obstacle is that `pop` can remove the minimum, so you must know what the minimum *was* before it was pushed. Both good solutions store history; they differ in how much.

### Version 1: pair each value with the minimum at its time

```
class MinStackPairs {
 std::stack<std::pair<int,int>> st; // (value, min at or below this level)
public:
 void push(int v) {
 int m = st.empty() ? v : std::min(v, st.top().second);
 st.push({v, m});
 }
 void pop() { st.pop(); }
 int top() const { return st.top().first; }
 int getMin() const { return st.top().second; } // always correct, $O(1)$
};
```

Every entry carries the minimum of everything at or below it, so popping automatically restores the previous minimum with no work. Cost: **8 bytes per element instead of 4** — exactly  $2\times$ .

### Version 2: a second stack that only records new minima

```
class MinStackAux {
 std::stack<int> st;
 std::stack<int> mins; // push only when v <= current min
public:
 void push(int v) {
 st.push(v);
 if (mins.empty() || v <= mins.top()) mins.push(v); // NOTE: <=
 }
 void pop() {
 if (st.top() == mins.top()) mins.pop(); // pop in lockstep
 st.pop();
 }
 int top() const { return st.top(); }
 int getMin() const { return mins.top(); }
};
```

```

push 5, 3, 7, 3, 8 then pop twice

main: [5] [5,3] [5,3,7] [5,3,7,3] [5,3,7,3,8]
mins: [5] [5,3] [5,3] [5,3,3] [5,3,3]
 ^ pushed because 3 <= 3 -- CRITICAL

pop (8): st.top()=8 != mins.top()=3 -> mins untouched
 main [5,3,7,3] mins [5,3,3] getMin = 3 correct
pop (3): st.top()=3 == mins.top()=3 -> pop both
 main [5,3,7] mins [5,3] getMin = 3 correct

If push had used strict "<", the second 3 would NOT be in mins.
The first pop of a 3 would then remove the ONLY 3 from mins, and
getMin would report 5 while a 3 is still in the stack.

memory: pairs always 2n (8 bytes per int)
 aux n + m, where m = number of non-increasing prefix records
 worst case (strictly decreasing input) 2n -- identical
 best case (increasing input) n + 1

```

#### THE KEY INSIGHT

You cannot recompute the minimum on **pop** — that would be  $O(n)$  — so you must have recorded it on **push**. Every “augment a stack with an  $O(1)$  query” problem is solved the same way: **store the answer alongside each element at push time**. It works for min, max, sum, GCD, and any other associative fold.

#### THE TRAP

`if (v < mins.top())` instead of `<=`. With duplicate minima, the auxiliary stack records the value once but the main stack holds it twice; the first pop removes it from `mins`, and `getMin()` then reports a value larger than an element still present. Symptom: correct on all-distinct inputs, wrong only after popping a duplicated minimum — Topic 46's trap in a different setting. Test `push 3, push 3, pop, getMin`; it must return 3.

min stack – push/pop/top/getMin all  $O(1)$  worst case · pairs: exactly  $2n$  space · auxiliary:  $n+1$  best,  $2n$  worst · must use `<=` when pushing minima

## 57. Min stack: the encoding trick

The interview follow-up is “can you do it in  $O(1)$  extra space?” The answer is yes, by storing an encoded value when a new minimum arrives, from which the previous minimum can be recovered. It is clever, it is genuinely asked, and it is a bad idea in production — all three facts are worth knowing.

```

class MinStackEncoded {
 std::stack<long long> st; // long long: see the overflow note
 long long mn = 0; // current minimum (valid if !empty)
public:
 void push(long long v) {
 if (st.empty()) { st.push(v); mn = v; return; }
 if (v < mn) {
 st.push(2 * v - mn); // ENCODED: always < v, flags "new min"
 mn = v;
 } else {
 st.push(v);
 }
 }
 void pop() {
 long long t = st.top(); st.pop();
 if (t < mn) mn = 2 * mn - t; // recover the PREVIOUS minimum
 }
 long long top() const {
 long long t = st.top();
 return t < mn ? mn : t; // encoded entries decode to mn
 }
 long long getMin() const { return mn; }
};

```

Why the encoding is recoverable and self-identifying

On a new minimum  $v$  with previous minimum  $m$  ( $v < m$ ), store  $e = 2v - m$ .

Since  $v < m$ :  $e = 2v - m = v - (m - v) < v < m$  = the new mn.

So  $e < mn$  always, and ONLY encoded entries satisfy that -- a real stored value is never below the current minimum, by definition.

To recover on pop, with  $mn == v$ :  $2*mn - e = 2v - (2v - m) = m$ . QED.

Trace: push 5, 3, 7, 2

|        |                               |              |          |
|--------|-------------------------------|--------------|----------|
| push 5 | st=[5]                        | mn=5         |          |
| push 3 | 3<5 -> e=2*3-5=1              | st=[5,1]     | mn=3     |
| push 7 | 7>=3 -> plain                 | st=[5,1,7]   | mn=3     |
| push 2 | 2<3 -> e=2*2-3=1              | st=[5,1,7,1] | mn=2     |
| top()  | 1 < mn(2) -> return mn = 2    |              | correct  |
| pop()  | t=1 < mn(2) -> mn = 2*2-1 = 3 |              | restored |
| pop()  | t=7, 7 >= 3, mn unchanged     |              | still 3  |
| pop()  | t=1 < mn(3) -> mn = 2*3-1 = 5 |              | restored |

#### THE KEY INSIGHT

The encoded value doubles as its own flag: because  $2v - m < v < m$ , any stored value below the current minimum *must* be encoded, and a genuine value never can be. No extra bit is needed. That self-identification is the actual cleverness, more than the arithmetic.

#### THE TRAP

$2 * v - mn$  overflows. With 32-bit ints,  $v = -2,000,000,000$  and  $mn = 2,000,000,000$  gives  $-6 \times 10^9$ , far outside `int` range — signed overflow is **undefined behaviour**, so the compiler may do anything. It is silent on the small values every test uses. Use `long long`, and say so when you present the trick; the interviewer is often waiting for exactly that observation. In production, prefer Topic 56 — 4 bytes per element is not worth undefined behaviour and an unreadable `pop`.

encoded min stack – all operations  $O(1)$  ·  **$O(1)$  extra space** beyond the values · requires  $2\times$  the integer width to avoid UB · correct but unmaintainable – know it, do not ship it

## 58. Queue from two stacks

Implement FIFO using only LIFO operations. Two stacks: everything enters the *in* stack, everything leaves the *out* stack, and the transfer from one to the other reverses the order — which is exactly the conversion from LIFO to FIFO.

```
class QueueFromStacks {
 std::stack<int> in_, out_;
 void shift() {
 // ONLY when out_ is empty
 while (!in_.empty()) { out_.push(in_.top()); in_.pop(); }
 }
public:
 void push(int v) { in_.push(v); } // $O(1)$ always
 void pop() {
 if (out_.empty()) shift(); // amortised $O(1)$
 out_.pop();
 }
 int front() {
 if (out_.empty()) shift();
 return out_.top();
 }
 bool empty() const { return in_.empty() && out_.empty(); }
};
```

push 1,2,3 then pop, push 4, pop, pop, pop

| op     | in_ (bot->top) | out_ (bot->top) | returns |                    |
|--------|----------------|-----------------|---------|--------------------|
| push 1 | [1]            | []              |         |                    |
| push 2 | [1,2]          | []              |         |                    |
| push 3 | [1,2,3]        | []              |         |                    |
| pop    | [1]            | [3,2]           | 1       | <- shift reverses  |
| push 4 | [4]            | [3,2]           |         |                    |
| pop    | [4]            | [3]             | 2       | <- no shift needed |
| pop    | [4]            | []              | 3       |                    |
| pop    | []             | [4]             | 4       | <- shift again     |

Each element is moved exactly ONCE from *in\_* to *out\_*, ever.

### The amortisation proof

Each element is pushed to *in\_* once, popped from *in\_* once, pushed to *out\_* once, and popped from *out\_* once — **four stack operations over its entire lifetime, regardless of how the calls are interleaved**. So  $m$  queue operations cost at most  $4m$  stack operations: amortised  $O(1)$ . The accounting method from Topic 9 applies directly — charge 4 credits per *push* and the transfer is prepaid.

The worst case for a single operation is still  $O(n)$ : pushing a million elements then calling *pop* once transfers all million. This is the deque disqualifier from Topic 6 — if you care about p99 latency rather than throughput, use a real deque.

#### THE KEY INSIGHT

The `if (out_.empty())` guard is what makes it amortised  $O(1)$ . Shifting whenever `in_` is non-empty instead would move elements back and forth repeatedly and give  $O(n)$  per operation — the same thrash-versus-hysteresis distinction as the shrink policy in Topic 10. Transfer only when you have no choice.

#### THE TRAP

Calling `shift()` unconditionally in `pop`, or shifting back from `out_` to `in_` on `push`. Both are correct and both destroy the complexity: alternating push and pop then moves the entire contents on every operation, giving  $O(n)$  per call and  $O(n^2)$  overall. Symptom: a queue that passes every correctness test and times out on the large input. Add an operation counter and assert the total transfers never exceed the total pushes.

queue from two stacks – push  $O(1)$  worst case, pop/front  $O(1)$  amortised,  $O(n)$  worst case · Space  $O(n)$  · exactly 4 stack ops per element for life · shift only when `out_` is empty

## 59. Stack from two queues

The reverse construction, and it is genuinely worse — which is the interesting part. There is no amortised trick here: one of the two operations must be  $O(n)$ , and you choose which.

### Costly push, $O(1)$ pop

```
class StackFromQueues {
 std::queue<int> q_;
public:
 void push(int v) { // $O(n)$
 q_.push(v);
 for (std::size_t i = 1; i < q_.size(); ++i) { // rotate the new element
 q_.push(q_.front()); q_.pop(); // to the front
 }
 }
 void pop() { q_.pop(); } // $O(1)$
 int top() { return q_.front(); } // $O(1)$
};
```

### $O(1)$ push, costly pop

```
class StackFromQueues2 {
 std::queue<int> a_, b_;
public:
 void push(int v) { a_.push(v); } // $O(1)$
 void pop() { // $O(n)$
 while (a_.size() > 1) { b_.push(a_.front()); a_.pop(); }
 a_.pop(); // the last one is the "top"
 std::swap(a_, b_);
 }
};
```



| Design                            | push | pop            | Amortised O(1) possible? |
|-----------------------------------|------|----------------|--------------------------|
| Queue from 2 stacks               | O(1) | O(1) amortised | <b>Yes</b>               |
| Stack from 2 queues (costly push) | O(n) | O(1)           | <b>No</b>                |
| Stack from 2 queues (costly pop)  | O(1) | O(n)           | <b>No</b>                |

#### THE KEY INSIGHT

The asymmetry is real and worth being able to explain. Two stacks give an amortised-O(1) queue because each element moves *once, permanently*. Two queues cannot give an amortised-O(1) stack, because reaching the most recent element requires cycling past all the older ones *every single time* — there is no state you can leave behind that makes the next operation cheaper. When an interviewer asks for both, saying this out loud is the answer they want.

#### THE TRAP

`for (i = 1; i < q_.size(); ++i)` where `q_.size()` is re-evaluated each iteration and the loop body pushes and pops — keeping the size constant, so the loop runs the intended number of times only by accident of the push and pop cancelling. Rewrite it as `int k = q_.size() - 1;` before the loop. Symptom on a naive variant that changes the size: an infinite loop, or a rotation by the wrong amount that silently produces FIFO order.

stack from two queues – O(n) on one operation, unavoidably · Space O(n) · no amortisation exists – the asymmetry with Topic 58 is the lesson

## 60. Undo and redo

The application everyone names when asked where stacks are used, and almost everyone gets one detail wrong. Two stacks: **undo** and **redo**. Undoing moves an action from one to the other. The detail is what a *new* action does to the redo stack.

```

class History {
 std::vector<Action> undo_, redo_; // vector: we need to trim the front
 std::size_t limit_;
public:
 explicit History(std::size_t limit = 100) : limit_(limit) {}

 void perform(const Action& a) {
 a.apply();
 undo_.push_back(a);
 redo_.clear(); // THE CRITICAL LINE
 if (undo_.size() > limit_)
 undo_.erase(undo_.begin()); // drop the OLDEST -- needs a vector
 }
 bool undo() {
 if (undo_.empty()) return false;
 Action a = undo_.back(); undo_.pop_back();
 a.revert();
 redo_.push_back(a);
 return true;
 }
 bool redo() {
 if (redo_.empty()) return false;
 Action a = redo_.back(); redo_.pop_back();
 a.apply();
 undo_.push_back(a);
 return true;
 }
};

```

Type A, type B, undo, type C      -- the redo branch is discarded

| action | undo stack | redo stack | state                      |
|--------|------------|------------|----------------------------|
| -----  | -----      | -----      | -----                      |
| type A | [A]        | []         | "A"                        |
| type B | [A,B]      | []         | "AB"                       |
| undo   | [A]        | [B]        | "A"                        |
| type C | [A,C]      | []         | "AC"      <- redo_ CLEARED |

^

B is gone forever. Redoing it would produce "ABC" or "ACB",  
neither of which is a state the user was ever in.

This is why every editor greys out Redo the moment you type after undoing.

#### THE KEY INSIGHT

Clearing the redo stack on a new action is a *correctness* requirement, not a UI nicety. History is a path, not a tree; once you branch, the abandoned branch's actions no longer compose with the current state. Keeping them lets the user redo an action whose preconditions no longer hold — which in a document editor means applying an edit at an offset that no longer exists.

### THE TRAP

Using `std::stack` and therefore being unable to enforce a bounded history, because the interface cannot remove the oldest entry (Topic 6). An unbounded undo stack in a program that snapshots state per action grows without limit: 10,000 edits with 50 KB snapshots is **500 MB**, and the process is OOM-killed with no stack trace anywhere near the undo code. Use a `std::vector` or `std::deque` and trim the front; the `erase(begin())` above is  $O(n)$  per trim, so use a `std::deque` and `pop_front()` if the limit is large.

undo/redo – all operations  $O(1)$  except bounded trimming · Space  $O(\text{limit} \times \text{action size})$  · clear redo on every new action · `std::stack` cannot bound history – use a deque

## 61. Sorting a stack with a stack

Sort a stack using only stack operations and one auxiliary stack — no arrays, no recursion into the elements. It is an artificial constraint, but it is a common interview exercise and it makes the insertion-sort structure unusually visible.

```
void sortStack(std::stack<int>& s) {
 std::stack<int> aux; // kept sorted, largest on top
 while (!s.empty()) {
 int cur = s.top(); s.pop();
 // move back anything that must sit above cur in aux
 while (!aux.empty() && aux.top() > cur) {
 s.push(aux.top()); aux.pop();
 }
 aux.push(cur); // cur lands in sorted position
 }
 s = std::move(aux); // s is now sorted, largest on top
}
```

`s = [3, 1, 4, 2]` (bottom -> top, so 2 is on top)

| step | s       | aux       | action                                    |
|------|---------|-----------|-------------------------------------------|
| 1    | [3,1,4] | []        | cur=2, aux empty -> push                  |
| 2    | [3,1,4] | [2]       |                                           |
| 3    | [3,1]   | [2]       | cur=4, aux.top=2 not > 4 -> push          |
| 4    | [3,1]   | [2,4]     |                                           |
| 5    | [3]     | [2,4]     | cur=1, 4>1 -> move back; 2>1 -> move back |
| 6    | [3,4,2] | []        | push 1                                    |
| 7    | [3,4,2] | [1]       |                                           |
| 8    | [3,4]   | [1]       | cur=2, aux.top=1 not > 2 -> push          |
| 9    | [3,4]   | [1,2]     |                                           |
| 10   | [3]     | [1,2]     | cur=4, 2 not > 4 -> push                  |
| 11   | [3]     | [1,2,4]   |                                           |
| 12   | []      | [1,2,4]   | cur=3, 4>3 -> move back                   |
| 13   | [4]     | [1,2]     | push 3                                    |
| 14   | [4]     | [1,2,3]   |                                           |
| 15   | []      | [1,2,3]   | cur=4 -> push                             |
| 16   | []      | [1,2,3,4] | done                                      |

`aux = [1,2,3,4]` bottom->top: sorted, largest on top.  
This is INSERTION SORT: each element is placed by shifting larger elements out of the way -- hence  $O(n^2)$  in the worst case.

#### THE KEY INSIGHT

This is insertion sort with the “shift elements right” step implemented as “move them back to the source stack.” Naming the algorithm is worth more than producing the code, because it immediately gives you the complexity ( $O(n^2)$  comparisons and moves,  $O(n)$  space) and the honest conclusion: if you are allowed an array, use `std::sort` and finish in  $O(n \log n)$ .

#### THE TRAP

Believing this is a reasonable way to sort. On 100,000 elements it performs roughly  **$5 \times 10^9$  moves** against `std::sort`'s  $\sim 1.7 \times 10^6$  comparisons — about **3000×** the work, and it touches two containers instead of one contiguous buffer. Say clearly in an interview that this exists only to satisfy the constraint. A book that presented it as a technique would be teaching you to write bad code.

sorting a stack – Time  $O(n^2)$  comparisons and moves · Space  $O(n)$  auxiliary · it is insertion sort  
· `std::sort` on a vector is  $O(n \log n)$  and  $\sim 3000\times$  faster at  $n = 10^5$

## PRACTICE SET — Topics 56-61

---

### A. Conceptual

1. Why can the minimum not simply be recomputed on `pop`? State the general principle this exemplifies for augmenting a stack with an  $O(1)$  query.
2. Give the exact space cost of the pairs min stack and of the auxiliary min stack, in the best and worst cases.
3. Why must the auxiliary min stack push on `<=` rather than `<`? Give the four-operation sequence that exposes the bug.
4. Which other  $O(1)$  queries can be added to a stack by the same technique, and what property must the operation have?
5. In the encoded min stack, why is any stored value below the current minimum necessarily an encoded one?
6. Derive the recovery formula `mn = 2*mn - t` from the encoding `e = 2v - m`.
7. Why does the encoded min stack require a wider integer type than the values it stores? Give a concrete pair of 32-bit values that overflows.
8. Would you ship the encoded min stack? Justify in one sentence.
9. In the two-stack queue, how many stack operations does each element undergo over its entire lifetime, and why is that number independent of the call interleaving?
10. What is the worst case for a single `pop` on the two-stack queue, and when does that make it the wrong structure?
11. Why is there no amortised- $O(1)$  stack built from two queues? Answer in terms of what state can be left behind between operations.
12. What exactly does `redo_.clear()` prevent? Describe a concrete corrupt state that omitting it would allow.
13. Why can a bounded undo history not be built on `std::stack`? Which Topic 6 disqualifier is this?
14. Name the classical sorting algorithm that `sortStack` implements, and say which step of it corresponds to moving elements back to the source stack.
15. Give the honest recommendation for sorting a stack's contents in production code.

### B. Tracing and analysis

1. Trace `MinStackAux` for `push 5, 3, 7, 3, 8, pop, pop, pop`. Give both stacks and `getMin()` after every operation.
2. Repeat with the buggy strict `<` comparison. Identify the first operation at which `getMin()` is wrong and give both the reported and correct values.
3. Trace `MinStackEncoded` for `push 5, 3, 7, 2` then three pops. Give the raw stack contents, `mn`, and the value `top()` returns at each step.
4. For a stack of 1,000,000 32-bit ints, compute the total bytes used by the pairs version, the auxiliary version on strictly increasing input, the auxiliary version on strictly decreasing input, and the encoded version.
5. Trace `QueueFromStacks` for `push 1, push 2, push 3, pop, push 4, pop, pop, pop`. Give both stacks after each operation and count the total number of individual stack pushes and pops performed.
6. For  $m$  interleaved queue operations on  $n$  elements, give the exact upper bound on total stack operations and the worst-case cost of a single `pop` when  $n = 1,000,000$ .

7. Trace `StackFromQueues` (costly push) for `push 1, push 2, push 3`. Give the queue contents after each push and count the total enqueue and dequeue operations.
8. Trace the undo/redo history for: `type A, type B, undo, undo, redo, type C, redo`. Give both stacks and the document state after each step, and say which operation returns `false`.
9. An undo history stores 50 KB snapshots. Compute memory at 100, 1000 and 10,000 actions, and give the action count at which a 512 MB container limit is exceeded.
10. Trace `sortStack` on `[3, 1, 4, 2]` (bottom to top) and count the total pushes and pops. Then give the count for the worst-case input of the same length and identify what that input is.

### C. Code tasks

1. **Build.** Implement all three min stacks. Verify all three agree on 100,000 random operation sequences drawn from values in `{1,2,3}` — the small alphabet is deliberate.
2. **Break it — the strict comparison.** Change the auxiliary min stack to push on `<`. Run `push 3, push 3, pop, getMin`. Report the returned value and the correct one. Then run your 100,000-operation fuzz and report the step index of the first divergence. Finally run the same fuzz with values drawn from a large random range and report how many divergences it finds.
3. **Break it — encoded overflow.** Implement `MinStackEncoded` with `int` instead of `long long`. Push `2000000000` then `-2000000000`. Run under `-fsanitize=undefined` and record the exact diagnostic. Then run without the sanitiser and report what `getMin()` returns.
4. **Break it — unconditional shift.** Remove the `if (out_.empty())` guard from the two-stack queue so `shift()` runs on every `pop`. Verify correctness is unaffected, then time 1,000,000 alternating push/pop operations with and without the guard. Report both times and the ratio.
5. **Instrument.** Add a counter of individual stack operations to the correct two-stack queue. Run 1,000,000 mixed operations and assert the total never exceeds 4× the number of pushes. Report the actual ratio.
6. **Break it — the missing redo clear.** Remove `redo_.clear()` from `perform`. Build a tiny text buffer where each action is “insert character c at offset k”. Run `type A, type B, undo, type C, redo` and report the resulting buffer contents. State whether it is a state the user was ever in.
7. **Fix.** Restore the clear, then convert the bounded history from `std::vector` with `erase(begin())` to `std::deque` with `pop_front()`. Time 1,000,000 `perform` calls with a limit of 100 under both implementations and report the ratio.
8. **Measure.** Time `sortStack` against copying into a `std::vector`, calling `std::sort`, and copying back, for `n = 1,000, 10,000` and `100,000` random values. Report all six figures and the ratio at each size. State the `n` at which `sortStack` exceeds one second.

## ANSWER KEY — Topics 56-61

---

### A. Conceptual

1. Because recomputing requires scanning the remaining elements, which is  $O(n)$  and the interface offers no way to inspect them anyway. Principle: **store the answer alongside each element at push time**.
2. Pairs: **exactly  $2n$**  always. Auxiliary:  $n + 1$  on a strictly increasing input (only the first value is ever a minimum),  $2n$  on a strictly decreasing input (every value is a new minimum).
3. Because with a duplicated minimum the main stack holds it twice but `mins` would hold it once, so the first pop removes it from `mins` while an equal value remains. Sequence: **push 3, push 3, pop, getMin** — must return **3**, buggy version returns the previous minimum or crashes on an empty `mins`.
4. Maximum, sum, product, GCD, bitwise AND/OR — any operation that is **associative**, so that the fold of everything below an element can be combined with the new element in  $O(1)$ .
5. Because  $e = 2v - m$  with  $v < m$  gives  $e = v - (m - v) < v$ , and  $v$  becomes the new minimum. A genuine stored value is **never below the current minimum by definition**, so anything that is must be encoded.
6. At pop time  $mn == v$ . Then  $2 \cdot mn - e = 2v - (2v - m) = m$ , the previous minimum.
7. Because  $2v - m$  can reach roughly twice the range of the value type. With 32-bit ints,  $v = -2000000000$  and  $m = 2000000000$  gives  $-6 \times 10^9$ , outside `int`. Signed overflow is **undefined behaviour**.
8. **No**. It saves 4 bytes per element at the cost of undefined behaviour on extreme values and a **pop** that no reviewer can verify by inspection — Topic 56's pairs version is correct, obvious, and fast.
9. **Four**: pushed to `in_`, popped from `in_`, pushed to `out_`, popped from `out_`. It is interleaving-independent because an element moves from `in_` to `out_` **at most once, permanently** — nothing ever moves back.
10.  **$O(n)$** , when a large batch of pushes is followed by the first pop. It is the wrong structure when **p99 latency matters rather than throughput** — the Topic 6 both-ends disqualifier; use a real deque.
11. Because reaching the most recent element requires cycling past all older ones, and after doing so the queue is back in the same shape — **no state is left behind that makes the next operation cheaper**. In the two-stack queue, the reversal performed once is permanently useful.
12. It prevents redoing an action from an **abandoned branch of history**, whose preconditions no longer hold. Concretely: type A, type B, undo (buffer is "A", redo holds B), type C (buffer "AC"), redo B → buffer becomes **"ACB" or "ABC"** — neither is a state the user ever saw.
13. Because `std::stack` **cannot remove its bottom element**. This is the **bounded-history / both-ends** disqualifier.
14. **Insertion sort**. Moving elements back to the source stack corresponds to **shifting larger elements right to open a gap**.
15. **Copy into a `std::vector`, call `std::sort`, copy back** —  $O(n \log n)$  instead of  $O(n^2)$ , on contiguous memory. `sortStack` exists only to satisfy an artificial constraint.

## B. Tracing and analysis

1.

| op     | main        | mins    | getMin |                            |
|--------|-------------|---------|--------|----------------------------|
| push 5 | [5]         | [5]     | 5      |                            |
| push 3 | [5,3]       | [5,3]   | 3      |                            |
| push 7 | [5,3,7]     | [5,3]   | 3      |                            |
| push 3 | [5,3,7,3]   | [5,3,3] | 3      | <- pushed because 3 <= 3   |
| push 8 | [5,3,7,3,8] | [5,3,3] | 3      |                            |
| pop    | [5,3,7,3]   | [5,3,3] | 3      |                            |
| pop    | [5,3,7]     | [5,3]   | 3      | <- both popped in lockstep |
| pop    | [5,3]       | [5,3]   | 3      |                            |

2. With strict **<**, **mins** after the fourth push is **[5,3]** not **[5,3,3]**. The first wrong result is the **second pop** (removing the inner 3): it pops 3 from **mins**, leaving **[5]**, so **getMin()** reports **5** while a **3** is still in the main stack. Correct value: **3**.

3.

| op     | raw stack | mn | top() |                          |
|--------|-----------|----|-------|--------------------------|
| push 5 | [5]       | 5  | 5     |                          |
| push 3 | [5,1]     | 3  | 3     | (1 < mn -> decode to mn) |
| push 7 | [5,1,7]   | 3  | 7     |                          |
| push 2 | [5,1,7,1] | 2  | 2     | (1 < mn -> decode to mn) |
| pop    | [5,1,7]   | 3  |       | mn = 2*2-1 = 3           |
| pop    | [5,1]     | 3  |       | 7 >= mn, mn unchanged    |
| pop    | [5]       | 5  |       | mn = 2*3-1 = 5           |

4. Pairs:  $1,000,000 \times 8 = \mathbf{8,000,000 \text{ B}}$ . Auxiliary, increasing:  $4,000,000 + 4 = \sim \mathbf{4,000,004 \text{ B}}$ . Auxiliary, decreasing: **8,000,000 B**. Encoded:  $1,000,000 \times 8$  (long long) + 8 = **8,000,008 B** — note the encoded version saves nothing here once widened, which is a fact worth knowing before choosing it.

5.

| op     | in_     | out_  | returns | cumulative stack ops                 |
|--------|---------|-------|---------|--------------------------------------|
| push 1 | [1]     | []    |         | 1                                    |
| push 2 | [1,2]   | []    |         | 2                                    |
| push 3 | [1,2,3] | []    |         | 3                                    |
| pop    | []      | [3,2] | 1       | 3 + (3 pops + 3 pushes) + 1 pop = 11 |
| push 4 | [4]     | [3,2] |         | 12                                   |
| pop    | [4]     | [3]   | 2       | 13                                   |
| pop    | [4]     | []    | 3       | 14                                   |
| pop    | []      | [4]   | 4       | 14 + 1 + 1 + 1 = 17                  |

Total **17** individual stack operations for 8 queue operations.

6. At most **4m** stack operations for m queue operations (4 per element, and at most m elements). Worst-case single **pop** at  $n = 1,000,000$ : **2,000,000 stack operations** (a million pops from **in\_** and a million pushes to **out\_**), roughly **2-8 ms**.

7. push 1: queue **[1]**, 1 enqueue, 0 dequeues. push 2: enqueue 2 then rotate  $1 \rightarrow \mathbf{[2,1]}$ , 2 enqueues, 1 dequeue. push 3: enqueue 3 then rotate  $2 \rightarrow \mathbf{[3,2,1]}$ , 3 enqueues, 2 dequeues. Totals: **6 enqueues, 3 dequeues** — quadratic in the number of pushes.

8.

| step   | undo  | redo  | document | note                                |
|--------|-------|-------|----------|-------------------------------------|
| type A | [A]   | []    | A        |                                     |
| type B | [A,B] | []    | AB       |                                     |
| undo   | [A]   | [B]   | A        |                                     |
| undo   | []    | [B,A] | (empty)  |                                     |
| redo   | [A]   | [B]   | A        |                                     |
| type C | [A,C] | []    | AC       | <- redo_ cleared, B and A-redo lost |
| redo   | [A,C] | []    | AC       | <- returns FALSE, redo stack empty  |

9.  $100 \times 50 \text{ KB} = \mathbf{5 \text{ MB}}$ ;  $1000 \rightarrow \mathbf{50 \text{ MB}}$ ;  $10,000 \rightarrow \mathbf{500 \text{ MB}}$ . The 512 MB limit is exceeded at **10,486 actions** — an afternoon of editing.



10. The Topic 61 trace performs **16 pushes and 16 pops** for  $n = 4$ . Worst case for length 4 is a **strictly decreasing** input (largest at the bottom, e.g. `[4,3,2,1]` bottom to top): every element must be moved past every other, giving the full  $n(n-1)/2 = 6$  extra move-backs on top of the  $2n$  baseline.

### C. Code tasks — what to verify

1. All three must agree at every step. The `{1,2,3}` alphabet guarantees repeated minima, which is the only regime where the three implementations can diverge.
2. Returns the **previous minimum instead of 3**, or crashes on an empty `mins` if the duplicate was the only element. The `{1,2,3}` fuzz should diverge within the **first few dozen steps**. With values from a large random range, expect **zero divergences in 100,000 steps** — the probability of a repeated minimum is negligible. That contrast is the whole lesson about test data.
3. UBSan reports `signed integer overflow: 2 * -2000000000 cannot be represented in type 'int'`. Without the sanitiser, `getMin()` returns a **wrapped, arbitrary value** — typically a large positive number — and the stack silently reports a minimum that is not in it.
4. Correctness is **unaffected**; only the cost changes. Alternating push/pop with the guard is  **$O(1)$  per operation**, without it each pop shifts the whole queue. Expect the guarded version to finish in **tens of milliseconds** and the unguarded one in **seconds to minutes** depending on how the alternation is shaped — a ratio in the **hundreds to thousands**.
5. The actual ratio should settle near **4.0** and never exceed it. If it exceeds 4, something is shifting more than once per element — check the guard.
6. The buffer ends as **“ACB”** (or **“ABC”**, depending on how the offset is recorded). **Neither is a state the user was ever in**, and if the action records an absolute offset that no longer exists, the redo may write out of bounds entirely.
7. `vector::erase(begin())` is  **$O(\text{limit})$**  per trim; `deque::pop_front()` is  **$O(1)$** . At a limit of 100 the difference is small — expect **1.5-4×** — but rerun with a limit of 100,000 and the ratio should exceed **1000×**.
8. Expect `std::sort` round-trip at roughly **0.1 ms / 1.5 ms / 15 ms** for the three sizes, and `sortStack` at roughly **2 ms / 200 ms / 20 s** — ratios of about **20×** / **130×** / **1300×**, growing as  $n$  does because one is  $O(n \log n)$  and the other  $O(n^2)$ . `sortStack` passes one second at around  **$n = 25,000\text{--}35,000$** .

# HANDNOTE SUMMARY CARD

## PART 8 - AUGMENTED DESIGNS

Topics 56-61

MIN STACK - you cannot recompute on pop ( $O(n)$ ); RECORD THE ANSWER AT PUSH TIME.  
works for any ASSOCIATIVE fold: min, max, sum, product, gcd, and/or.

```

v1 PAIRS st of (value, min at or below)
push: m = st.empty() ? v : min(v, st.top().second); st.push({v,m});
getMin: st.top().second space: exactly 2n. correct and obvious.

```

```
v2 AUXILIARY second stack holds only the running minima
push: st.push(v); if (mins.empty() || v <= mins.top()) mins.push(v);
 ^^ MUST be <=, not <
pop : if (st.top() == mins.top()) mins.pop(); st.pop();
space: n+1 increasing input, 2n decreasing input
TEST: push 3, push 3, pop, getMin -> must be 3
```

v3 ENCODED                    0(1) extra space. Know it, do NOT ship it.

```
push v < mn -> st.push(2*v - mn); mn = v; 2v-m < v < m, so an
pop t < mn -> mn = 2*mn - t; encoded entry is
top t < mn -> return mn; SELF-IDENTIFYING
```

needs 2x the integer width:  $2*(-2e9) - 2e9 = -6e9$  is signed overflow = UB  
once widened to long long it saves nothing anyway.

QUEUE FROM TWO STACKS - amortised  $O(1)$

```
push -> in_. pop/front -> if (out_.empty()) shift(); then out_.
shift: while (!in_.empty()) { out_.push(in_.top()); in_.pop(); }
```

PROOF: each element does exactly 4 stack ops for life (in-push, in-pop, out-push, out-pop), independent of interleaving.  $m$  ops  $\leq 4m$ .

worst case single pop:  $O(n)$ . p99 matters -> use a real deque (T6).

UNCONDITIONAL shift, or shifting back on push, gives  $O(n)$  per op. Guard it.

STACK FROM TWO QUEUES - no amortisation exists. Choose which op is  $O(n)$ .  
 costly push: enqueue then rotate size-1 elements      push  $O(n)$ , pop  $O(1)$   
 costly pop : drain all but one into b, swap            push  $O(1)$ , pop  $O(n)$   
 WHY THE ASYMMETRY: two stacks reverse ONCE, permanently. Two queues must  
 cycle past every older element EVERY time - no reusable state is left.

UNDO / REDO - two stacks, and one line everyone forgets  
 perform(a): a.apply(); undo.push(a); redo.CLEAR();    <- correctness, not UI  
 undo():     a = undo.pop(); a.revert(); redo.push(a);  
 redo():     a = redo.pop(); a.apply(); undo.push(a);  
 without the clear: type A, type B, undo, type C, redo -> "ACB", a state  
 the user was NEVER in, possibly at an offset that no longer exists.  
 BOUNDED history CANNOT use std::stack (no way to drop the oldest).  
     use deque + pop front. 10,000 actions x 50 KB snapshot = 500 MB -> OOM.

```

SORTING A STACK WITH A STACK = INSERTION SORT
while (!s.empty()) { cur = s.pop();
 while (!aux.empty() && aux.top() > cur) s.push(aux.pop());
 aux.push(cur); }
0(n^2) comparisons and moves, 0(n) space. aux ends largest-on-top.
n=100,000: ~5e9 moves vs std::sort's ~1.7e6 comparisons -> ~3000x.
exists only to satisfy the constraint. Real answer: vector + std::sort.

```

# Stacks inside algorithms

---

Where the stack is not the subject but the mechanism. Why iterative DFS visits in a different order than the recursive version, the three tree traversals without recursion, quicksort with a bounded stack, and the two places a stack appears in graph algorithms for entirely different reasons.

## 62. Iterative DFS and visit order

Depth-first search is the recursion-to-stack conversion of Topic 25 applied to a graph. The mechanical translation works, but it has one surprise that catches people in interviews: **the iterative version does not visit vertices in the same order as the recursive one** unless you take care.

```
void dfsIterative(const std::vector<std::vector<int>>& adj, int start) {
 std::vector<bool> seen(adj.size(), false);
 std::stack<int> st;
 st.push(start);
 while (!st.empty()) {
 int u = st.top(); st.pop();
 if (seen[u]) continue; // may be pushed more than once
 seen[u] = true;
 visit(u);
 // push in REVERSE so the first neighbour is processed first
 for (auto it = adj[u].rbegin(); it != adj[u].rend(); ++it)
 if (!seen[*it]) st.push(*it);
 }
}
```

graph: 0 -> [1, 2], 1 -> [3], 2 -> [3], 3 -> []

RECURSIVE dfs(0): visits 0, 1, 3, 2  
dfs(0) -> dfs(1) -> dfs(3) -> return -> dfs(2) -> 3 already seen

ITERATIVE, pushing neighbours in FORWARD order:  
pop 0, push 1, 2      stack [1,2]  
pop 2 <-- the LAST neighbour comes off first!  
visits 0, 2, 3, 1      -- a DIFFERENT but still valid DFS order

ITERATIVE, pushing in REVERSE order:  
pop 0, push 2, 1      stack [2,1]  
pop 1  
visits 0, 1, 3, 2      -- identical to the recursive version

Both are valid depth-first orders. Only one MATCHES the recursion.  
Problems that check output against an expected sequence need the reverse push.

### The mark-on-pop versus mark-on-push distinction

| Approach                 | Behaviour                                   | Consequence                                                             |
|--------------------------|---------------------------------------------|-------------------------------------------------------------------------|
| Mark seen on <b>pop</b>  | A vertex may sit on the stack several times | Matches recursion; stack can hold $O(E)$ entries                        |
| Mark seen on <b>push</b> | Each vertex enters the stack once           | Stack bounded by $O(V)$ , but the visit order can differ from recursion |

#### THE KEY INSIGHT

Swapping `std::stack` for `std::queue` in this exact code turns DFS into BFS. That is the cleanest illustration in the book of Topic 1's claim that the container *is* the algorithm: same loop, same marking, same adjacency traversal — one word changes the shape of the search from deep to wide.

### THE TRAP

Marking on push and then also checking `seen` on pop, or neither. Marking only on pop without the `if (seen[u]) continue;` guard visits vertices more than once, so a counting or accumulating DFS silently over-counts — and on a dense graph the stack grows to  $O(E)$  rather than  $O(V)$ , which for a complete graph on 10,000 vertices is **50 million entries** instead of 10,000. Symptom: correct on trees, wrong and memory-hungry on graphs with cycles.

iterative DFS – Time  $O(V + E)$  · Space  $O(V)$  marking on push,  $O(E)$  marking on pop · push neighbours reversed to match recursion · stack → queue converts DFS to BFS

## 63. Iterative inorder traversal

Inorder is the traversal most worth knowing iteratively, because on a binary search tree it yields the keys in sorted order and because the classic implementation is shorter and faster than the general stage-field version of Topic 26.

```
void inorderIterative(Node* root) {
 std::stack<Node*> st;
 Node* cur = root;
 while (cur || !st.empty()) {
 while (cur) { // walk as far LEFT as possible
 st.push(cur);
 cur = cur->left;
 }
 cur = st.top(); st.pop(); // the leftmost unvisited node
 visit(cur);
 cur = cur->right; // then handle its right subtree
 }
}
```

The loop condition `cur || !st.empty()` is what makes it work: the traversal is not finished when the stack empties, because `cur` may still point into an unexplored right subtree. Writing `while (!st.empty())` alone truncates the traversal after the root's right child.

```

tree:
 4
 /\
 2 6
 /\ /\
 1 3 5 7

```

| step | cur  | stack (bottom->top) | action                                |
|------|------|---------------------|---------------------------------------|
| 1    | 4    | []                  | descend left: push 4, 2, 1            |
| 2    | null | [4,2,1]             | pop 1, VISIT 1, cur = 1->right = null |
| 3    | null | [4,2]               | pop 2, VISIT 2, cur = 3               |
| 4    | 3    | [4]                 | descend: push 3                       |
| 5    | null | [4,3]               | pop 3, VISIT 3, cur = null            |
| 6    | null | [4]                 | pop 4, VISIT 4, cur = 6               |
| 7    | 6    | []                  | descend: push 6, 5                    |
| 8    | null | [6,5]               | pop 5, VISIT 5, cur = null            |
| 9    | null | [6]                 | pop 6, VISIT 6, cur = 7               |
| 10   | 7    | []                  | descend: push 7                       |
| 11   | null | [7]                 | pop 7, VISIT 7, cur = null            |
| 12   | null | []                  | both empty -> done                    |

output 1 2 3 4 5 6 7 -- sorted, as expected for a BST  
max stack depth = tree HEIGHT, which is 3 here, not the node count

#### THE KEY INSIGHT

The stack holds exactly the *ancestors of the current node whose right subtrees are still unexplored*. That is why its depth is the tree height, not the node count: on a balanced tree of  $10^6$  nodes it holds about 20 pointers. On a degenerate left-spine tree it holds all  $10^6$  — which is precisely the input that kills the recursive version, and here merely costs 8 MB of heap.

#### THE TRAP

Using `while (!st.empty())` as the outer condition. It works on the first left spine and then stops, because after visiting the root the stack is empty while `cur` points at the right child. Symptom: output that is correct up to the root and then simply ends — on the tree above you get 1 2 3 4 and nothing more. Both terms of the disjunction are required.

iterative inorder — Time  $O(n)$  · Space  $O(\text{height})$ : ~20 for a balanced  $10^6$ -node tree,  $n$  for a degenerate one · loop while `cur || !st.empty()`

## 64. Iterative preorder and postorder

Preorder is the easy one; postorder is the one interviewers ask about, and the two-stack trick makes it almost as easy.

## Preorder: root, left, right

```
void preorderIterative(Node* root) {
 if (!root) return;
 std::stack<Node*> st;
 st.push(root);
 while (!st.empty()) {
 Node* n = st.top(); st.pop();
 visit(n);
 if (n->right) st.push(n->right); // RIGHT first, so LEFT pops first
 if (n->left) st.push(n->left);
 }
}
```

## Postorder: the reversal trick

Postorder is left, right, root. Run a *modified preorder* that visits root, right, left — the exact reverse of postorder — and reverse the output. A second stack collects it.

```
void postorderIterative(Node* root) {
 if (!root) return;
 std::stack<Node*> st, out;
 st.push(root);
 while (!st.empty()) {
 Node* n = st.top(); st.pop();
 out.push(n); // collect in root-right-left order
 if (n->left) st.push(n->left); // LEFT first now -- mirrored
 if (n->right) st.push(n->right);
 }
 while (!out.empty()) { visit(out.top()); out.pop(); } // reverse it
}
```

```
tree: 1
 / \
 2 3
 / \
 4 5
```

```
preorder (root,left,right) -> 1 2 4 5 3
inorder (left,root,right) -> 4 2 5 1 3
postorder (left,right,root) -> 4 5 2 3 1
```

Two-stack postorder:

```
modified preorder visits 1 3 2 5 4 (root, RIGHT, LEFT)
reverse it -> 4 5 2 3 1 = postorder. Correct.
```

Space: the "out" stack holds ALL n nodes, so this is  $O(n)$  space, not  $O(\text{height})$ . The single-stack postorder below is  $O(\text{height})$  but needs a "last visited" pointer and is much easier to get wrong.

## The single-stack version, for completeness

```
void postorderOneStack(Node* root) {
 std::stack<Node*> st;
 Node* cur = root;
 Node* lastVisited = nullptr;
 while (cur || !st.empty()) {
 while (cur) { st.push(cur); cur = cur->left; }
 Node* peek = st.top();
 // go right only if a right child exists AND we did not just come back
 if (peek->right && lastVisited != peek->right) {
 cur = peek->right;
 } else {
 visit(peek);
 lastVisited = peek;
 st.pop();
 }
 }
}
```

### THE KEY INSIGHT

Postorder reversed is a mirrored preorder. Recognising that turns the hardest of the three traversals into the easiest one plus a reversal — and the reversal is itself a stack. Under interview pressure, produce the two-stack version and mention that the single-stack version exists and is  $O(\text{height})$ ; that answer is complete without being risky.

### THE TRAP

In the single-stack version, omitting the `lastVisited != peek->right` test. Without it, after returning from the right subtree the code descends into it again, forever. Symptom: an infinite loop with steadily growing memory on any tree that has a right child — and it terminates correctly on a pure left spine, which is exactly the shape people use for a quick test.

preorder  $O(n)$  time,  $O(\text{height})$  space · two-stack postorder  $O(n)$  time,  **$O(n)$  space** · single-stack postorder  $O(n)$  time,  $O(\text{height})$  space, needs `lastVisited` · push right before left in preorder

## 65. Iterative quicksort with bounded depth

Recursive quicksort has  $O(n)$  worst-case recursion depth on already-sorted input, which is a stack overflow on large arrays. Two changes fix it: make the recursion explicit, and always handle the *smaller* partition first — which bounds the stack at  $O(\log n)$  regardless of input.



```

void quicksortIterative(std::vector<int>& a) {
 std::stack<std::pair<int,int>> st;
 if (a.size() > 1) st.push({0, (int)a.size() - 1});
 while (!st.empty()) {
 auto [lo, hi] = st.top(); st.pop();
 if (lo >= hi) continue;
 int p = partition(a, lo, hi);
 // Push the LARGER side, then recurse into the smaller by pushing it
 // last so it is processed first. Stack depth stays O(log n).
 if (p - lo < hi - p) {
 st.push({p + 1, hi});
 st.push({lo, p - 1});
 } else {
 st.push({lo, p - 1});
 st.push({p + 1, hi});
 }
 }
}

```

Why smaller-first bounds the stack at  $O(\log n)$

The partition processed FIRST is at most half the current range. Anything left ON the stack is therefore at least as large as its sibling, so each stack entry covers at least twice the elements of the entry above it:

entry  $k$  covers  $\geq 2 * (\text{elements covered by entry } k+1)$

With  $n$  elements total, there can be at most  $\log_2(n)$  such entries.  
 $n = 10^6 \rightarrow$  at most 20 stack entries, 8 bytes each = 160 bytes.

WITHOUT the smaller-first rule, on already-sorted input every partition splits  $1 : n-1$ , so the stack grows to  $n$  entries:  
 $n = 10^6 \rightarrow 10^6$  entries = 8 MB on the HEAP (survivable)  
 the RECURSIVE version would need  $10^6$  FRAMES = 48+ MB  $\rightarrow$  SIGSEGV

#### THE KEY INSIGHT

“Recurse into the smaller partition first” is a two-line change that converts a worst-case  $O(n)$  stack into a worst-case  $O(\log n)$  stack, with no effect on the  $O(n \log n)$  average time or the  $O(n^2)$  worst-case time. It is what production quicksorts do — libstdc++'s introsort applies the same principle alongside its depth-limit switch to heapsort.

#### THE TRAP

Pushing the smaller partition *first*, which means the larger one is on top and gets processed first — exactly inverting the guarantee. The sort is still correct and the average case is unaffected, so every random test passes; only a sorted or reverse-sorted input reveals the  $O(n)$  stack. Symptom: memory that spikes to gigabytes on the one input shape that should be easiest.

iterative quicksort – Time  $O(n \log n)$  average,  $O(n^2)$  worst · Space  $O(\log n)$  with smaller-first,  $O(n)$  without · 20 stack entries for  $n = 10^6$  · no call-stack use at all

## 66. Topological sort and Tarjan's stack

Two graph algorithms use a stack, for entirely different reasons. Confusing them is common; distinguishing them cleanly is the point of this topic.

### Topological sort: the stack is an output buffer

Run DFS. When a vertex is *finished* — all its descendants explored — push it. The reverse of finishing order is a topological order. The stack here is doing nothing algorithmic; it is reversing a sequence.

```
void topoSort(const std::vector<std::vector<int>>& adj) {
 int n = adj.size();
 std::vector<int> state(n, 0); // 0 unvisited, 1 in progress, 2 done
 std::stack<int> order; // OUTPUT buffer, not a work stack
 std::stack<std::pair<int, std::size_t>> work; // (vertex, next child index)

 for (int s = 0; s < n; ++s) {
 if (state[s] != 0) continue;
 work.push({s, 0});
 state[s] = 1;
 while (!work.empty()) {
 auto& [u, i] = work.top();
 if (i < adj[u].size()) {
 int v = adj[u][i++]; // note: mutates the stored index
 if (state[v] == 1) throw std::runtime_error("cycle");
 if (state[v] == 0) { state[v] = 1; work.push({v, 0}); }
 } else {
 state[u] = 2;
 order.push(u); // FINISHED -- push to output
 work.pop();
 }
 }
 }
 while (!order.empty()) { std::cout << order.top() << ' '; order.pop(); }
}
```

### Tarjan's SCC: the stack is the algorithm

Tarjan's strongly-connected-components algorithm keeps a separate stack whose contents have a meaning: **the vertices visited so far whose component has not yet been closed**. When a vertex is found to be the root of an SCC (`low[u] == disc[u]`), everything above it on that stack is exactly its component, popped in one go.

TWO STACKS, TWO PURPOSES -- do not conflate them

topological sort:    stack = OUTPUT BUFFER  
                         push on FINISH, pop to print. Any reversing  
                         container would do; a vector + `std::reverse` is  
                         equivalent and often clearer.

Tarjan's SCC:        stack = THE SET OF OPEN VERTICES  
                         its CONTENTS carry meaning at every moment:  
                         "visited, and whose SCC is not yet closed"  
                         when `low[u] == disc[u]`, pop until `u` -- that  
                         run of vertices IS one strongly connected  
                         component. No other structure substitutes.

Test: could you replace the stack with a vector and a reverse at  
      the end?    YES -> it is an output buffer.  
                  NO -> it is load-bearing.

#### THE KEY INSIGHT

Ask of any stack in an algorithm: *do its contents mean something at every moment, or is it just reversing a sequence?* In topological sort it is reversing; in Tarjan and in every monotonic-stack problem of Part 7 the contents are load-bearing. That question tells you whether the stack can be swapped for something simpler and whether you actually understand the algorithm.

#### THE TRAP

In the iterative topological sort above, `auto& [u, i] = work.top();` binds a reference into the stack and then `work.push` happens inside the same iteration — the Topic 26 dangling-frame bug in a graph algorithm. With the default `std::deque` backing it works; with `std::vector` backing it is a use-after-free on the very next iteration. Copy `u` by value and re-fetch `work.top()` after any push.

topological sort – Time  $O(V + E)$ , Space  $O(V)$ , stack is an output buffer · Tarjan – Time  $O(V + E)$ , Space  $O(V)$ , stack contents are the open vertices · ask whether a reverse would substitute

## PRACTICE SET — Topics 62-66

---

### A. Conceptual

1. Why does iterative DFS visit vertices in a different order than the recursive version when neighbours are pushed in forward order? What is the fix?
2. Are both orders valid depth-first traversals? Under what circumstance does the difference actually matter?
3. Compare marking vertices seen on push versus on pop. Give the space bound for each and state which one matches the recursive visit order.
4. What single word change turns the iterative DFS loop into BFS, and what does that demonstrate about the relationship between container and algorithm?
5. In iterative inorder, what exactly does the stack contain at any moment? Derive the space bound from that description.
6. Why is the outer loop condition `cur || !st.empty()` rather than just `!st.empty()`? Describe the output you get with the wrong condition.
7. In iterative preorder, why are children pushed right-before-left?
8. State the relationship between postorder and preorder that makes the two-stack postorder work.
9. Give the space bound of two-stack postorder and of the single-stack version, and say why they differ.
10. What is the role of `lastVisited` in single-stack postorder, and what happens without it?
11. Why does "recurse into the smaller partition first" bound the quicksort stack at  $O(\log n)$ ? Give the argument in terms of what each remaining stack entry covers.
12. Does the smaller-first rule change quicksort's time complexity in any case?
13. In topological sort, what role does the stack play? Give the test that identifies this role.
14. In Tarjan's algorithm, state precisely what the stack's contents mean at any moment.
15. Give the one question to ask of any stack appearing inside an algorithm, and say what each answer tells you.

### B. Tracing and analysis

1. For the graph `0 -> [1,2], 1 -> [3,4], 2 -> [4], 3 -> [], 4 -> []`, give the visit order for: recursive DFS from 0; iterative DFS pushing forward; iterative DFS pushing reversed; and BFS.
2. For a complete graph on 1000 vertices, compute the maximum number of stack entries under mark-on-pop and under mark-on-push.
3. Trace iterative inorder on the BST with root 4, left subtree (2, children 1 and 3), right subtree (6, children 5 and 7). Give `cur` and the stack at every step, plus the maximum stack depth.
4. Give the maximum stack depth of iterative inorder for a balanced tree of 1,000,000 nodes and for a left-spine tree of the same size. Give the bytes for each assuming 8-byte pointers.
5. For the tree with root 1, left 2 (children 4, 5), right 3, give preorder, inorder and postorder by hand. Then trace the two-stack postorder, showing both stacks after each iteration.
6. Trace single-stack postorder on the same tree. Record every value taken by `lastVisited` and identify the iteration at which the `lastVisited != peek->right` test first prevents a re-descent.

7. An array of 1,000,000 elements is already sorted. Give the maximum stack depth for iterative quicksort with smaller-first, and without it. Convert both to bytes assuming an 8-byte pair per entry.
8. For the same sorted input, compute the recursion depth and total call-stack bytes the recursive version would need with a 48-byte frame, and state whether it survives an 8 MB limit.
9. For the DAG `0 -> [1,2], 1 -> [3], 2 -> [3], 3 -> []`, give the finishing order and the resulting topological order. Then state how many distinct valid topological orders this DAG has.
10. For each of these stacks, say whether it is an output buffer or load-bearing: the monotonic stack of Topic 45; the `out` stack of two-stack postorder; Tarjan's SCC stack; the `pending` stack of Topic 25's `fact_iter`; the `order` stack of topological sort.

### C. Code tasks

1. **Build.** Implement iterative DFS, all three iterative traversals, iterative quicksort and iterative topological sort. Verify each against a recursive reference on 10,000 random inputs.
2. **Break it — the visit order.** Push DFS neighbours in forward order. Compare the output against the recursive version on 1000 random graphs and report the fraction that differ. Then confirm that every differing output is still a valid DFS order by checking that each visited vertex is adjacent to some already-visited one.
3. **Break it — the missing seen guard.** Remove `if (seen[u]) continue;` from mark-on-pop DFS. Run it on a complete graph of 2000 vertices. Report the number of `visit` calls versus the vertex count, and the peak stack size. Predict both from theory first.
4. **Break it — the inorder loop condition.** Change the outer condition to `while (!st.empty())`. Run it on the seven-node BST and report the exact output. Explain in one sentence which nodes are lost and why.
5. **Break it — the postorder infinite loop.** Remove `lastVisited != peek->right` from single-stack postorder. Run it on a left-spine tree of 10 nodes and then on the seven-node balanced tree, with a 1-second timeout. Report which terminates and which does not, and explain why a left-spine test would have passed review.
6. **Break it — the quicksort push order.** Push the smaller partition first so the larger is processed first. Instrument the peak stack size. Run on 1,000,000 random elements and on 1,000,000 already-sorted elements. Report both peaks for the correct and the broken version — four numbers.
7. **Break it — the dangling structured binding.** In the iterative topological sort, keep `auto& [u, i] = work.top();` and run it with `std::stack<std::pair<int, size_t>, std::deque<...>>` and then with a `std::vector` backing, on a DAG deep enough to force reallocation. Report the ASan class for each.
8. **Measure.** Time recursive versus iterative inorder on a balanced tree of 10,000,000 nodes and report both. Then build a left-spine tree of 1,000,000 nodes and run both again, reporting which crashes and at what depth.

## ANSWER KEY — Topics 62-66

---

### A. Conceptual

1. Because a stack reverses: the **last neighbour pushed is the first popped**, so forward pushing explores the last neighbour first. Fix: **push neighbours in reverse order**.
2. **Yes, both are valid**. It matters only when the output is compared against an expected sequence — judge-style problems, golden-file tests, or any specification that fixes the order.
3. Mark on **push**: each vertex enters once, space  $O(V)$ , but the visit order may differ from recursion. Mark on **pop**: a vertex may sit on the stack several times, space  $O(E)$ , and it **matches the recursive order**.
4. Replace `std::stack` with `std::queue`. It demonstrates that **the container is the algorithm** — identical loop, marking and adjacency handling, yet the search changes from deep to wide.
5. It contains **the ancestors of the current node whose right subtrees are still unexplored**. Since ancestors form a root-to-node path, the count is at most the tree height, giving  $O(\text{height})$ .
6. Because after visiting a node the stack may be empty while `cur` still points into an unexplored right subtree. With the wrong condition the traversal **stops right after the root** — on the seven-node BST you get `1 2 3 4` and nothing else.
7. Because the stack reverses, so pushing right first makes **left pop first**, which is the required root-left-right order.
8. **Postorder reversed is a mirrored preorder** — a preorder that visits root, right, left. So run that and reverse the output.
9. Two-stack:  $O(n)$ , because the output stack accumulates every node before anything is visited. Single-stack:  $O(\text{height})$ , because it only ever holds ancestors. The difference is that the two-stack version defers all visiting to the end.
10. It records the node whose subtree was just finished, so the algorithm can tell **whether it is descending into the right child or returning from it**. Without it, the code descends into the right subtree again immediately — an **infinite loop**.
11. Because the partition processed first is at most half the current range, so anything *left on the stack* is at least as large as its sibling. Each stack entry therefore covers **at least twice the elements of the entry above it**, and with  $n$  elements there can be at most  $\log_2 n$  such entries.
12. **No**. Average stays  $O(n \log n)$  and the worst case stays  $O(n^2)$ . Only the space bound changes.
13. It is an **output buffer** — it reverses the finishing order. Test: **could a vector plus `std::reverse` substitute?** Yes → output buffer.
14. The vertices **visited so far whose strongly connected component has not yet been closed**. When `low[u] == disc[u]`, everything above and including `u` is exactly one SCC.
15. **“Do its contents mean something at every moment, or is it just reversing a sequence?”** Reversing → it can be swapped for a vector and a reverse. Meaningful → it is load-bearing and no substitution exists.

### B. Tracing and analysis

1. Recursive: **0 1 3 4 2**. Iterative pushing forward: **0 2 4 1 3**. Iterative pushing reversed: **0 1 3 4 2**. BFS: **0 1 2 3 4**.

2. Complete graph on 1000 vertices has 999,000 directed edges. Mark-on-pop: up to **999,000 entries** (bounded by E). Mark-on-push: **1000 entries** (bounded by V) — a ratio of about **1000×**.
3. Same trace as Topic 63: output **1 2 3 4 5 6 7**, maximum stack depth **3** (nodes 4, 2, 1 held simultaneously).
4. Balanced: height =  $\log_2(10^6) \approx 20$  pointers = **160 bytes**. Left spine: height = **1,000,000** pointers = **8,000,000 bytes  $\approx$  8 MB** — on the heap, so survivable, unlike the recursive version.
5. Preorder **1 2 4 5 3**; inorder **4 2 5 1 3**; postorder **4 5 2 3 1**. Two-stack trace: **st** pops 1, 3, 2, 5, 4 while **out** accumulates **1, 3, 2, 5, 4** bottom-to-top; draining **out** gives **4 5 2 3 1**.
6. **lastVisited** takes the values **4, 5, 2, 3, 1** in that order. The test first prevents a re-descent at node **2**, on the iteration immediately after node 5 is visited — without it, **cur** would be set to node 5 again.
7. Smaller-first:  $\log_2(10^6) \approx 20$  entries = **160 bytes**. Without: **1,000,000 entries = 8,000,000 bytes  $\approx$  8 MB** — on the heap, so it survives, but it is a 50,000× increase.
8. Recursion depth **1,000,000**, at 48 bytes per frame = **48,000,000 bytes = 48 MB**. Against an 8 MB limit it **does not survive** — it dies at roughly depth **174,762**.
9. Finishing order **3, 1, 2, 0** (or 3, 2, 1, 0 depending on neighbour order), so the topological order is **0, 2, 1, 3** (or 0, 1, 2, 3). This DAG has **two** valid topological orders, since 1 and 2 are incomparable.
10. Topic 45 monotonic stack: **load-bearing**. Two-stack postorder **out**: **output buffer**. Tarjan's SCC stack: **load-bearing**. **fact\_iter**'s **pending**: **output buffer** — a vector reversed would do. Topological sort's **order**: **output buffer**.

### C. Code tasks — what to verify

1. All must match the recursive reference exactly, provided the DFS pushes neighbours reversed. If only DFS diverges, that is the expected order effect, not a bug — check validity rather than equality.
2. Expect a **large majority to differ** — anything above about 60–80% for graphs with average degree above 2, since only vertices with fewer than two unvisited neighbours produce the same order. Every differing output must still pass the adjacency check; if one fails, the bug is real and not merely an ordering difference.
3. Predicted and actual: **visit** is called once per **stack entry that is not already seen**, so with the guard removed expect roughly **E = 3,998,000** visit calls against **2000** vertices, and a peak stack near **E**. Memory climbs into the tens of megabytes for a graph that should need 2000 entries.
4. Output is **1 2 3 4**. The entire right subtree (5, 6, 7) is lost, because after visiting the root the stack is empty while **cur** points at node 6, and the loop exits before exploring it.
5. The **left-spine tree terminates correctly** (no node has a right child, so the broken test is never exercised); the **balanced tree loops forever** and is killed by the timeout. A left-spine test is the natural quick check and it passes — which is exactly why this bug reaches review.
6. Correct version: peak **~20** on random input and **~20** on sorted input. Broken version: peak **~20–40** on random input (the partitions are near-balanced anyway, so the bug is invisible) and **~1,000,000** on sorted input. The random case hiding the bug completely is the lesson.
7. **std::deque** backing: **no error reported**, runs correctly. **std::vector** backing: **heap-use-after-free**, with a **READ of size 8** on the next iteration's access to **i**, and a **freed by thread T0** here frame pointing inside **push**'s reallocation. Same source, one backing safe and one a security bug — Topic 16 and Topic 26 recurring inside a graph algorithm.

8. On a balanced tree the recursive version is typically **equal or 5-20% faster**, thanks to the hardware return-address predictor (Topic 29's practice set). On the 1,000,000-node left spine the **recursive version segfaults** at roughly depth **170,000-200,000** while the iterative version completes using about 8 MB of heap. Convert for depth safety, not for speed.



# HANDNOTE SUMMARY CARD

## PART 9 - STACKS INSIDE ALGORITHMS

Topics 62-66

### ITERATIVE DFS

```
st.push(start);
while (!st.empty()) { u = st.top(); st.pop();
 if (seen[u]) continue; <- MANDATORY guard
 seen[u] = true; visit(u);
 for (v : reverse(adj[u])) if (!seen[v]) st.push(v); }
```

PUSH NEIGHBOURS REVERSED or the order will not match the recursive version  
(both orders are valid DFS; only one matches a golden-file expectation)  
mark on POP -> matches recursion, stack up to  $O(E)$  (K1000: 999,000 entries)  
mark on PUSH -> stack  $O(V)$ , order may differ (K1000: 1,000 entries)  
std::stack -> std::queue turns DFS into BFS. The container IS the algorithm.

### INORDER (left, root, right)

```
while (cur || !st.empty()) {
 while (cur) { st.push(cur); cur = cur->left; } -> depth = tree HEIGHT
 cur = st.top(); st.pop();
 visit(cur);
 cur = cur->right;
}
```

stack holds: the ancestors whose RIGHT subtrees are still unexplored  
-> depth = tree HEIGHT  
balanced  $10^6$  nodes: 20 ptrs = 160 B  
left spine  $10^6$  : 8 MB, on the HEAP  
(the recursive version dies here)  
"while (!st.empty())" alone TRUNCATES after the root: 1 2 3 4 and stop.

### PREORDER (root, left, right)

```
st.push(root);
pop, visit, push RIGHT then LEFT
 $O(\text{height})$ space
```

### POSTORDER (left, right, root)

```
postorder REVERSED == mirrored
preorder (root, RIGHT, LEFT)
two-stack: collect into out, then
drain. $O(n)$ SPACE, easy, safe.
one-stack: $O(\text{height})$, needs
lastVisited != peek->right or it
INFINITE LOOPS. terminates fine on
a left spine -> passes quick tests.
```

### ITERATIVE QUICKSORT - push (lo,hi) pairs, no call stack at all

```
process the SMALLER partition first (push the larger, then the smaller)
WHY: each remaining entry covers $\geq 2x$ the entry above it -> $\leq \log_2(n)$
 $n = 10^6$: smaller-first 20 entries (160 B) | wrong order 10^6 (8 MB)
recursive on sorted input: 10^6 frames x 48 B = 48 MB -> SIGSEGV
time is UNCHANGED: $O(n \log n)$ average, $O(n^2)$ worst. Only space changes.
random input hides the bug completely - test on SORTED input.
```

### TOPOLOGICAL SORT vs TARJAN - two stacks, two different purposes

```
toposort: push on FINISH, drain to print. = OUTPUT BUFFER
a vector + std::reverse substitutes exactly.
Tarjan : stack contents = "visited, SCC not yet closed". = LOAD-BEARING
low[u] == disc[u] -> pop until u; that run IS one SCC.
```

### THE QUESTION TO ASK OF ANY STACK IN AN ALGORITHM:

```
do its contents mean something at every moment, or is it just reversing?
reversing -> swap for vector + reverse, often clearer
meaningful -> load-bearing, no substitution exists
load-bearing: Part 7 monotonic stacks, Tarjan, shunting-yard operator stack
output buffer: toposort order, two-stack postorder out, fact_iter pending
```

WATCH FOR: auto& [u,i] = work.top(); followed by work.push(...) in the same iteration. deque backing: fine. vector backing: heap-use-after-free.  
(T16 and T26 recurring inside a graph algorithm.) Copy, or re-fetch top().



# Library, safety, interview

---

What the standard library actually does, what the other four languages you will be asked about do differently, the two ways stacks get you compromised rather than merely crashed, how to read the debugger output when they do — and the closing argument against the whole structure.

## 67. std::stack: deque vs vector, benchmarked

`std::stack` is an adaptor (Topic 19), and its default backing container is `std::deque` — not `std::vector`. That default is defensible and usually wrong for your workload, and knowing why is a reliable interview differentiator.

```
std::stack<int> a; // deque backing (default)
std::stack<int, std::vector<int>> b; // vector backing
std::stack<int, std::deque<int>> c; // explicit, same as a

// vector backing also unlocks reserve, via the protected member c:
struct ReservableStack : std::stack<int, std::vector<int>> {
 void reserve(std::size_t n) { c.reserve(n); } // c is the standard's name
};
```

### Why deque is the default

`std::deque` allocates fixed-size blocks — 512 bytes per block in libstdc++ — and keeps a separate array of block pointers. Growth adds a block and reallocates only the pointer array, so **existing elements never move**. That gives three properties `vector` lacks: references from `top()` stay valid across pushes (Topic 16), there is no  $O(n)$  reallocation pause (Topic 9), and peak memory never doubles. The committee chose predictability over speed.

| Property                           | deque backing                                    | vector backing |
|------------------------------------|--------------------------------------------------|----------------|
| ns per push+pop pair               | 2-4                                              | <b>0.8-1.5</b> |
| Indirections per access            | 2 (block index, then block)                      | <b>1</b>       |
| Worst-case single push             | <b>O(1)</b>                                      | $O(n)$         |
| <code>top()</code> survives a push | <b>Yes</b>                                       | No             |
| Peak memory for n ints             | <b><math>\sim 4n + \text{block index}</math></b> | up to $8n$     |
| Empty stack costs                  | 512 B block allocated eagerly                    | <b>0 B</b>     |
| <code>reserve()</code> available   | No                                               | <b>Yes</b>     |

deque layout (libstdc++, 512-byte blocks = 128 ints)

block index (contiguous array of pointers)

```
+-----+-----+-----+-----+
| ptr | ptr | ptr | - |
+---+---+---+---+
 v v v
```

[128 ints][128 ints][128 ints] <- blocks, NEVER moved once allocated  
^ top lives here

growth: allocate one new 512 B block, and only sometimes reallocate the small pointer array. Existing elements are untouched.

vector layout

```
+-----+
| 10 | 20 | 30 | .. | .. | .. | .. | .. | <- ONE block, doubles on growth
+-----+ every element MOVES
```

#### THE KEY INSIGHT

Use `std::stack<T>`, `std::vector<T>>` when you know the approximate size and want speed; keep the deque default when you hold references across pushes or need a hard bound on a single operation. Saying that sentence — rather than “`std::stack` uses a deque” — is what separates a memorised fact from an engineering judgement in an interview.

#### THE TRAP

Switching the backing to `std::vector` for a measured 2× speed win in code that holds a reference from `top()` across a `push`. It is a one-word change, the tests pass, and it introduces a use-after-free that fires only at reallocation boundaries. Symptom: intermittent corruption correlated with data size, appearing weeks after a “pure performance” commit. Audit for held references *before* changing the backing, not after.

`std::stack` backings – deque: 2–4 ns/op, O(1) worst-case push, references survive · vector: 0.8–1.5 ns/op, O(n) worst push, references die, `reserve()` available

## 68. The other languages: Python, Java, PHP, JavaScript

Interviews are language-agnostic until they are not. Four facts, one per language, cover almost everything you will be asked.

### Python: use a list, never `pop(0)`

```
st = []
st.append(4) # push -- amortised O(1)
st[-1] # top -- O(1), IndexError if empty
st.pop() # pop -- O(1), returns the value
st.pop(0) # O(n)! shifts every remaining element. This is the trap.
from collections import deque
q = deque(); q.append(4); q.popleft() # O(1) both ends -- for QUEUES
```

A Python list is a dynamic array with a growth factor of roughly **1.125** plus a constant — much gentler than doubling, so it reallocates far more often but wastes far less memory. `list.pop()` with no argument is the stack operation; `pop(0)` is O(n) and is the single most common performance bug in Python data-structure code.

### Java: `Stack` is deprecated in practice

```
Stack<Integer> bad = new Stack<>(); // extends Vector -- every method
 // synchronized, and it exposes
 // get(i), so it is NOT a stack
Deque<Integer> good = new ArrayDeque<>(); // what the Javadoc recommends
good.push(4); good.peek(); good.pop();
```

`java.util.Stack` extends `Vector`, so it inherits synchronisation on every method and a public `get(int)` — Topic 1's trap, in the standard library. Its own Javadoc recommends `ArrayDeque` instead. Note also that `Stack` iterates bottom-to-top while `ArrayDeque` iterates top-to-bottom, so a naive migration silently reverses any iteration.

## PHP and JavaScript: arrays are the stack

```
// PHP
$st = [];
array_push($st, 4); // or $st[] = 4; -- the [] form is faster
$top = end($st);
array_pop($st);
$s = new SplStack(); // a real doubly-linked list, usually slower
```

```
// JavaScript
const st = [];
st.push(4);
st[st.length - 1]; // top -- undefined if empty, no throw
st.pop(); // returns undefined on an empty array, silently
```

| Language   | Idiomatic stack | Empty pop behaviour                     | Depth limit                                |
|------------|-----------------|-----------------------------------------|--------------------------------------------|
| C++        | std::stack      | Undefined behaviour                     | RAM (heap)                                 |
| Python     | list            | <code>IndexError</code>                 | RAM; recursion 1000                        |
| Java       | ArrayDeque      | <code>NoSuchElementException</code>     | RAM; recursion ~10–20k                     |
| PHP        | array           | Returns <code>null</code> + warning     | RAM; <code>xdebug.max_nesting_level</code> |
| JavaScript | Array           | Returns <code>undefined</code> silently | RAM; recursion ~10k                        |

### THE KEY INSIGHT

Every one of these is an array-backed stack underneath — Part 2 describes all of them. The differences are entirely in the *error strategy* of Topic 5: C++ chose undefined behaviour, Python and Java chose exceptions, PHP and JavaScript chose silent sentinels. The JavaScript case is the dangerous one, because `undefined` propagates arithmetic as `NaN` instead of failing.

### THE TRAP

`list.pop(0)` in Python inside a loop. Draining a 100,000-element list front-first is  $5 \times 10^9$  element moves — several seconds — where `collections.deque.popleft()` is microseconds. The code is correct, reads naturally, and is 1000× too slow. It is the single most common cause of “my Python solution times out” on BFS problems.

language survey — all array-backed, all amortised  $O(1)$  · Python list growth  $\sim 1.125 \times$  · `pop(0)` is  $O(n)$  · Java `Stack` is synchronised and indexable; use `ArrayDeque` · JS returns `undefined` silently

## 69. Concurrency: the Treiber stack and ABA

A lock-free stack is the classic introduction to atomic compare-and-swap, and also the classic introduction to the ABA problem. R. K. Treiber described it in 1986; the algorithm is six lines and the bug it hides is worth more than the algorithm.

```
template <class T>
class TreiberStack {
 struct Node { T v; Node* next; };
 std::atomic<Node*> head_{nullptr};
public:
 void push(const T& v) {
 Node* n = new Node{v, head_.load(std::memory_order_relaxed)};
 // retry until nobody changed head_ between our read and our write
 while (!head_.compare_exchange_weak(n->next, n,
 std::memory_order_release, std::memory_order_relaxed)) {}
 }
 bool pop(T& out) {
 Node* old = head_.load(std::memory_order_acquire);
 while (old && !head_.compare_exchange_weak(old, old->next,
 std::memory_order_acquire, std::memory_order_relaxed)) {}
 if (!old) return false;
 out = old->v;
 delete old; // <-- THE BUG LIVES HERE. See below.
 return true;
 }
};
```

## THE ABA PROBLEM

stack: A -> B -> C      two threads, T1 and T2

T1: pop() reads head\_ = A, reads A->next = B.  
T1 is descheduled here, holding old=A, next=B.

```
T2: pop() -> removes A, DELETES it. stack: B -> C
T2: pop() -> removes B. stack: C
T2: push(X) -> new Node happens to be ALLOCATED AT A's OLD ADDRESS
 (allocators reuse freed memory -- this is normal, not rare)
 stack: A(=X) -> C
```

```
T1: resumes. CAS(head_ expected A, desired B) -- head_ IS A, so it SUCCEEDS.
 stack becomes: B -> ... but B WAS ALREADY POPPED AND FREED.
```

The pointer value is unchanged. The world behind it is not.  
Result: use-after-free, a resurrected node, and a corrupted stack.  
CAS compares BITS, not IDENTITY.

## The three standard fixes

| Fix                     | Mechanism                                                                                                | Cost                                                   |
|-------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Tagged pointer          | Pack a counter into the unused high bits, or use <code>atomic&lt;pair&gt;</code> with a double-width CAS | Needs <code>cmpxchg16b</code> ; counter can still wrap |
| Hazard pointers         | Each thread publishes what it is reading; reclamation waits                                              | Correct and general; a read barrier per access         |
| Epoch-based reclamation | Defer all frees to a quiescent epoch                                                                     | Fastest in practice; memory is freed late              |

#### THE KEY INSIGHT

ABA is not a stack bug — it is a property of compare-and-swap: CAS compares a *value*, and a pointer that returns to its old value tells you nothing about what happened in between. The stack is simply the shortest program in which it appears. Any lock-free structure built on CAS over reusable memory has the same exposure.

#### THE TRAP

Writing the Treiber stack, testing it under contention, and concluding it is correct. ABA needs a specific interleaving *plus* the allocator returning a recycled address, so it may not appear in millions of iterations and then appears in production. Neither ThreadSanitizer nor ASan reliably catches it, because at the moment of the CAS there is no data race — every access is atomic. Symptom: a rare corrupted stack with no reproducer. If you need a lock-free stack, use a library that implements reclamation; do not ship the six-line version.

Treiber stack – push/pop lock-free  $O(1)$  amortised, unbounded retries under contention · ABA requires tagged pointers, hazard pointers or epochs · TSan and ASan do not catch it

## 70. `alloca`, VLAs, and stack smashing

Two features let you allocate on the call stack at runtime, and one classic attack exploits the fact that the return address lives there. Both topics are about the same physical memory from Topic 22.

### `alloca` and variable-length arrays

```
void handle(std::size_t n) {
 char* buf = (char*)alloca(n); // n bytes carved from THIS frame
 // int vla[n]; // a C99 VLA -- same mechanism, not in C++
 ...
}
```

// freed automatically by the epilogue

Both simply subtract from `rsp`. There is no allocator, no free, and — critically — **no failure signal**. If `n` is large, `rsp` jumps past the guard page and the process dies. If `n` is attacker-controlled, `rsp` can be moved to point at other mapped memory entirely, which is a “stack clash” and is exploitable. This is why C++ never adopted VLAs, why the Linux kernel removed essentially all of them in 2018, and why `alloca` should be treated as forbidden in any function that touches untrusted input.



## Stack smashing and the canary

frame, high addresses at the top

```
+-----+
| RETURN ADDRESS | <-- the target
+-----+
| saved rbp |
+-----+
| STACK CANARY | <-- -fstack-protector inserts this
+-----+ random per-process value, checked
| char buf[64] | in the epilogue before RET
+-----+
| | rsp
+-----+
```

strcpy(buf, attacker\_input) with input longer than 64 bytes writes UPWARD through the canary, saved rbp, and into the return address. On RET, the CPU jumps wherever the attacker wrote. Buffers grow up; the stack grows down; the return address is directly in the path.

The canary does not prevent the overflow -- it detects it before RET:  
\*\*\* stack smashing detected \*\*\*: terminated  
Aborted (core dumped)

### THE KEY INSIGHT

The call stack stores control-flow data (return addresses) in the same region as your local buffers, and buffers grow toward the return address. That single layout fact is the origin of the entire buffer-overflow attack class. Modern mitigations — canaries, ASLR, non-executable stacks, shadow stacks — all work around it; none removes it.

### THE TRAP

Reading **\*\*\* stack smashing detected \*\*\*** as “a stack overflow.” They are unrelated: **stack overflow** is running out of stack depth and gives a plain **SIGSEGV**; **stack smashing** is a local buffer being overrun and is reported by the canary check with that specific message. Confusing them sends you looking at recursion depth when the actual bug is a **strcpy** ten lines away.

alloca/VLA – O(1), no failure signal, forbidden on untrusted sizes · canary via **-fstack-protector-strong**, ~1–3% overhead · “smashing” = buffer overrun, “overflow” = depth exhaustion

## 71. Debugging: gdb backtrace and ASan output

Every failure mode in this book announces itself in one of three ways. Learning to read them turns an hour of guessing into a minute of diagnosis.

## A stack overflow in gdb

```
$ gdb ./a.out
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
0x000055555555149 in boom (d=174759) at boom.cpp:3
(gdb) bt
#0 0x000055555555149 in boom (d=174759) at boom.cpp:3
#1 0x000055555555155 in boom (d=174758) at boom.cpp:3
#2 0x000055555555155 in boom (d=174757) at boom.cpp:3
(More stack frames follow...)
(gdb) bt -5 # the OUTERMOST five -- where the recursion started
(gdb) info registers rsp
rsp 0x7fffffffde000 0x7fffffffde000
(gdb) info proc mappings | grep stack
0x7fffffffde000 0x7fffffff000 0x21000 [stack]
```

Three signatures identify it: a **monotonous backtrace** of identical frames, gdb's **(More stack frames follow...)** truncation, and **rsp** sitting at the exact **low boundary** of the **[stack]** mapping. That last check is conclusive — no other bug puts **rsp** there.

## ASan on a stack buffer overrun

```
==12345==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7ffd4a2c
WRITE of size 4 at 0x7ffd4a2c thread T0
#0 0x4f1a2b in IntStack::push(int) stack.cpp:14
#1 0x4f2c10 in main main.cpp:22
Address 0x7ffd4a2c is located in stack of thread T0 at offset 292 in frame
#0 0x4f19f0 in main main.cpp:18
This frame has 1 object(s):
[32, 288) 'st' (line 19) <== Memory access at offset 292 overflows this
```

Read it in this order: **the error class** (which memory region), **READ or WRITE and the size, frame #0** (the instruction), and then the **object description** at the bottom — which names the variable and shows the overflow by exactly four bytes past a 256-byte object. That last block is the part people skip and it is the most informative.

| Message                         | Actual cause                                                            |
|---------------------------------|-------------------------------------------------------------------------|
| stack-buffer-overflow           | Local array overrun (Topic 2)                                           |
| heap-use-after-free             | Reference held across a reallocation (Topic 16)                         |
| SIGSEGV, monotonous bt          | Recursion depth (Topics 28–29)                                          |
| *** stack smashing detected *** | Canary tripped (Topic 70)                                               |
| double free ... tcache 2        | Missing null in a move ctor (Topic 20), or a node self-cycle (Topic 11) |

### THE KEY INSIGHT

The five messages in that table map one-to-one onto five different chapters of this book. Diagnosis is not detective work; it is table lookup. Learn the mapping and the debugging time for every bug in Parts 2 to 4 collapses.

### THE TRAP

Debugging a stack overflow without `ulimit -c unlimited` and without symbols. With no core dump you get one line of output and nothing to inspect; with `-O2` and no `-g` the backtrace is addresses only, and inlining may have merged the recursive frames so the depth is not even visible. Build the reproduction with `-g -O0 -fno-omit-frame-pointer`, and remember that this changes the frame size and therefore the depth at which it fails (Topic 23).

diagnosis – monotonous bt + rsp at the mapping edge = depth · ASan object block names the variable and the overflow distance · five messages, five chapters

## 72. Stack machines and Python bytecode

A large fraction of the world's code executes on a stack machine. The JVM, CPython, the CLR and WebAssembly are all stack-based virtual machines: instructions take no register operands, they pop their inputs from an operand stack and push their results. This is Part 5's postfix evaluator, promoted to a hardware abstraction.

```
>>> import dis
>>> dis.dis(lambda a, b: a + b * 2)
 1 0 LOAD_FAST 0 (a)
 2 LOAD_FAST 1 (b)
 4 LOAD_CONST 1 (2)
 6 BINARY_MULTIPLY
 8 BINARY_ADD
 10 RETURN_VALUE
```

The operand stack during `a + b * 2` with `a=3, b=4`

| instruction     | operand stack (bottom -> top) |
|-----------------|-------------------------------|
| LOAD_FAST a     | [3]                           |
| LOAD_FAST b     | [3, 4]                        |
| LOAD_CONST 2    | [3, 4, 2]                     |
| BINARY_MULTIPLY | [3, 8] pops 2 and 4, pushes 8 |
| BINARY_ADD      | [11] pops 8 and 3, pushes 11  |
| RETURN_VALUE    | [] pops the result            |

Compare with the postfix trace in Topic 31. It is the SAME MACHINE.  
The bytecode `a b 2 * +` IS postfix notation.

Why stack machines: no register allocation, compact encoding (most instructions need no operand at all), and trivial portability.  
Why register machines are faster: fewer memory touches per operation, which is why the JVM JIT compiles bytecode to register code at runtime.

CPython pre-computes each function's maximum operand-stack depth at compile time and stores it as `co_stacksize`, so the frame can be allocated in one go. You can read it directly:

```
>>> (lambda a, b: a + b * 2).__code__.co_stacksize
3
```

#### THE KEY INSIGHT

Compiling an expression to stack-machine bytecode is exactly the infix-to-postfix conversion of Topic 33, and executing it is exactly the postfix evaluation of Topic 31. The compiler in your interpreter is running shunting-yard. That is why Part 5 is not an antique exercise — it is the code generation step of every stack-based VM.

#### THE TRAP

Confusing the *operand stack* with the *call stack*. Each Python frame has its own operand stack of size `co_stacksize`, typically 3 to 10 slots, and it is empty between statements; the call stack holds the frames themselves and is what `RecursionError` is about. A deeply nested *expression* grows `co_stacksize`, not the recursion depth — and CPython's compiler will itself hit a recursion limit parsing a sufficiently nested expression, which is the same Topic 43 vulnerability one level down.

stack machines – JVM, CPython, CLR, WebAssembly · bytecode is postfix · `co_stacksize` is precomputed, typically 3–10 · register VMs are faster, stack VMs are smaller and more portable

## 73. The strongest case against stacks

The book ends where Topic 6 began, with the argument on the other side. If you finish a data-structures handbook more likely to reach for its subject, the book has made you worse at your job.

### 1. The interface is almost always narrower than the problem

Real code needs to inspect, log, serialise, iterate for diagnostics, or bound the history. `std::stack` supports none of it. In practice most “stacks” in production codebases are `std::vector` used with discipline, and that is the better choice: the same performance, plus the ability to iterate when you need to and to `reserve` up front. Reach for the adaptor only when the narrowing genuinely prevents a class of bug.

### 2. LIFO is the wrong service order for almost every system

Anything with users, requests or jobs wants FIFO, because LIFO makes tail latency proportional to burst size (Topic 6's scenario). LIFO is right for *nested* work — parsing, traversal, undo — and wrong for *queued* work. Systems that got this wrong do not fail tests; they fail their p99.

### 3. The best solution is frequently not a stack at all

| Problem                                | Better than a stack                                    |
|----------------------------------------|--------------------------------------------------------|
| Trapping rain water (Topic 52)         | Two pointers — $O(1)$ space instead of $O(n)$          |
| Min additions to make valid (Topic 40) | Two counters — $O(1)$ space                            |
| Balanced brackets, one type (Topic 38) | One counter — $O(1)$ space                             |
| Sorting a stack (Topic 61)             | <code>std::sort</code> — $O(n \log n)$ versus $O(n^2)$ |
| Sliding window maximum (Topic 55)      | Monotonic deque — a stack is simply wrong              |
| Bounded undo (Topic 60)                | Deque — a stack cannot drop its oldest entry           |
| Longest valid parentheses (Topic 39)   | Two counter passes — $O(1)$ space                      |

Seven of the seventy-three topics in this book have a better non-stack solution, and one more (Topic 61) exists only to satisfy an artificial constraint. That is roughly one in nine.

#### 4. Recursion is usually clearer, until depth is adversarial

The explicit-stack conversion of Topic 25 costs readability and, per Topic 29's measurements, is often *slower* than recursion because the CPU's return-address predictor makes **ret** nearly free while your stage-field branch mispredicts. Convert for depth safety on unbounded or attacker-controlled input. Do not convert because iterative code sounds more professional.

##### THE KEY INSIGHT

The stack's value is not speed — a vector is as fast and more capable. It is that **the restricted interface makes a class of error unrepresentable**, and that LIFO is the exact shape of nested resolution. When neither of those applies, something else is better. The test remains the one from Topic 6: can you say “the most recently added item is the one that must be resolved next” about your problem, in plain language, without straining?

##### THE TRAP

Finishing a book like this one and pattern-matching on structure rather than on problem shape — seeing brackets and reaching for a stack, seeing “maximum” and reaching for a monotonic stack. The recognition procedure of Topic 55 exists precisely to be run *before* the reflex. Its first step is a question about the problem, not about the answer.

the case against — ~1 in 9 topics here has a better non-stack answer · vector matches the speed and adds capability · LIFO is wrong for queued work, right for nested work · convert recursion for depth, not for style

## PRACTICE SET — Topics 67–73

---

### A. Conceptual

1. What is the default backing container of `std::stack`, and name three properties that choice buys.
2. Give the two situations in which you should keep the deque default, and the one in which you should switch to `std::vector`.
3. Why can a `std::stack` not be given a `reserve`, and what is the standard-blessed way to get one?
4. Why does an empty `std::deque` in libstdc++ cost memory when an empty `std::vector` does not?
5. What is the growth factor of a CPython list, and what trade-off does that choice make relative to doubling?
6. Why is `java.util.Stack` discouraged by its own documentation? Give two distinct reasons.
7. What silently changes if you migrate Java code from `Stack` to `ArrayDeque` without reading the iteration semantics?
8. Rank C++, Python, Java, PHP and JavaScript by how dangerous their empty-pop behaviour is, and justify the extreme ends.
9. Explain the ABA problem without referring to stacks.
10. Why do neither ThreadSanitizer nor AddressSanitizer reliably detect ABA?
11. Name the three standard reclamation fixes for ABA and give one cost of each.
12. Why do `alloca` and VLAs have no failure signal, and what makes that dangerous with attacker-controlled sizes?
13. Explain in one sentence why buffer overflows can hijack control flow, in terms of the frame layout of Topic 22.
14. Distinguish “stack smashing detected” from a stack overflow. What does each actually indicate?
15. Give the three signatures that identify a stack overflow in gdb, and say which one is conclusive.
16. In what order should you read an ASan report, and which section do people skip?
17. What is the relationship between CPython bytecode and postfix notation? What is the compiler doing that Part 5 described?
18. Distinguish the operand stack from the call stack in CPython. Which one does `RecursionError` concern, and which does `co_stacksize` describe?
19. Give three of the seven problems in this book with a better non-stack solution, with the better structure and the improvement in each case.
20. State the positive test for a stack problem, and the one property that justifies choosing `std::stack` over `std::vector` in production.

### B. Tracing and analysis

1. A `std::stack<int>` holds 1,000,000 ints. Compute the peak allocated bytes under deque backing (512-byte blocks plus an 8-byte pointer per block) and under vector backing immediately after a doubling from 524,288. Give the ratio.
2. Using the Topic 67 table, compute the wall-clock difference between deque and vector backing for 100,000,000 push+pop pairs, taking the midpoint of each range.

3. A Python list is built by appending 1,000,000 elements. Given a growth factor of roughly 1.125, estimate the number of reallocations, and compare with the count under doubling.
4. Draining a 100,000-element Python list with `pop(0)` in a loop: compute the total element moves and estimate the runtime at 2 ns per move. Compare with `deque.popleft()`.
5. Write out the full ABA interleaving for a stack `A -> B -> C` with two threads, listing the stack state and each thread's held variables at every step. Mark the exact instruction at which the invariant breaks.
6. A tagged pointer packs a counter into the top 16 bits of a 64-bit pointer. Compute how many push/pop cycles are needed to wrap the counter, and state whether that is a practical concern at  $10^7$  operations per second.
7. A function contains `char buf[64]` and is compiled with `-fstack-protector-strong`. Draw the frame layout and give the number of bytes an attacker must write past `buf` to reach the return address, assuming an 8-byte canary and 8-byte saved `rbp`.
8. Given this ASan fragment — `stack-buffer-overflow, WRITE of size 4, object [32, 288) 'st', access at offset 292` — state the object's size in bytes, the number of `int` elements it holds, and the index that was written.
9. Disassemble `a * b + c / d` mentally into CPython bytecode. List the instructions in order and give the operand stack after each, plus the resulting `co_stacksize`.
10. For each of the seven problems in the Topic 73 table, state the stack solution's complexity and the better solution's complexity, in both time and space.

### C. Code tasks

1. **Build and measure.** Benchmark `std::stack<int>` with deque backing, with vector backing, and with vector backing plus a `reserve` via the derived-class trick, for 10,000,000 push+pop pairs. Report ns per operation for all three and peak RSS for each.
2. **Break it — the backing swap.** Write code that holds `int& r = s.top();` across 100,000 pushes and writes through it. Run under deque backing and vector backing with `-fsanitize=address`. Report which is flagged and the exact error class. Then state the review rule you would adopt.
3. **Measure — the Python trap.** Time draining a 100,000-element list with `pop(0)` versus `collections.deque.popleft()`. Report both times and the ratio. Then repeat at 1,000,000 elements and report the ratio again, explaining why it changes.
4. **Observe — Java iteration order.** Push 1, 2, 3 into a `java.util.Stack` and into an `ArrayDeque`, then iterate each with a for-each loop and print. Report both sequences and state what would break in a migration.
5. **Break it — ABA.** Implement the Treiber stack. Write a harness with two threads that deliberately forces the interleaving: use a small custom allocator with a free list that always returns the most recently freed block. Run until you observe a corrupted stack or a use-after-free under ASan. Report the iteration count needed, then run the same harness under ThreadSanitizer and report whether it fires.
6. **Break it — stack smashing.** Write a function with `char buf[64]` and `strcpy` from `argv`. Feed it 60, 70 and 200 bytes. Compile with `-fno-stack-protector` and with `-fstack-protector-strong`, and report the four outcomes. Then run the 200-byte case under ASan and compare the diagnostic quality.
7. **Observe — the debugger.** Reproduce a stack overflow and capture: the gdb `bt` truncation message, `rsp`, and the `[stack]` mapping bounds from `info proc mappings`. Confirm `rsp` is within one page of the low bound and report the difference in bytes.

8. **Explore — bytecode.** Use `dis.dis` on three expressions of increasing nesting depth. Report the instruction sequences and `co_stacksize` for each, and confirm that the bytecode is the postfix form. Then find the nesting depth at which CPython's own compiler fails, and report the exception.



## ANSWER KEY — Topics 67-73

---

### A. Conceptual

1. `std::deque`. It buys: **O(1) worst-case push** (no reallocation), **references from `top()` survive pushes**, and **peak memory never doubles**.
2. Keep deque when you **hold references across pushes** or need a **hard bound on a single operation**. Switch to vector when you **know the approximate size and want throughput**.
3. Because `reserve` is not part of the adaptor's interface and `std::deque` has no such member. The standard names the underlying container `c` as a **protected** member, so **deriving from `std::stack`** and exposing `c.reserve(n)` is legitimate.
4. Because `libstdc++`'s deque **allocates its first 512-byte block eagerly** on construction, whereas an empty vector holds a null pointer and allocates nothing.
5. Roughly **1.125×** plus a constant. It **reallocates far more often** than doubling but **wastes far less memory** — the right trade for an interpreted language where the reallocation cost is small relative to interpretation overhead.
6. (i) It **extends `Vector`**, so every method is **synchronised**, costing performance in single-threaded code. (ii) It inherits `get(int)`, so it is indexable and therefore not a stack at all — Topic 1's trap in the standard library.
7. **Iteration direction**. `Stack` iterates **bottom to top**; `ArrayDeque` iterates **top to bottom**. Any for-each loop over the container silently reverses.
8. Most dangerous: **JavaScript** — `pop()` returns `undefined` silently, which propagates through arithmetic as `NaN` rather than failing. Least dangerous in practice: **Python and Java**, which throw immediately and name the problem. C++ is undefined behaviour, which is the most severe in principle but at least is caught by sanitisers and debug modes; PHP warns.
9. A compare-and-swap succeeds when the observed value matches the expected value, but a value that changed and changed back is indistinguishable from one that never changed. **CAS compares bits, not identity**, so it cannot detect that the world behind the value was replaced in between.
10. Because **there is no data race** — every access is atomic and correctly ordered, so TSan sees nothing wrong. ASan may catch the resulting use-after-free, but only if the specific interleaving *and* an address reuse both occur during the run, which may take millions of iterations or never happen.
11. **Tagged pointers** — needs a double-width CAS (`cmpxchg16b`) and the counter can still wrap. **Hazard pointers** — a read barrier and a published-pointer scan on every access. **Epoch-based reclamation** — memory is freed late, so peak usage rises.
12. Because both simply **subtract from `rsp`**; there is no allocator involved and therefore nothing to return an error. With an attacker-controlled size, `rsp` can be moved **past the guard page onto other mapped memory** — a “stack clash” — which is exploitable rather than merely fatal.
13. Because the **return address is stored in the same frame as local buffers, at a higher address**, and buffers are written upward — so an overrun writes directly through the saved registers into the return address, and `ret` then jumps wherever the attacker wrote.
14. **Stack smashing** = **a local buffer was overrun**, detected by the canary check before `ret`. **Stack overflow** = **depth exhaustion**, a plain `SIGSEGV` at the guard page. Unrelated causes, similar names.

15. (i) A **monotonous backtrace** of identical frames. (ii) gdb's **(More stack frames follow...)** truncation. (iii) **rsp at the low boundary of the [stack] mapping** — this one is **conclusive**, since no other bug puts it there.
16. Error class → READ/WRITE and size → frame #0 → **the object description block**. People skip the last one, and it is the most informative — it names the variable and shows the overflow distance.
17. CPython bytecode **is postfix notation**: operands are loaded onto an operand stack and operators pop them. The compiler is performing the **infix-to-postfix conversion of shunting-yard** (Topic 33), and the interpreter loop is the postfix evaluator of Topic 31.
18. The **operand stack** is per-frame, holds intermediate expression values, is typically 3-10 slots, and is empty between statements — its size is `co_stacksize`. The **call stack** holds the frames themselves and is what **RecursionError** concerns.
19. Any three of: trapping rain water → two pointers,  $O(n)$  space to  **$O(1)$** ; min additions to make valid → two counters,  **$O(1)$**  space; balanced brackets with one type → one counter,  **$O(1)$**  space; sorting a stack → `std::sort`,  $O(n^2)$  to  **$O(n \log n)$** ; sliding window maximum → monotonic deque (a stack is simply wrong); bounded undo → deque (a stack cannot drop its oldest); longest valid parentheses → two counter passes,  **$O(1)$**  space.
20. Test: **“the most recently added item is the one that must be resolved next.”** The justification for `std::stack` over `std::vector` is that **the restricted interface makes a class of error unrepresentable** — not performance, which is identical or better with a vector.

## B. Tracing and analysis

1. Deque: 1,000,000 ints at 128 ints per 512-byte block = **7813 blocks**  $\times$  512 = **4,000,256 B**, plus  $7813 \times 8 = 62,504$  B of block index  $\approx$  **4.06 MB**. Vector after doubling to 1,048,576: **4,194,304 B  $\approx$  4.19 MB**. Ratio **1.03 $\times$**  — nearly identical here; the vector's  $2\times$  peak only appears transiently *during* the copy, when both buffers are live, giving a true peak of  **$\sim 6.3$  MB**.
2. Deque midpoint 3 ns, vector midpoint 1.15 ns. Difference  $1.85 \text{ ns} \times 10^8 =$  **0.185 s**.
3. Growth by  $1.125\times$ : reallocations  $\approx \log(10^6)/\log(1.125) = 13.8/0.1178 \approx$  **117**. Under doubling: **20**. About **6 $\times$  more reallocations** for substantially less wasted memory.
4. `pop(0)` moves  $n-1$ , then  $n-2$ , ... =  $n^2/2 =$   **$5 \times 10^9$  element moves**, at 2 ns each  $\approx$  **10 s**. `deque.popleft()` is  $O(1)$  per call: **100,000 operations**, well under a millisecond. Ratio in the **tens of thousands**.

5.

| step                                              | stack   | T1 holds      | T2 action                             |
|---------------------------------------------------|---------|---------------|---------------------------------------|
| 1                                                 | A->B->C | old=A, next=B | (T1 descheduled here)                 |
| 2                                                 | B->C    | old=A, next=B | T2 pops A, DELETES A                  |
| 3                                                 | C       | old=A, next=B | T2 pops B, DELETES B                  |
| 4                                                 | A'->C   | old=A, next=B | T2 pushes X, allocated at A's address |
| 5                                                 | B->...  | --            | T1 CAS(expect A, desire B) SUCCEEDS   |
| ^^^ INVARIANT BREAKS HERE                         |         |               |                                       |
| head_ now points at B, which was freed at step 3. |         |               |                                       |

6. A 16-bit counter wraps after **65,536** increments. At  $10^7$  operations per second that is every **6.6 ms** — **emphatically a practical concern**. Sixteen bits is far too few; a double-width CAS with a full 64-bit counter is the usable form.
7. Layout from low to high: `buf[64]`, canary (8), saved `rbp` (8), return address (8). An attacker must write  **$64 + 8 + 8 = 80$  bytes** past the start of `buf` to *reach* the return address, i.e. **16 bytes past the end of `buf`**, and 88 bytes total to overwrite it fully.

8. Object spans [32, 288), so its size is **256 bytes = 64 ints**. Access at offset 292 is 4 bytes past the end, so the index written was **64** — one past the last valid index 63. This is exactly the Topic 2 `buf_[++n]` trap.

9.

```
LOAD_FAST a [a]
LOAD_FAST b [a, b]
BINARY_MULTIPLY [ab]
LOAD_FAST c [ab, c]
LOAD_FAST d [ab, c, d]
BINARY_TRUE_DIVIDE [ab, c/d]
BINARY_ADD [ab + c/d]
RETURN_VALUE []
co_stacksize = 3 (maximum simultaneous depth, reached before the divide)
```

10. Rain water: stack  $O(n)/O(n) \rightarrow$  two pointers  $O(n)/O(1)$ . Min additions:  $O(n)/O(n) \rightarrow$  counters  $O(n)/O(1)$ . Balanced brackets one type:  $O(n)/O(n) \rightarrow$  counter  $O(n)/O(1)$ . Sorting a stack:  $O(n^2)/O(n) \rightarrow \text{std::sort } O(n \log n)/O(\log n)$ . Sliding window max: stack is **incorrect**  $\rightarrow$  deque  $O(n)/O(k)$ . Bounded undo: stack **cannot express it**  $\rightarrow$  deque  $O(1)/O(\text{limit})$ . Longest valid parens:  $O(n)/O(n) \rightarrow$  two counter passes  $O(n)/O(1)$ .

### C. Code tasks — what to verify

- Expect deque **2-4 ns/op**, vector **1.5-2.5 ns/op**, vector with reserve **0.8-1.5 ns/op**. Peak RSS should be roughly **4 MB**, **~6.3 MB** (transient double during the final copy) and **4 MB** respectively — reserve removes both the pause and the transient peak.
- Deque backing: **no error**. Vector backing: **heap-use-after-free**, **WRITE of size 4**, with the freeing frame inside `push_back`. Review rule: **never hold a reference returned by `top()` across any mutation, regardless of backing** — and audit for held references before changing a backing container.
- At 100,000: expect roughly **1-3 s** for `pop(0)` against **under 10 ms** for `popleft()`. At 1,000,000 the `pop(0)` time grows by about **100×** (quadratic) while `popleft()` grows by **10×** (linear), so the ratio itself grows by roughly **10×**. Watching the ratio change is the demonstration that one is  $O(n^2)$  and the other  $O(n)$ .
- `Stack` prints **1 2 3**; `ArrayDeque` prints **3 2 1**. A migration that iterates the container — for logging, serialisation, or equality — **silently reverses**, with no compile error and no exception.
- With a most-recently-freed-first allocator the interleaving becomes reachable in **thousands to low millions** of iterations rather than never; without it you may run indefinitely without observing it. ASan should eventually report **heap-use-after-free**. ThreadSanitizer should report **nothing** — there is no data race. That silence is the entire point of the exercise.
- 60 bytes: **fits, no error, both builds**. 70 bytes: overruns `buf` into the canary — `-fno-stack-protector` gives **silent corruption or nothing visible**, while `-fstack-protector-strong` gives **\*\*\* stack smashing detected \*\*\***. 200 bytes: without the protector, likely a **SIGSEGV at an arbitrary address** after `ret` jumps into garbage; with it, the same canary message. Under ASan you get **stack-buffer-overflow with the exact variable and offset** — far more actionable than the canary's abort, which tells you only that *something* overflowed.
- `rsp` should be within **one page (4096 bytes)** of the low bound of the `[stack]` mapping. That page is the guard page. If the difference is much larger, you have not actually overflowed — look for a different cause.
- `co_stacksize` should rise with expression nesting, typically **2, 3, 4** for successively nested binary expressions. CPython's own parser fails at roughly **100 nested parentheses in one expression** (the exact figure varies by version), raising **SyntaxError** or **MemoryError**, or in

older versions `s_push: parser stack overflow`. That is Topic 43's vulnerability appearing inside the language's own compiler.

# HANDNOTE SUMMARY CARD

## PART 10 - LIBRARY, SAFETY, INTERVIEW

Topics 67-73

```
std::stack IS AN ADAPTOR. DEFAULT BACKING IS std::deque, NOT vector.
=====
ns per push+pop deque vector (empty deque = 512 B)
worst-case single push 0(1) 0.8-1.5
top() survives a push YES 0(n)
reserve() no yes (derive; member is `c`)
KEEP DEQUE if refs are held across pushes or a per-op bound is needed;
USE VECTOR if the size is roughly known and throughput matters.
swapping backing for speed while holding a top() ref = UAF firing only at
reallocation boundaries, weeks after a "performance" commit
```

### OTHER LANGUAGES - all array-backed; they differ in ERROR STRATEGY (T5)

```
Python list: append / [-1] / pop() IndexError. growth ~1.125x.
 list.pop(0) is 0(n) -- 100k drain = 5e9 moves ~ 10 s. Use deque.
Java ArrayDeque, NOT java.util.Stack (extends Vector: synchronized +
 get(i) = not a stack). Stack iterates BOTTOM-UP, ArrayDeque TOP-
 DOWN -- migration silently reverses any for-each.
JS arr.push / arr[arr.length-1] / arr.pop() returns undefined SILENTLY,
 propagating as NaN instead of failing. Most dangerous of the five.
```

### TREIBER STACK (lock-free) + ABA

```
push: n->next = head; while (!CAS(head, n->next, n)) {} pop: symmetric
ABA: T1 reads head=A,next=B and stalls. T2 pops A, pops B, pushes X which
 the allocator places AT A'S ADDRESS. T1's CAS(expect A) SUCCEEDS,
 installing the already-freed B. CAS COMPARES BITS, NOT IDENTITY.
TSan sees NO data race; ASan needs the interleaving AND address reuse.
fixes: tagged pointer (cmpxchg16b; a 16-bit tag wraps in 6.6 ms at 1e7
ops/s), hazard pointers, epoch reclamation. Use a library.
```

alloca / VLA - just "sub rsp". NO failure signal. Forbidden on untrusted n:  
the size moves rsp PAST the guard page = stack clash. C++ never adopted  
VLAs; the kernel removed them in 2018.

### STACK SMASHING != STACK OVERFLOW

```
smashing = local buffer overrun; the canary trips before RET, printing
*** stack smashing detected *** (-fstack-protector-strong).
[buf][canary 8][saved rbp 8][RET ADDR]: buffers grow UP, so the
return address sits directly in the path.
overflow = depth exhaustion, plain SIGSEGV at the guard page
```

### DIAGNOSIS - five messages, five chapters

```
stack-buffer-overflow local array overrun T2
heap-use-after-free ref held across a realloc T16
SIGSEGV + monotonous bt recursion depth T28-29
*** stack smashing detected canary T70
double free ... tcache 2 move ctor / node self-cycle T20, T11
gdb: monotonous bt + "(More stack frames follow...)" + rsp within ONE
 PAGE of [stack]'s low bound in `info proc mappings` <- CONCLUSIVE
ASan order: class -> RW+size -> frame #0 -> OBJECT BLOCK (names it)
```

STACK MACHINES: JVM, CPython, CLR, WebAssembly. BYTECODE IS POSTFIX.  
dis.dis(lambda a,b: a+b\*2) -> LOAD a, LOAD b, LOAD 2, MULTIPLY, ADD.  
The compiler runs SHUNTING-YARD (T33); the eval loop is T31.  
co\_stacksize = per-frame OPERAND depth (3-10); != the CALL stack.

### THE CASE AGAINST - ~1 in 9 topics here has a better non-stack answer

```
rain water -> two pointers 0(1) min-add -> counters 0(1)
brackets(1 type) -> counter 0(1) sort -> std::sort 0(n log n)
window max -> monotonic DEQUE bounded undo -> deque
longest valid parens -> two counter passes, 0(1)
a vector matches the speed AND adds capability. LIFO is wrong for QUEUED
work, right for NESTED work. Convert recursion for DEPTH, not style. THE
ONLY JUSTIFICATION: the narrow interface makes an error class unrepres-
entable. TEST: "the most recently added item is the one that must be
resolved next" -- in plain language, without straining.
```



END MATTER

# Reference

---

The whole book compressed: a master cheatsheet in three cards, a table mapping problem phrasings to techniques, and a spaced-repetition tracker to fill in by hand.

# MASTER CHEATSHEET — THE CORE

EVERYTHING ABOUT STACKS - MASTER CARD 1 of 3

THE CORE

## THE CONTRACT

```
push(v) n -> n+1, top == v
pop() n -> n-1 [n >= 1]
top() returns xn [n >= 1]
empty(), size() total
```

## ONLY TWO ERROR STATES:

```
underflow (pop/top on empty)
overflow (push past capacity)
```

## THE LAWS

```
top(push(s,v)) == v
pop(push(s,v)) == s <- the undo law
```

## THE POSITIVE TEST (run it before coding)

```
"the most recently added item is the
one that must be resolved next"
if you cannot say that in plain
language, it is NOT a stack problem.
```

WHEN IT IS THE WRONG STRUCTURE (~1 topic in 9 here has a better answer)

```
need the oldest -> queue (stack->queue turns DFS into BFS)
need min/max/k-th -> heap
need element k -> vector (you narrowed the interface by accident)
need both ends -> deque (sliding window max, work stealing)
need bounded history -> deque (std::stack cannot drop its bottom)
arbitrary range query -> segment tree
a vector matches the speed AND adds capability. The ONLY justification for
std::stack is that the narrow interface makes an error class unrepresentable.
```

## THE INVARIANT (assert all four)

```
n <= cap data[0..n-1] live data[n..cap-1] raw, reading is UB
cap == 0 <=> data == nullptr
n MEANS THREE THINGS: live count | next-write index | one-past-the-top
every off-by-one is a confusion between two of them. top() = data[n-1]
store a COUNT, never a top POINTER (realloc dangles; empty needs data-1)
```

## THE FOUR TEMPLATES WORTH MEMORISING

1. push, in this order and no other

```
if (n == cap) grow(); // may throw -> stack unchanged
new (data + n) T(v); // may throw -> n not yet bumped
++n; // CANNOT throw -> COMMIT POINT
```
2. monotonic stack skeleton

```
for (i = 0; i < n; ++i) {
 while (!st.empty() && COMPARE(a[st.top()], a[i])) {
 j = st.top(); st.pop(); RESOLVE(j, i);
 }
 st.push(i); // ALWAYS INDICES
} // then handle whatever is left
```
3. shunting-yard pop condition

```
while (top != "(" && (prec(top) > prec(t) ||
 (prec(top) == prec(t) && leftAssoc(t)))) emit(top);
```
4. recursion -> explicit stack

```
frame struct = every var still live AFTER the recursive call
"call myself" -> push + continue; "return v" -> pop + deliver to top
2+ call sites -> a STAGE field (= the return address as data)
```

## GROWTH AND AMORTISATION

```
factor k > 1 gives amortised O(1). Constant +c gives n^2/(2c) copies.
2x: 20 reallots/1M, total copies < n, peak 2n, CANNOT reuse freed blocks
1.5x: 35 reallots/1M, ~2n copies, peak 1.5n, CAN reuse after ~4 steps
accounting proof: charge 1 + k/(k-1) per push (3 at 2x). Half the copied
elements were pushed since the last growth, each banking 2 -> exact.
amortised != average. An adversary CAN defeat average, CANNOT defeat this.
single push worst case is STILL O(n). reserve() makes it worst-case O(1).
shrink: trigger n < cap/4, target cap/2. Equal triggers -> O(n) per op.
std::vector NEVER shrinks. Release: std::vector().swap(v);
```

## THE TWO STACKS

```
ADT heap, limit = RAM, you manage it
call stack 8 MB (ulimit -s 8192), grows DOWN, rsp, guard page, FATAL
max depth = stack_bytes / frame_bytes. ~200k frames at -O2, 8 MB
 8 MB/48 B = 174,762 8 MB/176 B = 47,662 (a JSON parser)
 one char buf[4096] cuts it 86x. -O0 frames are 2-4x -O2.
converting recursion to a heap stack raises the ceiling ~1000x.
```



# MASTER CHEATSHEET — THE ALGORITHMS

EVERYTHING ABOUT STACKS - MASTER CARD 2 of 3 THE ALGORITHMS

=====

EXPRESSION PARSING (T30-37) the 3 notations are 3 walks of ONE tree  
prefix=preorder infix=inorder (AMBIGUOUS) postfix=postorder  
postfix eval L->R: b = pop(); a = pop(); push(op(a,b)); RIGHT first  
prefix eval R->L: a = pop(); b = pop(); LEFT first  
wrong order: + and \* still pass, - and / silently reverse  
end state: EXACTLY ONE value must remain  
"(" is a BARRIER, never compared, only removed by its ")" or the end check  
left-associative pops on equal precedence, right-associative does not  
8/4/2 = 1 (left) 2^3^2 = 512 (right) TEST BOTH  
unary minus is a TOKENISATION problem: rewrite to "~" (arity 1, prec 3,  
right-associative) in a pre-pass. "-" is unary iff first, or after an op or "("  
two-stack eval = shunting-yard where popping APPLIES instead of EMITS  
pipeline: lex -> markUnary -> evaluate. "100" as 1,0,0 = LEXER bug.

MATCHING AND NESTING (T38-43)

brackets: 3 failure modes -> empty on close / wrong type / left open at end  
forgetting "return st.empty()" is the #1 bug: "((((" passes  
ONE type -> a counter suffices, O(1). TWO+ -> stack ("([])")  
longest valid parens: push INDICES, sentinel -1, best = max(best,i-top())  
score of parens / decode string: PUSH A FRESH ACCUMULATOR ON OPEN,  
FOLD INTO THE PARENT ON CLOSE <- the shape of every nested evaluation  
simplify path: use a VECTOR (output read bottom-up). "." at root is a  
NO-OP - absorbing it is the CWE-22 path-traversal fix.  
decode string: num = num\*10 + (c-'0') or 12[a] gives "aa".  
output is EXPONENTIAL: 10[10[10[10[10[a]]]]] = 25 B -> 1,000,000  
deep-nesting DoS: input nesting depth == parser recursion depth.  
176 B frame, 8 MB -> a 95 KB payload kills the process. No exception.  
FIX = a depth limit (one counter). Raising ulimit is NOT a defence.

MONOTONIC STACK (T44-55) the stack holds the UNRESOLVED elements

O(n): each index pushed once, popped at most once.  
next greater pop while a[top] < a[i] write at POP  
next smaller pop while a[top] > a[i] write at POP  
previous greater pop while a[top] <= a[i] read BEFORE push  
previous smaller pop while a[top] >= a[i] read BEFORE push  
LEFTOVERS = the elements with no answer. Give them an explicit default.  
ties: contribution problems need ONE strict side and one non-strict.  
TEST [2,2], [3,3,3], [1,2,2,1] on every one of these problems.  
daily temps ans[j] = i - j default 0, NOT -1  
stock span span[i] = i - st.top() (or i+1) use <=, not <  
circular scan 2n, idx = i%n, PUSH ONLY IF i < n. Leftovers = maxima.  
histogram w = st.empty() ? i : i - st.top() - 1; append sentinel 0  
max rectangle h[c] = cell=='1' ? h[c]+1 : 0 then histogram per row  
rain water TWO POINTERS, O(1) space - the stack version is worse  
subarray mins contribution (j-L)\*(R-j)\*a[j]; ASSERT counts = n(n+1)/2  
remove k pop while k>0 && back() > c; then tail, zeros, "" -> "0"

AUGMENTED (T56-61) / INSIDE ALGORITHMS (T62-66)

min stack: RECORD THE ANSWER AT PUSH TIME (any associative fold).  
aux stack must push on <=, not <. encoded trick needs 2x int width.  
queue from 2 stacks: shift ONLY when out\_ is empty. 4 stack ops per  
element for life -> amortised O(1). Single pop worst case O(n).  
stack from 2 queues has NO amortisation - the asymmetry is the answer.  
undo/redo: redo.CLEAR() on every new action is CORRECTNESS, not UI.  
iterative DFS: push neighbours REVERSED to match recursion; seen-guard  
mandatory; stack -> queue converts DFS to BFS.  
inorder: while (cur || !st.empty()) - both terms, or it truncates.  
postorder reversed == mirrored preorder (root,RIGHT,LEFT), then reverse.  
quicksort: process the SMALLER partition first -> stack O(log n) not O(n).  
ASK OF ANY STACK: do its contents MEAN something, or is it just reversing?

# MASTER CHEATSHEET — TRAPS AND TESTS

EVERYTHING ABOUT STACKS - MASTER CARD 3 of 3 TRAPS + TESTS

THE UNIVERSAL TEST MATRIX - run these against ANY stack code you write

```
empty 0 elements: pop, top, size, and the answer for n=0
single 1 element: exercises the st.empty() branch of any formula
two equal [2,2] : the ONLY test that catches strict-vs-non-strict
all equal [3,3,3] : tie ownership in contribution problems
strictly incr worst case for previous-* and histogram stack depth
strictly decr worst case for next-* depth; leftovers never resolved
at capacity n == cap exactly: the growth boundary
alternating push/pop at the boundary: catches shrink thrash
degenerate tree a left spine: kills every recursive tree algorithm
adversarial 100k nested brackets: the DoS test nobody runs
```

THE TRAPS, IN THE ORDER THEY BITE

```
T2 buf[++n_] = v index 0 garbage; overrun reported 63 ops later
T5 sentinel that is legal data silent loss, no crash, passes review
T8 newcap = cap * 2 with cap == 0 null write on the FIRST push
T10 naive shrink at cap/2 alternating push/pop -> O(n) per op
T11 head_ = n; n->next = head_ self-cycle; INFINITE LOOP IN THE DTOR
T12 benchmark with no volatile sink -O2 deletes it; 0.1 ns/op is fake
T14 ++n before the placement new ~T() on garbage, LATER, in the dtor
T15 s.pop(); v = s.top(); returns the element BELOW. Most common.
T16 auto& x = s.top(); s.pop(); UAF under every backing; int tests pass
T17 i < s.size() - 1 0-1 = 1.8e19. UBSan reports NOTHING.
T18 clear() as n = 0 with a dtor leak shows as "still reachable"
T19 no push(T&&) overload every std::move silently COPIES
T20 move ctor without noexcept vector COPIES on growth, 5-30x slower
T20 move ctor without nulling src double free at scope exit
T21 -DNDEBUG deleted your asserts verify by shipping a false one
T23 frame measured at -O0, shipped at -O2 (or the reverse - worse)
T26 Frame& f = st.top() then push dangles on vector backing only
T27 a destructor-bearing local silently un-tails a tail call
T29 unique_ptr next member recursive dtor you never wrote
T31 pop order right-then-left + and * pass, - and / reverse
T33 no unmatched-paren checks UB reported inside the std library
T34 ^ left-associative 2^3^2 = 64 instead of 512
T37 single-character lexing 100/10 breaks, 3+4 passes
T38 return true instead of empty() "((((" passes
T39 no -1 sentinel every run measured from a wrong origin
T41 "... popping an empty vector size_t underflow AND CWE-22
T46 testing only distinct values hides EVERY tie bug in Part 7
T48 pushing during the 2nd pass wrong only on decreasing input
T50 width = i - top() (no -1) [5] still passes; [2,1,2] does not
T52 no "if empty break" in trapStack water where there is none
T56 aux min stack pushing on < reports a min larger than an element
T57 2*v - mn in int signed overflow = UB
T58 unconditional shift O(n) per op; all tests still pass
T60 no redo.clear() a state the user was never in
T62 no seen-guard on pop O(E) stack, over-counted visits
T64 no lastVisited infinite loop; left spine passes
T65 larger partition first only sorted input reveals it
T67 backing swapped with a live ref UAF weeks after a "perf" commit
T69 Treiber stack shipped as-is ABA; TSan sees no race
T70 alloca on an untrusted size stack clash, exploitable
```

FIVE MESSAGES, FIVE CHAPTERS

```
stack-buffer-overflow local array overrun T2
heap-use-after-free reference held across a realloc T16
SIGSEGV + monotonous bt recursion depth T28-29
*** stack smashing *** canary (a BUFFER overrun, not depth) T70
double free ... tcache 2 move ctor, or a node self-cycle T20, T11
```

## PATTERN RECOGNITION

---

Read the left column in the problem statement; the middle column is what to write. This table is the fastest path from a question to a first line of code, and it is worth reading straight through the night before an interview.

| The problem says...                                 | Technique                                               | Topic  |
|-----------------------------------------------------|---------------------------------------------------------|--------|
| "valid parentheses", "balanced", "matching tags"    | Push openers, match on close, check empty at end        | 38     |
| "one bracket type only"                             | A counter — $O(1)$ space, no stack                      | 38, 40 |
| "longest valid", any <i>length</i> of a matched run | Index stack with a $-1$ sentinel                        | 39     |
| "score", "decode", "nested repetition"              | Accumulator per level, fold into the parent on close    | 40, 42 |
| "simplify path", "canonical"                        | Vector of components; <code>..</code> pops if non-empty | 41     |
| "evaluate this expression"                          | Two-stack infix evaluation                              | 35, 37 |
| "convert to postfix", "RPN"                         | Shunting-yard                                           | 33     |
| "next greater / smaller element"                    | Monotonic stack, write at pop                           | 45     |
| "days until", "how long before", "distance to"      | Monotonic stack, write <code>i - j</code>               | 47     |
| "span", "consecutive preceding"                     | Monotonic stack, read before push                       | 49     |
| "circular", "wraps around"                          | Scan $2n$ , push only when <code>i &lt; n</code>        | 48     |
| "largest rectangle", "maximal area"                 | Histogram with a sentinel 0                             | 50, 51 |
| "sum over all subarrays of the min/max"             | Contribution: <code>(j-L) × (R-j)</code>                | 53     |
| "lexicographically smallest after removing k"       | Greedy stack, pop while <code>back() &gt; c</code>      | 54     |
| "trapped water", "container"                        | <b>Two pointers</b> — $O(1)$ space                      | 52     |
| "sliding window maximum"                            | <b>Monotonic deque</b> — not a stack                    | 55     |
| "get the minimum in $O(1)$ "                        | Record the answer at push time                          | 56, 57 |
| "implement a queue using stacks"                    | Two stacks, shift only when <code>out</code> is empty   | 58     |
| "undo", "history", "back button"                    | Two stacks; clear redo on a new action                  | 60     |
| "without recursion", "iteratively"                  | Four-step conversion; stage field if 2+ call sites      | 25, 26 |
| "traverse the tree iteratively"                     | Inorder walk-left loop; postorder = reversed preorder   | 63, 64 |
| "depth-first", "explore all paths"                  | Explicit stack, push neighbours reversed                | 62     |
| "detect a cycle", "strongly connected"              | Tarjan — the stack contents are load-bearing            | 66     |
| "the input may be deeply nested"                    | A depth limit, checked before recursing                 | 43     |
| "this must not pause", "real time"                  | Fixed capacity, or node-backed; never a growable array  | 13     |

## REVISION TRACKER

---

Fill this in by hand. The schedule that works is: learn a part, revise it the next day, do its practice set within three days, then revise at one week, one month and three months. The practice column is the one that matters — re-reading a part feels productive and is not. If a revision date arrives and you cannot reproduce that part's summary card from memory, that is the signal to redo the practice set rather than to read again.

| Part                          | First | Rev 1<br>+1d | Practice | Rev 2<br>+7d | Rev 3<br>+30d | Rev 4<br>+90d |
|-------------------------------|-------|--------------|----------|--------------|---------------|---------------|
| 1. Foundation (1-6)           |       |              |          |              |               |               |
| 2. Memory (7-13)              |       |              |          |              |               |               |
| 3. Core operations (14-21)    |       |              |          |              |               |               |
| 4. The call stack (22-29)     |       |              |          |              |               |               |
| 5. Expression parsing (30-37) |       |              |          |              |               |               |
| 6. Matching, nesting (38-43)  |       |              |          |              |               |               |
| 7. Monotonic stack (44-49)    |       |              |          |              |               |               |
| 7. Monotonic stack (50-55)    |       |              |          |              |               |               |
| 8. Augmented designs (56-61)  |       |              |          |              |               |               |
| 9. Inside algorithms (62-66)  |       |              |          |              |               |               |
| 10. Library, safety (67-73)   |       |              |          |              |               |               |
| Master cheatsheet             |       |              |          |              |               |               |

### Self-check before you call a part done

| Can you, from memory...                                 | Part | Done |
|---------------------------------------------------------|------|------|
| State the positive test and the four disqualifiers      | 1    |      |
| Prove amortised $O(1)$ push by the accounting method    | 2    |      |
| Write push in the correct order and say why             | 3    |      |
| Compute max recursion depth from two numbers            | 4    |      |
| Write shunting-yard including both paren checks         | 5    |      |
| Name the three bracket failure modes                    | 6    |      |
| Reproduce the four-way direction table                  | 7    |      |
| Write largest-rectangle including the width formula     | 7    |      |
| Explain why two queues cannot give an amortised stack   | 8    |      |
| Write iterative inorder with the correct loop condition | 9    |      |
| Explain ABA without mentioning stacks                   | 10   |      |
| Argue against using a stack, with three examples        | 10   |      |

# HOW TO USE THIS BOOK

---

## If you have three months

One part per week, in order. Read the topics on day one, do the practice set on day two or three — actually writing the code, actually breaking it, actually reading the sanitiser output — and revise from the summary card on day seven. The build-break-observe-fix tasks are not optional garnish; observing a failure is what installs the trap in memory, and reading about one does not.

## If you have a week

Read Part 1 and Topic 6 first, then Part 7 in full, then the practice sets for Parts 5, 6 and 7. Parts 2 to 4 can be skimmed for their traps. Part 7 is the densest interview material in the subject and Topic 55 is the single highest-value page for a screen tomorrow.

## If you have one evening

The three master cards and the pattern-recognition table. Then write, from memory and without looking, the monotonic-stack skeleton and the histogram width formula. If both come out right, you are in reasonable shape.

## What to carry away

Three things outlast the specific algorithms. First, **a restricted interface is a promise**, and the promise is what buys the guarantees — not the data layout. Second, **amortised is a worst-case statement about sequences**, which is why `reserve` and shrink hysteresis matter and why an adversary cannot defeat them. Third, and most useful in practice: **ask what each element is waiting for**. That question writes the monotonic-stack comparison, chooses between pop-time and push-time resolution, and tells you when the answer is not a stack at all.

The last of those is also the honest summary of Topic 73. This book argues for its subject for seventy-two topics and against it in the seventy-third, and the seventy-third is the one that will save you the most time. A stack is the right tool when the most recently added item is the one that must be resolved next — and a `std::vector` is the right tool a good deal more often than a data-structures handbook would suggest.

---

Volume II of Core Computer Science. Volume I covered linked lists, and the recursive-destructor crash of Topic 29 is the seam between them — a bug that belongs equally to both structures and to neither. Code in this volume targets C++17 and was written against libstdc++ on x86-64 Linux; the measured figures are from that platform and should be treated as orders of magnitude rather than constants.

